

ARCHITECTURE OBSERVATION NOTES

架构观察笔记

从 Redis、MySQL、Kafka 说起

2026年9月修订版

序言

十年前我就想写这本书，一直拖到现在。当时已经用了不少年的 Redis、MySQL、Kafka，也觉得它们的设计有相通之处，却说不清这些相通之处究竟能帮助我做什么。后来做系统、处理故障、重新审视自己的决定，才逐渐有了一些可以写下来的观察。

最初整理笔记时，我按软件分别写。Redis 的持久化写在 Redis 里，Kafka 的日志写在 Kafka 里，各自看都能说清，放在一起却看不出它们处理了哪些相同的问题。后来改成按主题组织，内存、文件、复制、恢复这些事情才有了可以直接比较的位置。这本书保留了这种写法。

我想追问的事情并不复杂。一个决定解决了什么问题？它依赖哪些条件？如果换一种实现，会失去什么，又会得到什么？这些问题常常在软件已经选定之后才出现。例如，低延迟满足了，但容量增长以后怎么办；主节点可以自动切换，但先前返回成功的写是否还在；查询可以从缓存返回，但缓存里的状态凭什么算数。

只知道功能存在，无法回答这些问题。知道实现名称也不够。“用了日志”“有三个副本”“支持事务”都还只是论证的开始。日志要保存哪些内容，副本何时确认，事务包括哪些状态，才决定这些功能能为业务承担什么责任。

所以书里记录三层内容。首先是具体机制。Redis 怎样表示一组数据，InnoDB 怎样组织页，Kafka 怎样追加和读取一段日志，都需要讲到操作过程。省去这些细节，后面的取舍就没有依据。其次是机制之间的比较。同一个要求，可以在不同层实现；一个系统里合理的成本，换到另一个系统里可能无法接受。最后是组合后的决定。当数据库、缓存和消息同时参与一次请求时，各组件单独成立的保证，并不会自动成为整个请求的保证。

第10章因此安排了一个完整的设计推演。它从库存问题出发，列出假设、比较候选、确定成功响应的含义，再检查提交、重试和故障切换。这是用于说明判断过程的设计示例，不是对我当年项目细节的补写。书中已有的亲历会照实保留；没有记录的吞吐、会议过程和实现结果，不会借示例写成发生过的事情。

现实中的决定比示例还受更多条件约束：什么时候交付，谁来维护，现有系统能改多少，业务愿意接受怎样的失败。把这些条件补齐以后，也未必只剩一个答案。两个方案都能满足要求时，维护成本、迁移风险和团队已有能力就可能改变选择。这类判断可以有明确立场，也应允许不同意见。

我不希望这本书用人的性格概括产品。Redis 的低延迟不能回答所有缓存一致性问题，MySQL 的事务不能自动覆盖另一个服务的写入，Kafka 的副本也不等于业务处理恰好发生一次。它们是观察样本。沿着一次请求、一次提交或一次恢复追下去，比给软件排一个总名次更有用。

这里也保留一些失败后的认识。记录这些认识，是为了说明原来的判断少考虑了什么，不是为了展示经历。一次事故能够推翻当时的某个假设，却不能证明同类系统只能采用一种设计。我会把确定的事实、据此作出的判断和仍受条件限制的部分尽量分开。

本书面向有两年以上开发经验、用过三款软件中至少一款的后端工程师，也面向需要参与选型、维护和故障处理的人。它不是完整的使用手册。为了理解机制，必要的数据结构、操作系统和事务概念仍会解释；命令和配置用来说明具体行为，不能脱离版本、环境和故障假设作为生产配置直接套用。

本版以 Redis 7.x、MySQL 8.0.x 和 Apache Kafka 3.9.0 为观察基线。采用固定基线是为了使讨论可以核对，并不表示这些版本适合所有新项目。涉及特定小版本的行为会单独说明，关键机制也尽量给出能够继续追查的官方资料。

我对架构的兴趣，始终落在这些具体的决定上。读懂一个实现之后，再问一句它为什么这样安排；准备借用一个做法之前，再检查自己是否具有相同条件。如果这份笔记能帮助你把某个原本含糊的决定说明白，它就完成了很实际的一件事。

第1章 引言：为什么是这三个软件

本章导读

我面试过数百名高级开发和架构师候选人。谈到 Redis、MySQL、Kafka，很多人可以熟练解释日常用法；问题换成某个机制为什么存在、换一种实现会怎样，谈话就容易停在“它性能更好”或“这样更可靠”。我自己整理笔记时也遇到过类似困难：机制记得不少，却不容易把一个决定的依据完整说出来。

比如，Redis 通常由主线程执行命令，这可以解释操作之间怎样排序，却不能直接证明单线程在所有负载下都更快。MySQL 有事务，可以帮助约束一次数据库修改，却不能自动保证数据库之外的通知也成功。Kafka 使用追加写日志，解释了它如何记录事件，但消费者已经产生的外部效果还需要另外处理。

这一章先确定观察对象和提问方式。三个软件分别提供什么接口，同一个“成功”在不同条件下表示什么，以及一条实现事实怎样进入设计判断。后面的章节会逐项展开这些问题，而不会把三个产品的概览当作答案。

1.1 为什么把这三个软件放在一起

1.1.1 它们经常参与同一次业务，却承担不同责任

这三个软件，我做了十几年系统，几乎天天在用。它们很适合放在一起观察，因为一项业务常常同时需要结构化状态操作、事务记录和异步事件传递。一次下单可能查询 Redis 中的商品展示信息，把订单写入 MySQL，再由 Kafka 向其他服务提供事件。这里的组合只是常见例子，不是每个订单系统都应照搬的配置。

这几项工作有不同的访问特点。商品展示可能反复读取少量热点字段；订单修改往往涉及若干行之间的约束；下游服务则需要按自己的速度处理一串事件。把数据都放进同一种系统，功能上未必做不到，但应用可能承担更多转换、索引、并发控制和恢复工作。

问题也并非天然分成三个互不相交的类别。Redis 可以保存需要持久化的数据，MySQL 可以承接事件表，Kafka 可以通过 compaction（压实）保留键的较新状态。选择它们作为样本，不意味着存储系统只有这三种，也不意味着它们的能力边界恰好互补。更有价值的是：同一种要求落到不同接口上，会改变必须维护的状态与需要支付的成本。

以计数为例。如果只要求原子增加一个键，Redis 的整数操作提供了直接接口。如果计数变化必须与另一行记录一起提交，事务边界就成为重要条件。如果还要求其他服务分别消费每次变化，事件是否完整保留、消费到哪里又是另一组问题。“计数”这个业务词没有变，正确性要求却已经不同，因而不能仅根据操作名字选软件。

图1-1列出本书反复观察的三类工作。它描述接口与职责的差别，不表示任何系统被固定在某一类中。



三者是观察样本；能力可以组合，以上分类不覆盖全部后端基础设施。

图 1-1 三类常见工作与观察样本：结构化状态操作、关系与事务、事件记录与消费。

一次请求穿过多个组件时，职责之间的交接尤其值得检查。缓存返回的数值是否可以作为扣款依据，事件写入是否与订单提交属于同一事实，消费者失败后怎样知道已经完成哪些工作，都发生在单个组件接口之外。第10章的设计推演会把这些交接明确写出来。

1.1.2 样本的价值在于可以比较具体决定

Redis 的一个特点是把操作放到数据所在的位置。应用不必先把整个集合取回来，再计算成员是否存在。MySQL 用关系、查询语言和事务组织操作，应用可以声明需要哪些记录，由优化器和存储引擎共同完成访问。Kafka 则把事件按分区持续追加，让不同消费者独立保存读取进度。它们提供给应用的基本单位不同，因此应用所承担的工作也不同。

这些差别适合用来研究抽象。一个接口隐去了什么，又保留了哪些必须由调用者面对的事实？Redis 的集合成员操作隐去了具体编码，却没有隐去键所在分片的限制。SQL 隐去了多数页访问过程，却没有让查询成本消失。Kafka 的分区日志允许重复读取，却没有替消费者决定重复业务操作是否安全。

再向内看，同一组约束会在多章重新出现。内存中的对象要尽量紧凑，但紧凑表示可能增加插入成本；日志适合追加，却还需要定位、截断与保留策略；副本可以增加故障后的恢复来源，但同步副本也会引入等待和状态管理。这些关系由机制本身支撑，不需要假设某个产品始终偏爱速度或可靠性。

本书并不试图解释三款软件的全部能力。Redis Modules、MySQL 的所有引擎、Kafka 周边的所有连接器，都超出主要范围。选取一个机制的理由，是它能够回答当前章节的问题。没有共同比较对象时，允许只有一款软件的实现值得展开；三家并列不是要求每个小机制都找到三个形式相似的对应物。

1.1.3 观察的结果应当能改变一个决定

读到“哈希表平均查找快”，还不足以决定如何保存排行榜。排行榜需要按分数取一个范围，且分数会改变；此时，查找单个成员、更新排序位置和顺序遍历不是同一个操作。Redis 为什么同时维护查找与排序结构，第2章会沿着这些操作解释。理解到这里，机制才开始影响应用的数据组织。

同样，“增加副本提高可用性”只说明了可能的收益。副本保存到什么位置、哪些副本有资格接替主节点、失去联系后是否允许写入，决定了故障期间能维持什么保证。第7章与第9章分别讨论拓扑和历史对齐，避免用一个副本数量回答所有问题。

我更愿意把有用的观察记成一个有条件的判断：在什么操作和失败假设下，某种安排解决了什么问题，剩下什么代价。它未必简短，但能在条件变化后重新检查。只有一个形容词的结论通常很难复用，因为无法判断新系统是否仍满足那个形容词背后的前提。

1.2 三个软件分别让应用操作什么

1.2.1 Redis：类型与命令使一部分业务操作留在服务端

Redis 是数据结构服务器。字符串、哈希、列表、集合、有序集合和 Stream 等类型，给出比“读写一段字节”更细的操作接口。例如，`ZADD` 更新有序集合成员及其分数，`LPUSH` 向列表头部添加元素。应用可以要求服务端完成一次结构操作，而不必把整个值读出后再写回。Redis 数据类型说明介绍了这一接口取向；本书只采用7.x基线中已有的类型。

一个原子命令减少了应用中某些读改写竞争。例如，用两个独立请求完成“读取计数再增加”，两次请求之间可能有其他修改；由服务端完成整数增量，则不需要应用在这一段间隙维持互斥。但原子性有对象和执行范围：单条命令的行为不能直接推广到几个独立请求，更不能推广到数据库和消息系统之间的联合修改。

Redis 的主要数据集保存在内存中，键和值的组织方式直接影响容量。对于少量元素，紧凑编码可以减少指针和对象头开销；规模扩大后，切换到适于查找或修改的结构。客户端通常仍使用同一组命令。因此，类型是接口约定，编码是内部表示，两者不能混为一谈。第2章解释表示的成本，第5章再看这层间接关系隔离了哪些变化。

常规命令执行主要由主线程串行完成。Redis6.0引入可选的I/O线程处理部分网络工作，并不等于所有命令可以并行修改同一份数据。串行执行减少了常规命令路径上并发修改数据结构的协调，但单个耗时操作也会延迟后续命令。这里不存在由“单线程”直接推导“更快”的结论：请求大小、命令复杂度、网络 and CPU 占用都可能改变限制因素。

数据在内存中，也不等于没有持久化。RDB（快照文件）保存某一时点的数据状态，AOF（仅追加文件，Append-Only File）保存可用于恢复的操作记录。它们改变重启时的数据来源和可接受的丢失窗口，并没有把内存里的访问结构变成磁盘缓冲池。第4章会分别讨论访问路径和恢复路径，防止把这两个职责合成一句“用内存换速度”。

Redis因此很适合观察一种设计安排：用明确的类型和操作，把部分计算留在保存数据的进程中。代价也来自这里。复杂操作可能长时间占用执行线程，大结果集要经过网络，容量不能只按业务字段大小估计。是否值得采用这种安排，取决于操作是否确实匹配其接口，以及应用怎样处理持久化和跨组件一致性。

1.2.2 MySQL：关系接口之下还有页、版本和提交过程

应用通过SQL描述查询和修改，通过事务规定哪些修改应一起生效。MySQL服务层负责连接、语句处理和执行计划，存储引擎承担具体的数据访问。本书讨论MySQL时，未特别说明的存储机制以默认引擎InnoDB为对象。把服务层和引擎分开很必要：binlog属于服务层，redo log属于InnoDB，它们并不是同一份日志的两个名字。

InnoDB用页组织存储，默认页大小为16KB，聚簇索引的叶子页保存行记录。B+树支持按键定位与范围访问，缓冲池（Buffer Pool）把正在使用的页保存在内存。一次看起来很简单的SQL，可能涉及多次索引访问、行过滤或排序；查询语言的便利并不消除这些内部工作。第2章从访问过程解释结构，第4章解释页如何进入内存以及何时写回。

并发读写还需要说明可见性。MVCC（多版本并发控制，Multi-Version Concurrency Control）保留重建旧行版本所需的信息，让普通一致性读依据读取视图选择可见版本。它使某些读写不必互相等待，但没有取消数据库里的锁。更新、锁定读、元数据操作与普通一致性读的规则不同，不能用“读不阻塞写”概括所有SQL。

在InnoDB默认的可重复读隔离级别下，同一事务中的一致性读通常使用首次此类读取建立的快照；在读已提交隔离级别下，每次一致性读建立自己的快照。这会影响两个连续查询为什么看到相同或不同结果。这里的“一致性读”是特定读取机制，不等同于分布式系统讨论的线性一致性。MySQL8.0一致性非锁定读明确区分了这些条件。

例如，一笔事务先读到某商品剩余十件，另一笔事务随后提交扣减。前一笔事务再次普通查询，可能仍看到原来的快照。不能据此认定副本复制延迟，也不能据此推导它执行更新时仍只会处理快照中的版本。判断读到旧值的原因，需要先说明读发生在哪里、用什么隔离级别、是不是锁定读。第9章讨论副本旧值之前，会保留这项区别。

redo log（重做日志）服务于崩溃恢复，undo log（回滚日志）支持回滚和旧版本重建，binlog（二进制日志）记录服务层的变更历史，供复制和恢复等用途使用。长事务可能使读取视图仍然需要的旧版本无法及时清理。这是MVCC降低部分读写冲突所伴随的维护成本，而不是某个参数配置不当才会出现的偶然现象。MySQL8.0多版本机制说明了undo的这些职责。

事务为业务提供了很有用的边界，但“支持事务”不是“不丢不错”的产品保证。是否强制刷盘、发生哪类故障、是否切换到尚未获得提交记录的副本，都会影响结果。即便数据库内的事务完全成立，向另一个服务发送通知也未必属于同一个提交过程。第4章先拆开持久化与复制确认，第10章再处理数据库之外的状态。

1.2.3 Kafka：记录顺序与消费进度分开保存

Kafka提供分区追加日志。主题（Topic）可以包含多个分区，每个分区中的记录由偏移量（offset）标识位置。生产者追加记录，消费者从某个位置读取；不同消费组可以独立推进。记录被一个消费者读过，不代表其他消费者不能再读，也不意味着它立刻从日志删除。

这种分离使生产和消费具有不同的时间安排。下游暂时停止，生产者仍可能继续写入；消费者恢复后从可用位置继续读取。条件是所需记录仍被保留，而且消费进度及相应业务状态可以确定。日志保留期耗尽、进度丢失、外部处理结果无法识别时，“支持重放”并不自动给出恢复方案。

分区提供了局部顺序，也提供了并行单位。增加分区可能提高能够并行处理的工作量，但不保证吞吐随节点数量线性增长。热点键分布、单分区顺序要求、网络、存储和消费处理能力都可能成为限制。原先依赖同一分区顺序的业务，改变分区安排以后，还必须检查键的路由和已有记录的顺序含义。

Kafka把多条记录组成批次，在请求、压缩、磁盘写入和网络传输中摊薄固定成本。批次越大并不总是越好：等待凑批可能增加单条记录的等待时间，批次涉及的数据也要占用内存。页缓存和适当路径上的零拷贝可以减少复制与系统调用，但具体路径还受加密、格式转换等条件影响。这些机制共同解释吞吐，而不是“磁盘顺序写”一句话就能覆盖全部原因。

副本机制又是另一组约定。ISR（同步副本集合，In-Sync Replicas）描述达到特定同步条件的副本集合，生产者的acks和主题的min.insync.replicas影响确认条件。它们需要结合故障和选主规则解释，不能把ISR中的副本简单叫作“绝对安全”。第7章讲哪些副本有资格参与，第9章讲副本怎样确定自身位置属于哪段历史。

本书采用Kafka3.9.0，以KRaft模式为主要部署背景。控制器维护集群元数据，数据分区的复制又有自己的状态和流程，不能把控制器共识直接当成所有业务记录的提交机制。历史上依赖ZooKeeper的做法，只在解释演化时注明。Kafka的日志、批量和复制设计可继续查阅3.9设计文档。

1.2.4 比较之前，先把比较条件写出来

如果同一张表里，Redis比较的是缓存命中，MySQL比较的是刷盘提交，Kafka比较的是大批量追加，得出的延迟或吞吐排名就没有共同对象。类似地，把MySQL单机事务与Redis Cluster故障切换放在一起，也不能直接排出谁的一致性更强。比较需要先确定操作，再说明完成条件和失败条件。

表1-1只比较三款软件向应用暴露的主要对象，以及应用仍要承担的工作。

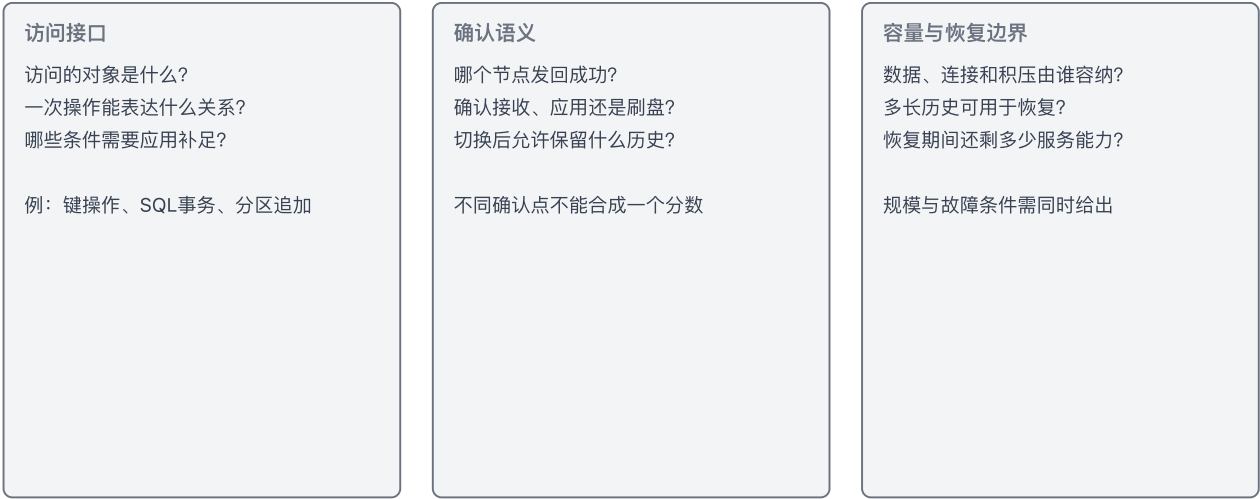
表 1-1 三种接口及其未覆盖的责任

维度	Redis	MySQL/InnoDB	Kafka
常用操作对象	键及其结构化值	表、行、查询与事务	主题中的分区记录
常见操作形式	对类型执行命令	声明查询或事务修改	追加、按offset读取
内部组织线索	对象编码、字典与有序结构	页、索引、版本与日志	日志段、批次与索引

维度	Redis	MySQL/InnoDB	Kafka
接口外仍需明确	跨请求与跨组件约束	事务外效果及副本读取条件	消费进度、外部效果与重放处理
容量与恢复问题	内存占用、持久化和副本来源	工作集、页写回、日志与备份	保留策略、分区分布和历史来源

这些代价并不只存在于某一个产品中。三者都需要管理内存，也都需要面对网络故障。差别在于哪些工作进入主要访问路径，哪些状态被纳入接口保证，哪些责任留在应用中。理解这一点，才能比较具体实现怎样分配成本。

图1-2把比较前需要固定的条件列在一起。



在同一工作负载与确认条件下比较；产品名称本身不提供性能或一致性排名。

图 1-2 比较软件前先确定条件：访问接口、确认语义、容量与恢复边界。

即使两个系统都返回“写成功”，一个可能表示主节点已经执行，另一个可能包含特定日志刷盘或副本确认。没有把这个差别展开，性能数字可能衡量的根本不是同一件事。后文的对比将尽量围绕这些条件展开，不使用无条件的产品总评分。

1.3 从实现事实走到设计判断

1.3.1 使用知识与判断依据怎样连接

使用知识是理解设计的起点。知道某条命令会阻塞、某个配置改变确认方式，才能观察实际系统。但把使用注意事项记住以后，还需要解释它为什么存在，以及边界会怎样移动。

以Redis的 KEYS 为例，危险不来自命令名字，而来自一次遍历的工作量进入了服务请求路径。在键很多的实例中，一次性扫描可能影响其他请求。把它换成分步迭代，改变的是单次承担的工作量与调用方式；它并没有使扫描总成本消失，也没有保证每一步恰好返回固定数量的元素。第2章讨论渐进式rehash时，会遇到同样的“分步执行”和“限制最坏延迟”之间的区别。

类似地，知道MySQL可以配置日志刷盘，只是说明存在选择。决定怎样配置，还要知道丢失的是哪种状态：一次进程崩溃、整台机器断电、主节点失联后切换到另一个副本，并非同一种故障。一个设置能够改善第一种情况，不代表覆盖另外两种情况。

表1-2把具体操作与它引出的判断问题对应起来。

表 1-2 使用信息怎样进入设计判断

已知使用信息	需要继续确认的机制	可能改变的决定
大范围扫描影响请求延迟	一次调用处理多少工作，能否分步，迭代期间数据能否变化	后台扫描方式、数据分区与限速
提交时可以等待刷盘	哪份日志何时持久化，恢复还需要什么	成功响应条件和容许的丢失窗口
Kafka可以增加分区	键如何路由，顺序范围怎样变化，瓶颈在哪里	是否扩分区，以及业务是否要迁移状态
副本可以自动接替	接替者拥有什么历史，原主是否还可能接写	故障时拒写、等待或容忍何种结果

这几项判断没有共同的万能答案。共同之处是：先把实现对结果的影响说清，再决定那个结果是否可以接受。作者的经验可以帮助发现问题，但不能替代这一步论证。

1.3.2 约束、演化和抽象是三个观察角度

约束先于比较结论。需要范围查找、希望降低页访问次数、更新经常发生，这些条件会影响索引结构；并不是给一个结构标上“高效”，它就自然适合。约束也有冲突，且不一定都能满足。如果要求在失去联系时继续接受所有写，又要求所有节点立即读到同一最新状态，就必须进一步界定网络故障模型和成功含义，不能只寻找更强的配置。

演化可以暴露原来没有突出的成本。Redis为网络工作引入I/O线程，并没有同时把常规命令执行全部并行化，这提示网络开销与数据结构并发修改是不同问题。Kafka把集群元数据管理从外部ZooKeeper依赖转向KRaft，改变了控制面的组织和运维方式，也没有使数据分区复制变成控制器日志的一部分。研究演化需要追踪改变了哪项职责，不能仅凭版本时间顺序猜测全部设计动机。

抽象则决定哪些变化不必让调用者知道。MySQL向应用暴露的是SQL和事务，页与索引主要是内部实现对象；Kafka向消费者暴露分区位置，读取进度因而成为使用接口的一部分。抽象并非越少细节越好：隐藏了调用者必须处理的失败条件，会使接口看似简单，却把问题推迟到故障时。

这三个角度不是完整方法论，也不能替代测试、需求分析和运维知识。它们的作用是帮助追查一个决定。约束解释选择为何成立，演化帮助寻找条件变化，抽象说明责任如何划分。不同问题可能只需要其中一个角度，不必每次都按固定格式作答。

1.3.3 用一次写入说明条件为什么不能省

下面是一个说明性例子，不对应某次真实事故。应用把库存从十改成九，服务返回成功；随后进程退出，重启后又看到十。第一步并不是立即判定“日志不可靠”，而是确定重启到底使用了哪份状态。

如果修改只在内存中，恢复来源是较早的快照，结果可能符合原配置。若操作已写入操作系统缓冲，但尚未完成要求的刷盘，那么进程崩溃与整机掉电又可能有不同结果。若原节点上的写仍然存在，只是业务切换到了未获得该写的副本，则问题属于复制与接替条件。三种情形都可以表现为“成功之后数据不见”，修正措施却不相同。

再把请求的一侧补齐：数据库已经提交，但响应在网络中丢失，客户端只得到超时。这时重试可能再次扣减；不重试又可能使用户误以为操作失败。持久化解决了状态是否保存，却没有替客户端确定这次业务操作是否已经完成。稳定的请求标识、结果查询和幂等处理因此成为组合设计的一部分。

日志内容也要单独说明。Redis AOF以可重放的命令形式保存恢复信息；InnoDB redo围绕页修改和相应恢复操作组织；Kafka分区日志本身保存记录。不能因为都叫日志，就认为恢复顺序、幂等条件和附属状态相同。Kafka不用把分区记录先恢复成另一套数据库行，但它仍要处理日志尾部、索引及生产者状态等恢复工作。

一个有意义的结论可以是：在本地日志满足既定持久化条件、恢复实现遵守其协议的前提下，进程或机器故障后的状态有确定的恢复依据。接下来还需要说明副本故障切换和业务重试。这样的表述比“日志保证不丢”长，却把后续需要解决的问题留在了明处。

1.3.4 什么证据足以支撑一次选择

机制说明提供“为什么可能成立”的依据，测试提供给定环境下的行为观察，两者不能互相替代。一次压测看到平均响应很低，无法证明故障切换时已确认写仍在；一次故障实验没有丢数据，也无法证明所有切换时序都安全。设计保证需要协议和配置边界，容量判断需要实际负载，两类证据应分别记录。

负载描述也需要与决定有关。单条记录大小、热点分布、读写比例、并发连接、批量策略和确认方式都会影响结果。如果要比较两种写入方案，至少应让它们完成相同业务工作，并要求相同的成功含义。否则一方少做了持久化或确认，数字更好看，却没有回答原来的问题。

证据不足时，也不必把全部判断改成含糊的“综合考虑”。如果某方案在已知故障条件下会失去业务不能丢的状态，就可以排除；如果某项能力尚未在目标负载下验证，就应把它列为待证条件。明确说明选择依赖什么，比给每句话添加“通常”“大概”更便于后续修正。

我会把自己的判断写在这种边界上。对一个方案表示赞成，不表示它适用于所有团队；不选择某个机制，也不表示那个机制设计失败。能够说出什么条件变化会让我改判，是这份笔记应当保留的部分。

1.4 本书的组织、基线与阅读范围

1.4.1 主题之间有联系，但不是一条严格依赖链

起初按软件整理笔记时，同类机制离得很远，难以比较。改成按主题后，Redis的持久化、InnoDB的日志和Kafka的记录才有机会在相同问题下讨论。这是本书采用主题结构的直接原因。

第2章讨论表示与协议，第3章讨论初始化、就绪和退出，第4章讨论内存、磁盘和持久化。第5章追查职责和变化的隔离，第6章讨论访问控制，第7章讨论分片、副本和故障决策。第8章进入文件布局与兼容，第9章讨论历史和进度的对齐，第10章用一个有明确假设的设计示例组合前面的判断。

图1-3保留了这组主题关系。



图 1-3 全书主题与关联：主题分组服务于阅读，不能当作严格的前置依赖顺序。

第6章安全不是理解第7章分片的必备前提，第8章页布局也不必等到掌握全部集群机制以后再读。真正的依赖落在具体概念上：讨论副本落后需要先知道日志位置；讨论页恢复需要先区分数据页和redo。需要回看时，正文会给出相应章节，而不要求读者每次从第一章重新开始。

1.4.2 固定版本，是为了让事实可以核对

本版使用以下观察基线。部署模式与小版本差异也需要说明，不能只写产品名字。

表 1-3 本版观察基线

软件	主要基线	讨论时补充的边界
Redis	7.x开源版本	7.0/7.2等新增命令或配置显式注明；单机、Sentinel、Cluster分别说明
MySQL	8.0.x，存储以InnoDB为主	8.0内部配置演化另注；异步复制、半同步、组复制分开
Kafka	Apache Kafka3.9.0	主要按KRaft；ZooKeeper标历史背景；分层存储说明启用条件

这是一组用于写作和核查的版本，不是软件采购或升级建议。生命周期、许可和最新默认值会变化，生产部署需要另查对应发行版的官方材料。本书不以“最新”替代准确版本，也不推断较新版本在所有负载下都比历史版本更快。

历史实现只在能够解释当前设计时出现。例如，紧凑编码为什么更换布局，旧格式给升级带来什么约束。当前机制先讲清，再指出旧机制不同在哪里，避免读者把已经移除的实现当作基线。来源中的提案描述设计意图，发行版代码说明实际实现，两者也要区别。

1.4.3 术语和图表的解释范围

术语首次出现时给出中英文或缩写说明。工程中通行的redo log、binlog、MVCC、ISR、GTID和offset继续使用英文；缓冲池、分区、副本、快照和日志段主要用中文。相同概念尽量保持一个叫法。特别是offset指位置或偏移量，不使用物理学里的“位移”。

一个词在不同主题下可能指向不同保证。例如，事务的一致性读、分布式读写的一致性模型、副本数据最终对齐，不能仅凭共享“一致性”三个字互相推导。遇到这种词，本书会说明观察对象，而不会把定义工作全部交给术语表。

图用来展示结构、次序或状态关系，表用来集中比较相同维度。流程图中的箭头表示发生顺序还是数据流，应由上下文说明；对比表没有列出的条件，也不会因此消失。图表后的文字只补充图中不容易直接表达的关系。

1.4.4 哪些问题留给完整设计

有经验的读者可以从正在处理的主题进入。例如，重启恢复看第3、4章，故障切换看第7、9章，文件兼容看第8章。横向表有助于找到差异，但表里的结论依赖前面的机制；没有把握的地方仍需要回到具体过程。

第10章不是把前面九章再摘要一次。它将明确一个示例业务不能接受哪些错误，比较不同方案，然后沿写入、确认、重试和恢复检查组合后的状态。假设的规模和约束会明说，示例验证也不会冒充生产实测。一个方案在这些假设下成立，只能说明该决定有可追查的依据。

周期、维护能力和遗留系统同样会改变选择。本书无法替每个团队填写这些事实，却可以把需要事实的位置留在设计过程里。若讨论最终仍只得到“Redis快、MySQL可靠、Kafka吞吐高”，就没有比开头的产品印象前进多少。完整的讨论至少应说明：为什么选择了这一种安排，以及还为其承担什么责任。

1.5 小结

Redis、MySQL和Kafka适合放在一起观察，因为它们用不同接口和内部组织处理部分相同问题，差别能够具体追查。比较必须说明操作、确认与失败条件，产品名字不能替代这些条件。约束、演化和抽象帮助发现依据，实际选择仍要结合机制、负载和故障证据。下一章从最直接的对象开始：相同数据需要怎样访问，才决定它应如何表示和传输。

第2章 数据结构与协议：为各自的目标而设计

本章导读

我做过一个排行榜测试。MySQL表没有score索引时，`ORDER BY score DESC LIMIT 100`在数据少的时候很快，增长到数百万行后，查询耗时明显上升。加上适用的索引以后，有序扫描改善了查询。这段经历不能证明数据库不适合排行榜，它首先说明：同一句SQL，执行工作可能从扫描、排序变成沿索引读取。

后来使用Redis有序集合，问题又变成高频更新的成本、按分数读取的代价，以及排行榜是否需要与其他业务状态一起提交。跳表和B+树都能提供有序访问，但所在介质、节点组织和事务要求不同。仅比较复杂度，解释不了一次完整请求。

本章沿两个对象展开：数据在服务端怎样表示，请求在网络中怎样表示。前者决定一次访问需要查找、移动或维护多少内容，后者决定双方怎样划定消息边界、复用工作和处理版本变化。它们有关联，却没有“某种结构必然配某种协议”的一一对应。

2.1 先把一次操作拆开

2.1.1 相同的数据，可能要求不同的访问

一组用户和分数，可以按用户查分数，也可以取分数最高的一百名，还可以查询分数落在某区间的人。三个请求返回的数据来自同一集合，但需要的组织不同。哈希查找帮助定位成员，有序结构帮助定位范围，范围结果还需要逐个输出。查找起点的成本与返回M个元素的成本应分别计算。

如果分数持续变化，有序结构还要更新位置。如果每次变化都产生一条历史事件，追加日志能够保存过程，但仅凭日志无法直接获得当前完整排名，仍需处理和维护相应状态。这样比较，才能看清Redis有序集合、MySQL索引和Kafka事件记录各自承担了哪部分工作。

操作发生在哪里同样重要。随机访问内存也有缓存未命中和指针跳转成本，并非免费；访问不在缓冲池中的数据库页可能需要存储I/O。数据结构的一个节点是小对象还是整页，会影响一次查找触及多少内存位置、多少页，以及修改以后要维护哪些关系。

2.1.2 复杂度没有包含完整的请求时间

$O(1)$ 、 $O(\log N)$ 描述的是操作规模增长时某类工作如何变化，不能直接当成延迟等级。读取一个很长的值，即使通过哈希表很快找到它，复制和发送仍与结果大小有关。一次 $O(N)$ 扫描如果N很小且数据连续，也可能比指针较多的结构更经济。

扩容又增加了另一种区别：平均每次承担多少工作，与最慢一次承担多少工作。预留空间和渐进迁移可以摊薄集中开销，却未必提供严格的单次耗时上限。评价服务端结构时，需要把平均成本、尾部延迟和过渡期占用分开。

协议也要分维度。编码说明一个整数或字符串变成哪些字节；消息边界说明接收多少字节才算一条完整消息；交互方式说明谁先发送、是否等待以及怎样匹配响应；版本规则说明旧客户端遇到新服务端会怎样。文本与二进制属于编码问题，请求/响应属于交互问题，不能放在同一排作为互斥路线。

2.2 内部数据结构：访问过程比结构名称更重要

2.2.1 Redis字符串：长度、容量与重新分配

Redis内部广泛使用SDS（简单动态字符串，Simple Dynamic String）。它在字节内容之前保存长度等元数据，使字符串不必依靠结尾的零字节确定有效长度。保存一段含有零字节的数据时，显式长度也避免了C字符串终止规则造成的歧义。

常用SDS头保存已用长度 `len`、可用容量相关字段 `alloc` 和类型标记 `flags`。不同宽度的头适配不同长度，极短字符串使用的`sdshdr5`把长度编码在标志字节里，不具有与其他头完全相同的容量字段。对外仍得到一个指向内容的指针，内部操作通过头部取得必要信息。图2-1说明这份元数据放在哪里。



图 2-1 SDS与C字符串的内存布局：SDS显式记录有效长度，示意头部不代表所有头类型具有相同字段。

多保存长度，会增加头部占用，却避免每次计算长度都遍历内容。这一收益适用于反复需要长度的操作；它并没有使字符串搜索、复制或比较也变成 $O(1)$ 。一个字段解决的是一个具体问题，不能据此把整个字符串实现标成常数时间。

追加内容时，还需要区分长度与容量。已有空闲空间足够，就写入新内容并修改长度；空间不足，才申请扩大的存储。在Redis7.2.5的SDS实现中，允许预分配的扩容路径会在较小容量时按倍数扩展，达到 `SDS_MAX_PREALLOC` 后改为增加固定余量；同时也存在只分配当前所需空间的路径。这是调用策略，不是每次扩容都无条件翻倍。SDS实现可核对两条路径。

把容量预留给后续追加，可以减少反复重新分配，但空闲容量仍占用内存。缩短内容也不必立即归还全部空间；需要收缩时，相关接口可以调整分配。因此“取长度更快”“减少重新分配”和“实际容量最小”是不同目标，SDS用头部和容量管理明确处理这些目标之间的关系。

2.2.2 有序集合：成员定位与顺序访问分别解决

Redis较大的有序集合使用字典配合跳表。字典帮助按成员查找已有分数，跳表按照分数及同分时的成员顺序组织节点。更新一个成员的分数时，要先知道它是否存在，再维护排序关系；只用一种索引很难同时让两类访问都直接。

跳表底层包含完整的有序节点，上层保存较稀疏的前向连接。查找时，从高层向前，遇到再前进就会超过目标的位置时向下一层继续。插入时根据节点随机获得的层数，修改相应前驱的连接。Redis还维护跨度信息，用于计算排名或按名次定位；底层反向连接则服务于逆序遍历。

图2-2以查找17为例。上层连接让查找跳过部分节点，最终仍需要在接近目标的位置确认具体顺序。

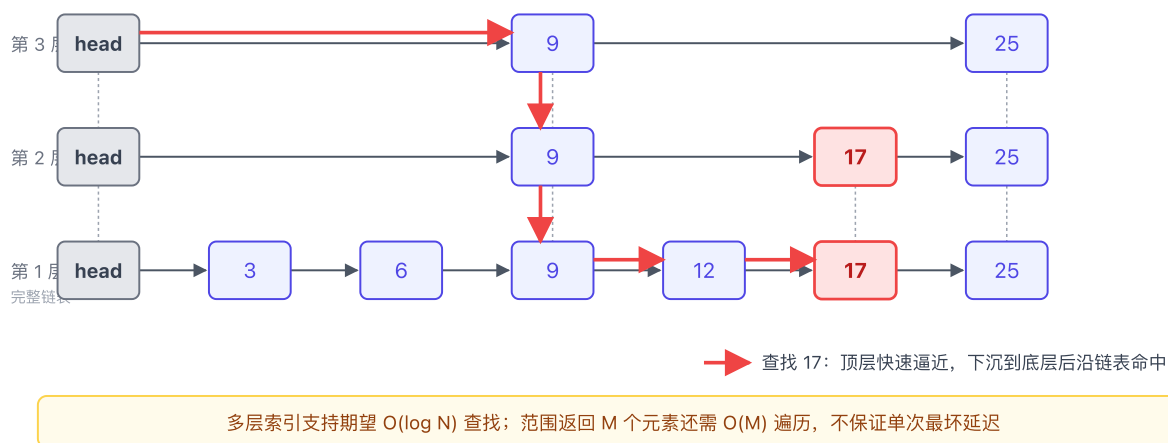


图 2-2 跳表的多层索引与查找路径：查找17时逐层缩小候选范围。

在随机层数假设下，跳表查找、插入和删除具有期望 $O(\log N)$ 成本。查询一个范围时，先定位起点，再顺序输出 M 个结果，成本应写成 $O(\log N + M)$ ，而不是把整个请求都记为 $O(\log N)$ 。这正是开篇排行榜需要区分的两项工作：找到排名起点，与返回一百个成员。

跳表不需要平衡树的旋转过程，但仍有节点、指针和跨度的维护成本。它与红黑树谁更节省缓存访问，取决于节点布局 and 具体实现，不能由名字作总判断。这里值得观察的是，Redis选择了能够直接表达范围和排名操作的组织，并用另一份字典补足成员查找。

较小的有序集合还可以采用紧凑编码。此时查找可能需要扫描连续内容，但在规模受限时，减少分配和指针开销的收益可能更大。客户端使用的是同一组有序集合命令，因此必须根据实际编码和数据规模理解成本，不能认为每次ZSET操作必定走跳表。

2.2.3 字典：迁移状态如何进入正常操作

字典用于Redis键空间，也用于若干对象的内部表示。通过哈希值找到桶，桶内处理冲突，能够在合适负载和分布条件下获得平均常数级的查找。哈希函数、冲突链长度以及键本身的计算成本，仍会影响一次操作。

扩容时，如果一次性迁移所有元素，工作会集中到某个时刻。Redis采用渐进式rehash：先建立新表，在一段过渡期内同时保留旧表和新表，并记录迁移位置。后续部分字典操作及相应维护过程推进迁移。新元素放入新表，查找和删除按照迁移状态检查可能存在元素的位置。

图2-3表现迁移尚未完成时的状态。图中旧表和新表的名字是示意，不要求读者把它们当成某个小版本的结构字段名。

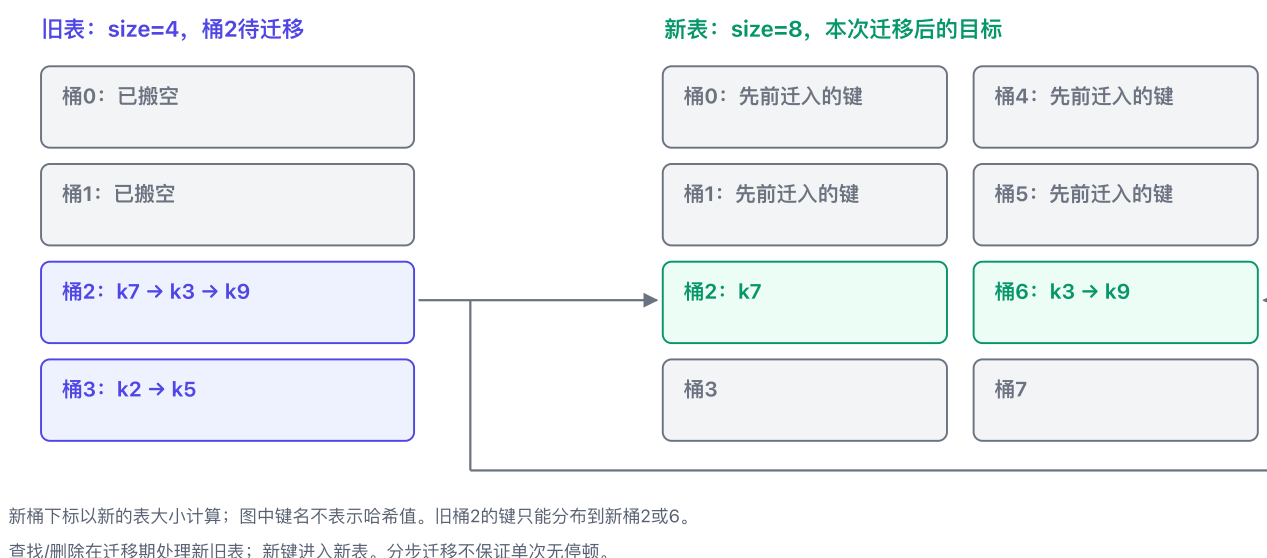


图 2-3 渐进式rehash的双表迁移：过渡期允许元素分布于两张表中。

扩容后，一个旧桶的元素未必仍在同一新桶。需要根据新表大小确定目标位置，随后更新连接并减少旧表中的元素计数。迁移完成后，旧表释放，正常查找改用新表。能够正确处理“尚未迁移”和“已经迁移”的元素，是把工作分步进行的前提。

分步不等于每次没有停顿。一次迁移步骤可能处理一个桶中的多个元素，新表申请也需要资源；实现还要限制扫描空桶的工作，避免空桶过多时一次调用持续过久。Redis7.2.5的 `dictRehash` 明确包含空桶访问限制，以及迁移桶内全部元素的过程。字典实现支持的是分摊集中工作，不能推出严格的单次延迟保证。

过渡期还会同时占用两份表结构，查找也可能多检查一个位置。这里的设计取舍十分具体：降低一次性迁移的集中成本，增加一段时间的状态管理和资源占用。SDS预留容量与字典渐进迁移都处理增长，但前者先多分配空间，后者让旧新布局并存，不能只用“摊还”两个字略过差别。

2.2.4 紧凑编码：减少指针以后，修改怎样进行

listpack把多个条目放在连续内存中，条目按内容选择编码，并保存足够的信息支持向前或向后遍历。没有为每个元素分配独立节点和指针，可以提高小对象的内存密度。代价是中间插入、删除或变长修改可能移动后面的字节；连续布局不会让任意修改变成常数时间。

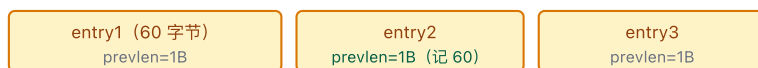
理解listpack为什么这样安排，需要看一项历史对照。旧ziplist的条目保存前一个条目的长度。前一个条目变长时，后一个条目用于保存前驱长度的字段可能扩展；这一扩展又改变了后一个条目的总长度，使变化继续传播。长度小于254时可用短字段表示，越过边界可能改用更长表示，形成连锁更新。

listpack把用于反向遍历的长度信息放到条目自身尾部，描述本条目，而不依赖前一个条目的长度。一个条目变长，不再迫使后续条目因为“前驱长度字段不够”而逐个扩展。图2-4比较的正是这种依赖关系。

ziplist：每个元素记录「前一元素的长度」

元素 = [prevlen 前一元素长度] + [encoding] + [data]

变更前：entry1 长度 = 60 字节 (< 254)

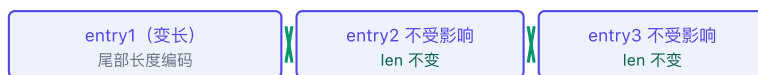


变更后：entry1 增大到 ≥ 254 字节 → 触发连锁



若后续元素也跨过长度编码阈值，prevlen的变化可能继续传播

listpack：每个元素只记录「自己的长度」



listpack移除对前驱长度的依赖；插入或扩张仍可能移动后续字节，不能据此保证常数耗时

图 2-4 ziplist与listpack的长度依赖：listpack消除前驱长度变化引起的连锁扩展，仍可能移动连续数据。

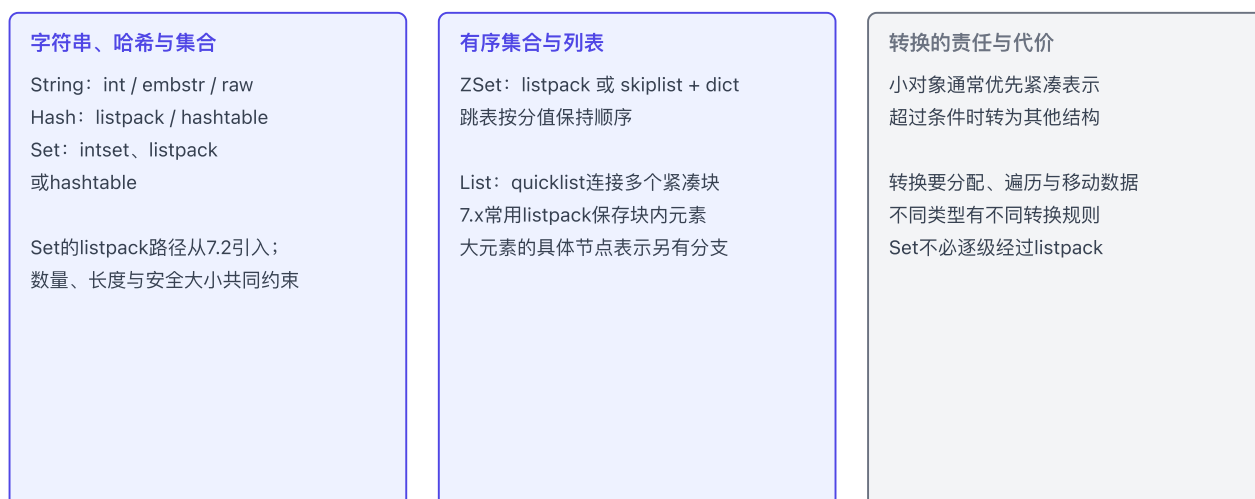
这里的改善是局部编码不再引用会变化的相邻条目长度，不是所有写入都不再搬动数据。Redis7.x使用listpack实现相关紧凑编码，旧格式出现在本书时属于演化背景。文件中怎样兼容历史格式，第8章另行讨论。

列表还需要避免一块连续存储无限增大。quicklist用链表连接多个紧凑块，Redis7.x的相关块采用listpack。跨块访问需要沿链移动，块内修改则在较小的连续区域内完成。块太小会增加节点和指针开销，块太大又可能增加局部移动与压缩的成本。它是在整体链式结构中保留局部连续布局，并非必须在链表和数组之间二选一。

intset则针对小整数集合：所有元素采用统一宽度，保持有序，查找可以二分。插入超过当前整数宽度范围的值时，需要扩大表示并转换已有元素；有序插入本身仍可能移动数组内容。原有较窄值不会因为新值较宽而丢失，但转换不免费。删除大值以后，也不应假定每次都会立即回到最窄表示。

对象类型相同，具体编码可以变化。Redis7.2增加了集合的listpack编码；intset遇到非整数内容时，能否转换成listpack，还受元素数量、元素长度和安全大小检查等条件约束，不是“小于某个数量就一定转换”。超过适用边界会使用哈希表。版本和转换条件可核对Redis7.2.5集合实现。

图2-5汇总的是不同结构承担的工作，以及小规模对象可能进行的编码转换。



图列常用路径，不覆盖每一种内部结构；类型语义与编码选择在服务端共同维护。

图 2-5 Redis底层结构与编码转换：转换受类型、数量、元素大小和版本条件约束。

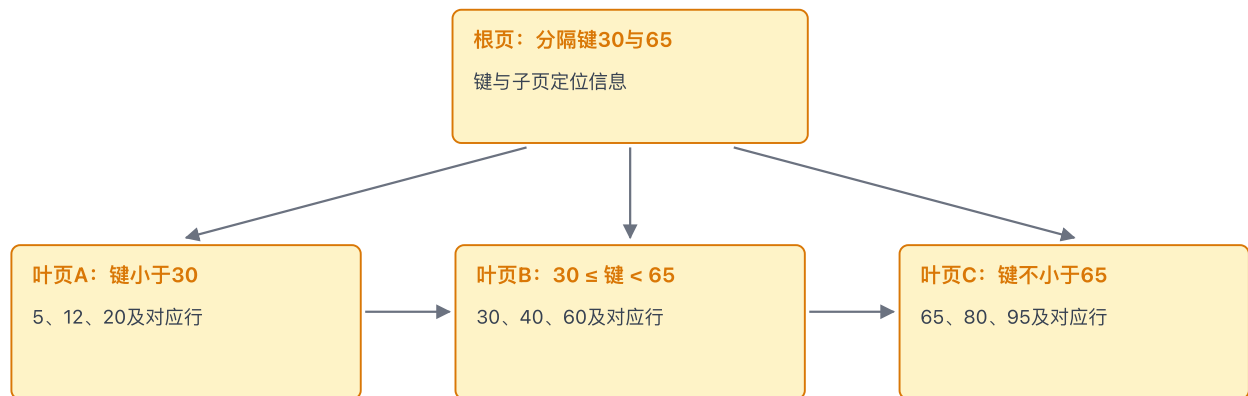
结构转换对命令语义可以透明，对延迟和占用却未必透明。一个键跨过转换条件后，数据仍然相同，分配、查找和修改路径已经改变。这也是容量测试不能只使用几个极小键的原因。

2.2.5 InnoDB：把有序访问组织成页

InnoDB的常规表和索引以B+树组织，节点对应页。与跳表每个节点保存少量对象不同，一个内部页可以容纳多个分隔键和子页引用。一次取得页后，可以在页内缩小查找范围，再访问目标子页。较高的分支数量使树不必很高，但实际高度取决于键宽、行宽、填充情况和数据量。

聚簇索引叶子页保存行记录，二级索引叶子页则主要包含索引键及用于定位聚簇记录的主键信息。两者不能都画成“叶子保存完整行”。如果查询需要的内容不在二级索引中，定位后可能还要访问聚簇索引；如果满足覆盖条件，则可能减少这一步。索引选择因而与返回哪些列有关。

图2-6画的是聚簇索引的示例树。图中采用默认16KB页作说明，不表示所有InnoDB实例都只能使用这种页大小。



叶层按键顺序关联，范围读取先定位起点再继续访问；图中省略反向链接。

默认页大小16KiB；树可以包含更多内部层。叶页保存行数据的说明针对聚簇索引。

树层数不等于固定物理I/O次数：已有缓存、预读和访问对象都会改变实际读取。

图 2-6 InnoDB聚簇索引B+树示例：内部页帮助定位，叶子页保存记录并支持有序扫描。

按键查找沿树向下；范围查询先定位起点，再沿叶子页顺序访问。树高不等于磁盘I/O次数：根页和内部页可能已经在缓冲池，目标页也可能命中内存。一次查询还可能访问多个索引或返回大量数据，因此“两三次I/O即可查任意数据”并不是由B+树结构保证的。

插入时，目标页可能没有足够空间，需要分裂并调整父层引用。首先改变的是内存中的页和相应恢复记录，脏页何时写回另由存储管理处理，不是每次分裂都立即独立写一次磁盘。分裂还会影响空间利用率和后续扫描的页分布。第4章会接着解释写回与日志先后次序。

行格式决定一条记录在页里怎样表达。DYNAMIC行格式需要记录变长字段长度、NULL信息和列内容，聚簇记录还带有事务及回滚相关信息。较长变长列可以放到页外，行内保留20字节引用；是否外置依赖整行大小、页大小和列内容，不是列被声明为TEXT就必定不在页内。MySQL8.0行格式说明给出了这些条件。

页外存储使一个长字段不必占据过多索引页空间，却使读取该字段增加访问步骤。只查询短字段与查询完整大对象，成本可能不同。因此“页尽量装满”也不是唯一目标：保留修改空间、控制长列影响和满足访问需求，都要进入组织过程。精确字段布局留到第8章，本章关注它怎样改变访问。

自适应哈希索引是另一个例子。InnoDB根据观察到的搜索模式，为缓冲池中部分索引内容建立基于键前缀的哈希访问路径；它没有取代B+树，也不是为每一个热点键单独生成一套完整索引。维护和并发访问本身有成本，某些负载可能收益不足。自适应哈希索引说明因此要求结合负载比较其收益。主存储保持有序，热点路径可以另加辅助结构，两者解决的是不同层面的访问问题。

2.2.6 Kafka：把重复元数据集中到批次

Kafka日志由记录批次RecordBatch组成，一个批次可以包含一条或多条记录。生产者按分区积累记录，达到大小条件或等待条件后形成可发送批次；一个网络请求又可以携带多个分区的数据。因此批次、请求和日志段是三个不同单位，不能因为都支持批量就混为一谈。

把元数据集中在批次头，能够避免每条记录都重复保存完整的基础信息。记录使用相对批次起点的offset差值和相对基准时间戳的差值，批次级属性说明压缩、事务等状态。小差值配合变长整数可以减少部分编码开销，但具体收益仍取决于内容和数值，不是每条元数据恒定只需几个字节。

表2-1保留V2批次的键头字段，详细字节边界将在第8章与恢复、兼容一起讨论。

表 2-1 RecordBatch V2的主要字段与职责

字段	作用
baseOffset 、 batchSize	给出批次起点和长度；长度不包含最前面的这两个字段
partitionLeaderEpoch 、 magic	标识相关主节点任期与记录格式版本
CRC	校验规定覆盖范围的批次内容，不是整个请求的校验
attributes	编码压缩、时间戳类型、事务与控制批次等属性
lastOffsetDelta	相对起点的末尾offset差值
baseTimestamp 、 maxTimestamp	记录时间戳编码与范围所需信息
producerId 、 producerEpoch 、 baseSequence	支持幂等生产和事务相关检查的状态
recordsCount	批次内记录数量

图2-7展示批次头与记录的关系。



固定部分共61字节。batchLength不包含最前12字节；CRC从attributes开始，覆盖其后批次内容。
baseTimestamp为批次基准时间戳，不要求是最小值；消费者API仍可以逐条暴露记录。

图 2-7 Kafka RecordBatch V2布局：61字节固定部分与之后的记录；CRC覆盖范围按格式定义。

格式的固定部分占61字节，但批次成本不止这61字节。记录还有自己的长度、键值、头信息，压缩和协议封装也要参与计算。一个批次内相似内容可能更容易压缩，不相似或已经压缩的数据则未必获得相同收益。具体字段与校验范围可核对Kafka3.9消息格式。

Broker可以在满足格式和配置条件的路径上保留批量表示，但接收过程仍需要校验，并可能涉及压缩或格式相关处理。批次并不都原样落盘，单条记录也可能需要检查。同样，消费者API最终可以向应用逐条提供记录，线上传输的批次边界不等于应用只能一次处理完整批次。

日志索引提供从offset到附近文件位置的定位，之后再扫描找到相应记录；它不是按任意业务字段建立的数据库索引。若业务需要按用户查询历史记录，需要另外组织状态或索引。Kafka保留了顺序日志这一主要访问路径，也把任意字段查询的工作留给了其他处理环节。

2.3 通信协议：编码、边界与交互分别比较

2.3.1 RESP：可读控制信息与二进制安全内容并存

RESP（Redis序列化协议，Redis Serialization Protocol）通过类型前缀和长度等信息表示数据。客户端通常把命令及参数编码成字符串数组，服务端返回相应类型。普通命令采用请求/响应，Pipeline允许连续发送多个请求再接收响应，订阅与RESP3 Push等机制还包含主动推送。因此请求/响应并不是Kafka独有的属性。

RESP2的常见前缀包括简单字符串 `+`、错误 `-`、整数 `:`、批量字符串 `$` 和数组 `*`。下面用可见转义表示 `SET key value` 请求的字节，`\r\n` 表示回车与换行，不是要把反斜杠字符直接发给服务端。

```
*3\r\n$3\r\nSET\r\n$3\r\nkey\r\n$5\r\nvalue\r\n
```

*3 声明三个数组元素，每个 \$ 之后给出内容的字节长度。接收方知道有效载荷应读多少字节，因此值可以包含零字节或换行，不依赖“看到换行就结束整个值”。RESP具有ASCII控制表示，不应简写成只能发送可打印内容的纯文本协议。

长度前缀还要求解析器处理分包。一次网络读取可能只收到半个长度字段，也可能收到几条完整请求及下一条的一部分。解析状态需要保留，直到取得声明的内容和结束标记。协议容易辨读，可以帮助调试，却不能消除边界检查、长度限制和错误处理。

Pipeline主要减少逐请求等待的网络往返，命令仍独立编码和执行。它不是事务，也不保证多条命令要么全部成功、要么全部失败；发送很多请求还可能积累客户端与服务端缓冲。减少往返和减少服务端计算是不同目标。

Redis6.0引入RESP3与 HELLO 协商。7.x客户端可以使用RESP2，也可以协商RESP3；Map、Set、Boolean和Push等类型使一些返回值更精确。譬如哈希内容不再只能依靠命令上下文解释一串扁平数组。切换协议需要客户端正确处理返回类型，不能只认为“换了版本号，应用不受影响”。这些行为见RESP规范。

2.3.2 MySQL：握手能力、SQL与结果编码不是同一层

MySQL经典协议先进行连接握手与认证，再进入命令阶段。能力标志帮助双方约定支持的特性，认证过程由插件及连接条件决定；认证响应并不都采用“把密码加密后传回”的方式。TLS提供的传输保护与认证响应怎样计算，也属于不同机制，第6章会进一步说明。

协议包有长度和序号等边界信息，负载中还可以有定长整数、长度编码整数和长度编码字符串。长度编码整数用不同前缀区分后续宽度，某些标记还具有NULL等专门含义。它们解决如何解析字段的问题，并不意味着所有结果列都以最紧凑的二进制数值发送。

普通 COM_QUERY 对应的文本结果集，与预处理语句使用的二进制结果集需要分开。文本结果集中，行的列值主要以长度编码字符串等方式传递；二进制结果集结合NULL位图和字段类型使用相应编码。MySQL官方分别定义文本结果集与二进制结果集。

这使“二进制一定更小”也成为需要条件的判断。一个小整数的十进制文本可能很短，一个较大的整数可能用定长二进制更紧凑；字符串还需要长度与内容。比较应该以实际列类型和结果分布计算，而不是仅看协议类别。

预处理语句则复用另一类工作。客户端发送带占位符的SQL，服务端返回语句标识；随后执行可以携带标识和参数。这里示意的是协议过程，不是可直接输入mysql客户端的命令。

```
COM_STMT_PREPARE "SELECT score FROM ranking WHERE user_id = ?"  
服务端返回 statement_id 及参数、结果元信息
```

复用语句表示可以减少反复发送和解析相同SQL结构的部分工作，参数传递也有明确类型。它不意味着执行计划永不调整、元数据变化不再检查，更不意味着多次执行天然合成一个批次。Pipeline节省等待往返，预处理复用语句，批量INSERT把多行纳入一个语句，它们优化的工作不同，可以同时存在。

2.3.3 Kafka：每类API的版本与批量边界

Kafka经典请求头包含API标识、API版本和用于匹配响应的 `correlation_id`，常用头版本还包含 `client_id`。这里的 `client_id` 用于识别客户端请求来源，不等于经过认证的用户身份。请求体按API及版本解释，例如Produce、Fetch与Metadata具有不同字段。

头部也存在版本。支持灵活版本的头可以包含tagged fields，不应把所有请求说成完全相同的固定四字段格式。客户端通过ApiVersions了解Broker支持范围，再选择双方共同支持的API版本；若不存在共同版本，就应处理不兼容，而不是仅凭文件里有一个版本字段就自动兼容。具体规则见Kafka3.9协议。

版本号解决的是按哪种结构解析。仅凭版本号不能确定未知字段是否可以忽略、字段含义是否变化，以及旧客户端是否能理解新行为。可选扩展可以降低升级联动，改变核心语义仍可能要求显式版本升级和调用者适配。

Produce请求可以包含多个分区的数据，每个分区部分携带记录批次。Fetch请求给出要读取的分区和起点等条件，返回相应记录内容。一次请求能带多条记录，不等于协议不允许单条记录；在低流量下，批次可能很小，等待凑批与立即发送之间仍有代价。

传输也不保证每次都采用相同的零拷贝路径。加密、解压、格式转换或其他处理会改变数据是否需要进入用户态。协议布局提供了批量传递的条件，具体实现能否避免额外复制，需要结合配置与路径判断。第4章会从页缓存和I/O再解释这一点。

图2-8按相同维度对比三者。编码方式、批量形式与兼容规则各有自己的位置。

观察样本	编码与交互	成本与兼容
Redis / RESP	类型前缀与长度字段 字符串负载二进制安全 支持请求/响应	Pipeline减少逐条等待 不把多命令变成事务 RESP2与RESP3有不同类型集
MySQL协议	命令与结果集都有结构 文本结果与二进制结果并存 支持请求/响应	预处理复用语句及参数绑定 不等于多条记录合成一批 握手交换能力标志
Kafka协议	各API采用二进制结构 请求头标明API与版本 支持请求/响应	RecordBatch摊销记录开销 API版本与扩展字段支持演进 消费者API仍可逐条返回记录

编码方式、批量方式和交互模型是不同维度。

图 2-8 三种协议的编码、批量与兼容方式：RESP、MySQL经典协议与Kafka协议都不能由单一标签概括。

消息边界和请求关联信息解决解析与匹配，不能证明业务执行成功。连接断开以后，客户端可能不知道最后一个请求是否已经执行。协议有响应，并不消除响应丢失后的业务重试问题；这个边界在第10章会重新出现。

2.4 横向对比：优化的是哪部分工作

表 2-2 数据表示与协议的共同问题

问题	Redis	MySQL/InnoDB	Kafka
怎样快速定位	字典、跳表或小对象扫描	B+树，部分访问可用辅助哈希	按分区offset定位到日志附近
怎样连续访问	有序集合底层遍历、紧凑块	叶子页和页内记录顺序	批次与日志段顺序读取
小对象怎样减少开销	SDS小头、listpack、intset	行格式、页内记录组织	批次共享头、差值编码
增长与修改的成本	编码转换、rehash、连续数据移动	页修改、分裂、索引维护	新批次追加及后续保留清理
怎样减少重复交互	Pipeline连续发送独立命令	预处理复用语句；可另用多行语句	按分区积累批次，请求承载多个批次
返回值怎样编码	有类型前缀与长度，内容二进制安全	文本/二进制结果集按命令路径区分	API定义的二进制字段与记录批次
兼容如何表达	HELLO选择RESP版本，命令返回类型约定	握手能力、协议及认证插件约定	ApiVersions、每API版本和扩展字段

三者都在减少重复工作，但减少的位置不同。保存长度避免重复扫描，辅助索引避免重复走某段查找路径，批次避免每条记录重复保存相同元数据。采用其中一种办法，还需要支付维护这份额外状态的代价。结构越复杂不一定越好，状态维护与实际访问是否匹配才决定收益。

2.5 把开篇排行榜再检查一次

先只比较“读取前一百名”这个操作。没有适用索引的SQL可能扫描并排序，有序索引可能使它沿序读取；Redis有序集合也可以先定位再返回结果。这一层比较需要确认执行计划、集合编码和返回数量，不必先给数据库或缓存排快慢。

然后增加“分数频繁变化”。数据库要维护有关索引，Redis也要维护成员查找与顺序关系；两边都不是仅改一个整数即可。再增加“分数与付款状态一起生效”，事务和跨系统写入开始影响选择。如果Redis里的排名是从已提交业务记录派生，还需要定义刷新延迟与重建过程。这时Redis仍可能适合，但理由已经从“跳表复杂度低”扩大到了完整的状态职责。

下面的SQL只展示如何检查索引是否帮助这类查询，假定存在测试表 `ranking(user_id, score)` 及适用索引；`EXPLAIN` 查看计划，不执行负载基准。

```
EXPLAIN SELECT user_id, score
FROM ranking
ORDER BY score DESC
LIMIT 100;
```

计划中的访问路径与排序信息比SQL外形更能说明工作量。实际耗时还受缓冲池、并发更新、数据分布等影响。这段检查不能证明生产性能，也不能决定是否迁移到另一个系统；它只检验“现有索引是否可能支持本次有序访问”这一命题。

如果通过补一个索引即可满足要求，引入第二个状态系统可能不值得；如果高频排名和业务事务能够清楚分离，采用派生排名又可能降低热点查询压力。两种选择都需要证据。开篇那次测试告诉我的首先是要检查实际访问过程，而不是把一次对比变成产品结论。

2.6 架构启示：明确改变了什么成本

2.6.1 结构选择与具体操作

决定表示之前，我会先列出最常发生的访问及其返回规模，再看修改会维护哪些状态。一个只有“查找快”的说明不足以支持排行榜，一个只有“追加快”的说明也不足以支持当前状态查询。字典加跳表、B+树加辅助哈希表明，同一份数据可以有多个访问路径；增加路径以后，写入要处理各路径的一致性，恢复还要区分哪些索引直接加载、哪些可以重新生成。

我也不会把平均复杂度当成延迟承诺。渐进rehash已经减少集中迁移，但一次桶迁移的工作与分配成本仍然存在。对尾部延迟敏感的接口，需要观察转换、扩容和返回大结果时的行为，而不能只测已经稳定的小集合。

2.6.2 小对象策略要连同转换过程一起评价

紧凑编码为小数据节省空间，但其价值依赖规模约束。应用中的键可能缓慢增长，也可能一次写入大量元素。若只比较转换前的占用，就遗漏了转换时的临时内存、复制成本和转换后的长期占用。

这一点也适用于自己设计的数据结构。可以先采用紧凑布局，但需要说明何时改变表示、过渡期谁负责解释旧新结构、失败后如何恢复。Redis的编码切换和字典双表迁移提供了具体例子，不意味着所有结构都应该自动升级。

2.6.3 协议简单，应体现为责任明确

我自己设计新协议，会提前写出第二个版本要怎样出现。若改变返回类型，旧客户端如何辨别；若增加可选字段，无法识别它的解析器如何继续；若连接中断，调用者怎样判断能否重试。这些行为比“头部里预留一个版本号”更接近兼容设计。

协议易读有帮助，能够直接看见控制字段可以降低部分排障成本；二进制字段也能借助工具检查。客户端实现是否可靠，还取决于分包、错误、认证、超时和资源限制。不能用几行示例代码或一个下午的开发时间，替代对这些职责的评价。

2.7 小结

数据结构的收益落在具体操作上，查找、输出、修改和转换的成本需要分别说明。SDS、渐进rehash、紧凑编码、B+树和RecordBatch都减少了某类重复工作，同时增加了相应的状态或限制。协议比较应分开编码、消息边界、交互与兼容，不能用三个不同维度的标签代替分析。下一章把对象从一条数据扩展到整个进程：这些结构和外部接口按什么顺序准备好，才可以接受业务请求。

第3章 生命周期管理：优雅启动与关闭

本章导读

我亲身经历过一次凌晨三点的 `kill -9`。当时图省事，觉得Redis重启就好了，没等AOF刷完就杀了进程。第二天发现丢了十几条订单数据。从那以后，我不再把关闭服务当成一个只需要关注进程是否消失的操作。

这段经历不能概括所有强制退出的结果。没有持久化的Redis可能失去整个数据集，已经满足持久化条件的写也不会仅因进程被杀就必然丢失；数据库可能需要恢复未写回的页，集群还可能更换提供服务的节点。相同的终止信号，只负责让进程停止，并不替软件规定下一次恢复使用什么状态。

启动也有类似区别。端口已经打开，数据未必加载完；数据能够读取，节点未必获得集群中的服务资格；进程报告就绪，一项具体业务操作仍可能受恢复任务影响。本章沿这些状态的变化，说明启动和关闭分别需要完成哪些工作。

3.1 问题的本质：运行状态必须有明确含义

3.1.1 存活、可连接和可服务是不同条件

一个进程可以存在但没有监听端口，也可以已经监听但尚未完成数据恢复。操作系统接受连接，只说明传输端点存在；请求能否执行，由应用状态决定。恢复期间返回明确的未就绪错误，可能是有意设计，而不是打开端口太早造成的未定义行为。

就绪也有范围。一个从节点可以响应本地查询，但复制进度可能不满足业务要求；一个Broker已经启动，但某个分区尚未具备预期的副本条件。把所有检查简化为“进程在不在”，会漏掉这些差别。把所有检查都放进一个过度严格的探针，又可能使局部故障触发不必要的整体退出。

因此，启动流程首先是一组条件的建立。配置被解析并校验，必要资源可以使用，本地数据达到实现规定的恢复状态，外部职责已经确认，才允许相应请求通过。不同条件可以有不同完成时刻，有些恢复工作也可能继续在后台进行。

3.1.2 退出不是启动的机械反向过程

启动可以从持久化状态重建内存，关闭却不一定保存内存的完整快照。MySQL可以先保证恢复日志满足要求，把部分页处理留给以后；Kafka保存的业务记录本来就在日志里，还需终止后台任务并协调角色；Redis可以根据配置和显式选项决定是否保存RDB。

退出要说明尚未完成的工作如何处理。哪些请求已经接受，哪些修改已经确认，哪些连接还能继续发请求，哪些职责需要转移，都需要确定。未完成的工作可以等待、明确失败，或留到恢复时处理，但不能因为进程退出成功就认为业务结果已经明确。

图3-1把启动、服务、关闭和异常退出之间的关系分开画出。

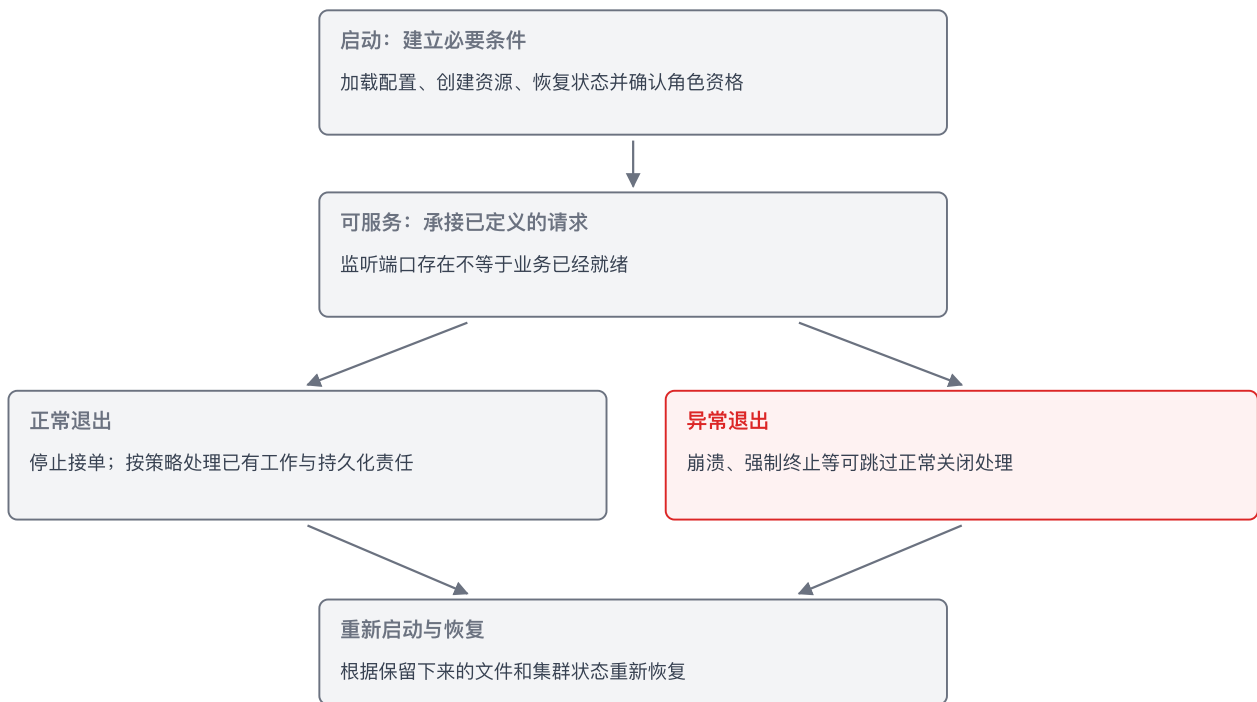


图 3-1 启动、服务与退出的状态关系：正常关闭和异常终止留下的恢复工作不同，关闭不等于制作完整快照。

异常退出可能发生在任何阶段，包括尚未完成初始化和正在关闭时。恢复机制必须处理这些中断点，不能只处理“正常运行后被杀”这一种情况。临时文件是否可识别、已完成阶段能否重复执行、外部角色是否会过期，都属于这个问题。

3.1.3 关闭速度要和下一次恢复一起衡量

把工作移到下次启动，能够缩短本次停机等待，却可能延长恢复时间。并行恢复可以减少总耗时，但会增加磁盘、CPU或内存压力。也有优化通过减少不必要工作提高速度，并非所有“快”都只是推迟代价。

本章比较的不是三款软件谁启动最快，而是它们必须重建哪些状态、怎样推迟或并行处理，以及何时可以对外承担责任。只有在数据量、故障类型和目标状态相近时，耗时才有可比较的含义。

3.2 Redis：加载键空间与停止写入之间的边界

3.2.1 默认配置先建立，合法性仍要继续校验

Redis先建立默认配置，再读取配置文件和启动参数。这样，未显式指定的项具有已知来源，后续代码不必在每次使用时检查是否已经赋值。但默认值只是初始化安排的一部分：字段范围、参数组合、文件权限和实际资源是否满足要求，仍需要检查。

例如，端口数字合法不等于端口可绑定；数据目录字符串存在不等于进程可以写入；内存配置单独合法，也未必适合当前机器与数据量。统一默认值减少漏配风险，却不能证明整个生命周期都不会出现未初始化或状态不一致的问题。

配置加载后建立事件循环、数据库对象、共享对象及监听等基础设施。启动时预先创建常用对象，可以减少运行时首次使用的一部分分配工作，但后续命令仍会申请内存，数据加载也会分配对象。因此预分配不能保证核心路径再无分配延迟，收益需要限定到被提前完成的那些工作。

信号处理也在这时准备。SIGTERM、SIGINT通常请求进入正常关闭路径；SIGKILL不能被应用捕获，应用没有机会在收到它以后再刷盘或通知副本。具体服务管理器怎样发送信号、给进程留多久，由其单元或容器配置决定，不能仅凭 `systemctl stop` 这个命令推断完整时序。

3.2.2 端口可以先存在，数据操作仍受到加载状态限制

Redis要把所选持久化来源中的数据恢复为内存键空间。监听资源的建立与数据完成加载不是同一时刻，加载过程可以处理受限制的事件，但普通数据命令受到加载状态检查。能够建立TCP连接，并不等于相应键已经恢复完整。

这一安排使诊断信息与业务执行可以分开。一个恢复缓慢的实例，如果完全不提供任何可观察信息，外部难以判断它是在推进还是已经失败；如果允许未经约束地读写部分数据，又会把不完整状态交给应用。明确区分允许的管理操作和被拒绝的数据操作，才是限制条件。

图3-2按职责展示启动过程，监听准备不画成业务已经就绪。



依赖顺序避免使用未准备的状态；默认初始化不保证运行期不会出现配置或内存问题。

图 3-2 Redis从配置到业务就绪：基础设施准备、持久化加载与数据命令放行分别具有条件。

恢复来源依配置选择。启用AOF时，启动按照AOF恢复路径及其文件集合处理，而不是把目录里所有RDB和AOF文件按时间挑出“最新的一份”。如果选择的AOF损坏，也不能不说明原因就退回一个较旧RDB并继续服务；那会改变应恢复的状态。

Redis7.0引入多部分AOF。清单组织base与后续增量文件，base可以采用RDB前导或命令格式，取决于相关配置；加载后还需要按顺序处理增量内容。“加载AOF”因此不等于从第一条历史写命令开始无限重放。第4章讨论这种安排怎样服务持久化，第8章再讲文件集合和切换。

独立RDB恢复的是快照时点的状态，AOF恢复的是所选完整文件集合表达的状态。正常维护的AOF通常覆盖较新的写入，但“启用了AOF”不能证明任意文件都严格比任意RDB更新。实例来源、配置变化、文件完整性与成功持久化的位置都要一致。

加载耗时取决于恢复文件的大小、格式、对象数量、内存分配和机器资源，不能只按文件字节数估计。相同字节量由许多小对象或少量大对象构成，恢复工作也可能不同。一个容量很大的实例，重启时还需要足够内存容纳恢复后的对象，不能只确认磁盘文件能读到。

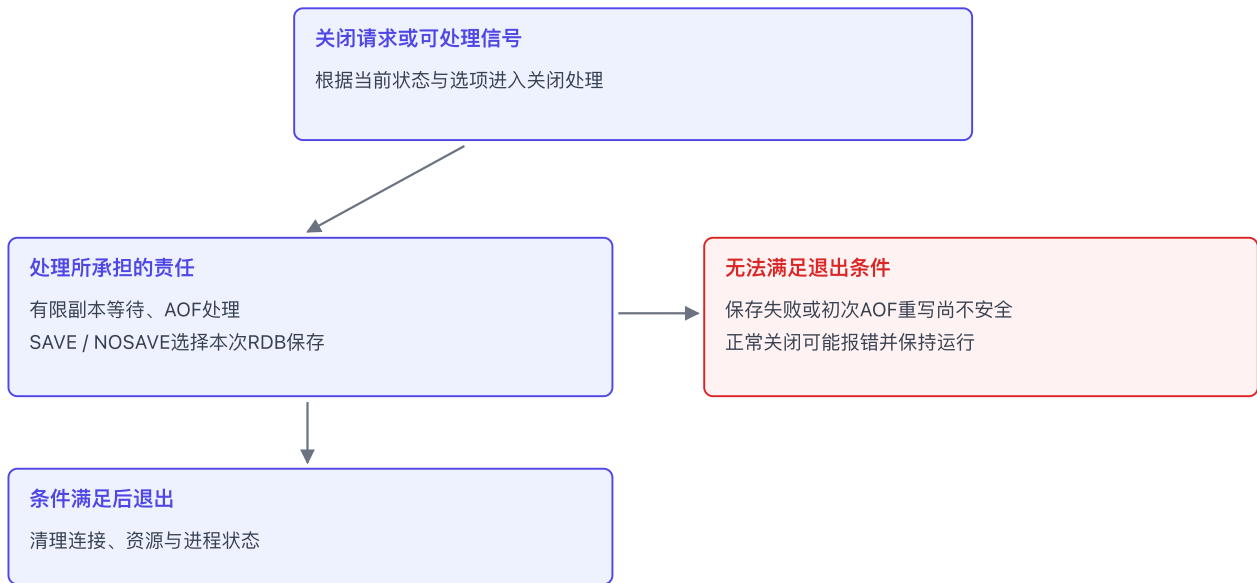
3.2.3 正常关闭提供哪些选择

客户端可以通过 `SHUTDOWN` 请求关闭，进程也可以经信号进入关闭路径。保存RDB、处理AOF、副本等待和资源释放都属于这条路径，但各项是否发生与选项、配置及当前状态有关。

Redis7.0起，正常关闭可以在有限时间内等待落后副本追上当前复制位置，并暂停相应客户端写入。这不是只有Kafka才有的“集群职责”：Redis作为主节点退出，同样会影响副本、客户端和故障转移。等待也只是降低风险，不等于在任何故障下保证另一个节点拥有全部写入。

`SAVE` 强制本次关闭保存RDB；`NO SAVE` 跳过本次RDB保存，不表示关闭AOF，也不表示此前所有状态都已安全保存在别处。没有RDB保存点且没有AOF的实例，关闭以后也不会仅因走了正常路径就拥有恢复文件。选择哪条路径，需要先知道当前数据是否可重建，以及重建是否真的可完成。

图3-3列出正常关闭的条件分支与失败出口。



FORCE等选项改变失败处理：NOSAVE只控制RDB，不自动取消AOF责任。

图 3-3 Redis正常关闭的条件与失败出口：副本等待、RDB选择、AOF处理与无法安全退出的情况。

后台保存或重写任务也不是一律直接终止。一般重写任务与初次建立可恢复AOF来源的任务，安全条件不同。保存失败或不能立即安全持久化时，正常关闭可以报错并继续运行。7.0新增的 `NOW`、`FORCE`、`ABORT` 分别涉及跳过等待、强制退出和取消关闭，不能把它们作为缩短时间的无代价选项。SHUTDOWN文档说明了这些条件。

这些选项需要按退出条件分别选择，不能直接组合成通用的生产命令。为了查看当前持久化和复制状态，可以在具有相应管理权限的连接上使用以下只读命令。

```
INFO persistence
INFO replication
```

前者能够提供加载、后台保存及AOF相关状态，后者提供角色和复制进度等信息。输出用于确认当前阶段，不能单独证明某次业务写已由所有需要的恢复来源保存。正式退出仍需要按部署方式处理客户端入口和职责交接。

3.3 MySQL：恢复页内容，也恢复事务结果

3.3.1 配置覆盖方向必须能追查

MySQL会读取多个选项文件，并按配置组选择适用项。在Unix类环境中，官方列出的顺序是先读取系统范围的文件，再读取后面的相应文件；同名选项通常后读取者优先。不能把“先找到”理解为“优先级更高”，也不能笼统说 `/etc/my.cnf` 覆盖 `~/my.cnf`。

除此之外还有启动参数、包含文件、持久化系统变量及启动方式的影响。`mysqld-auto.cnf` 用于保存通过 `SET PERSIST` 等方式持久化的变量，不是普通手写选项文件；`--user` 又有采用首次出现值的安全例外。具体读取规则见MySQL8.0选项文件。

一个配置排查应把声明值与有效值分开。文件里写了什么，只能说明某个输入；运行实例最终使用什么，还需要查询实例。排查时如果连到错误端口或另一套数据目录，继续修改同名文件也不会解释现象。这属于部署识别，不是再增加一层配置就能解决。

例如，下面的只读SQL可以确认若干与本章有关的有效值，所需权限取决于账号配置。

```
SELECT @@version, @@datadir,  
       @@global.innodb_fast_shutdown,  
       @@global.innodb_flush_log_at_trx_commit,  
       @@global.sync_binlog;
```

它不能替代对选项来源的追查，也没有改变配置。把实例版本和目录一起记录，是为了避免用一份文件解释另一实例的行为。

3.3.2 redo恢复与事务回滚具有不同职责

InnoDB启动需要准备缓冲池、打开有关表空间和日志，建立数据字典所需的状态，然后执行必要的恢复。MySQL8.0的数据字典位于InnoDB表中，不能继续按早期版本把所有元数据都归入 `ibdata1`。系统表空间、独立undo表空间和数据字典的职责各不相同，本章不把路径名字当作恢复算法本身。

数据页写回可以落后于事务修改。redo保存使页达到后续状态所需的恢复信息，checkpoint记录恢复扫描所依据的位置。恢复时扫描有效redo，并结合页的状态判断需要应用哪些记录；已经反映在页里的修改不应再次破坏性应用。恢复不是不加判断地把每条日志重新执行一次SQL。

redo恢复与事务是否最终提交是不同问题。某些未提交事务的修改也可能进入日志和页，必须依据事务状态进一步处理。undo提供撤销所需的信息，帮助回滚未完成事务。涉及MySQL服务层binlog与InnoDB的协调时，还需按内部提交协议判定处于准备状态的事务，不能只看某一页上有没有出现新值。

图3-4把页恢复、提交状态判定与回滚区分开。



可接受连接不表示每项后台恢复工作已经结束。

图 3-4 InnoDB崩溃恢复的职责：redo恢复页状态，事务状态决定后续处理，部分回滚可以继续在后台执行。

一个重要边界是恢复工作与接受连接未必严格串行到底。InnoDB在完成必要redo恢复后，可以尽早接受连接；未提交事务的回滚能够由后台线程继续进行，新事务可能因此遇到锁冲突。处于XA准备状态的事务还具有专门规则。MySQL可以在部分回滚尚未完成时开放网络。InnoDB恢复说明明确描述了后台回滚。

这解释了为什么“ready for connections”与“恢复影响已经消失”不同。一条查询可以开始执行，目标记录却仍受恢复中事务影响。运维看到进程存活、连接成功，也不能立即认定所有业务路径都恢复到正常延迟。

恢复时间受待应用redo、需回滚工作、存储速度和可用内存等影响。长事务可能留下更多回滚工作，但“所有大事务都必须拆小”也不是无条件规则：拆分会改变原子性。合适的判断是明确事务需要保护哪些业务约束，再减少不必要地延长事务的等待与操作。

3.3.3 关闭档位是在分配工作，不是在选择是否保持正确

`innodb_fast_shutdown` 控制InnoDB正常关闭时进行多少清理。它的0、1、2不构成三个一致性等级。尤其是档位2仍先刷新日志，与直接SIGKILL跳过应用清理有重要区别。

表 3-1 `innodb_fast_shutdown`的工作安排

值	正常关闭中的主要安排	后续影响
0	完整purge、change buffer合并及相应关闭工作	本次关闭可能较久，减少遗留清理；升级要求按具体版本路径确认
1，默认	跳过完整purge和change buffer合并，完成正常快速关闭所需工作	部分维护留待后续，不等于清理毫无价值

值	正常关闭中的主要安排	后续影响
2	刷新日志，以需要崩溃恢复的方式关闭	下次启动承担相应恢复，不能等同未刷日志的强杀

purge清理不再需要的旧版本，change buffer合并把延迟处理的二级索引修改落实到对应页。正常运行允许它们逐步完成，不等于它们对状态维护没有贡献。关闭时是否全部完成，取决于下一次启动和升级操作的要求。参数定义说明了三个值的范围。

这种安排仍没有给出统一的关闭时间上限。日志与数据写入可能受到存储压力影响，未完成事务需要处理，复制及其他后台任务也要停止。配置值只说明允许推迟什么工作，不说明剩余工作必定在多少秒内完成。

3.3.4 停止一个数据库实例，会影响哪些外部角色

应用连接应停止向即将下线的实例发起新事务，已接受的事务要按数据库的关闭规则处理；在故障切换架构里，还要决定谁获得新的写入口。关闭主节点不会自动使所有从节点都已经消费完它的binlog，主节点本地完成刷盘也不等于副本已经应用。

相反，关闭一个从节点会影响读取流量、复制追赶和可能的接替资格。如果负载均衡仍把查询发给已经退出的节点，数据库本身再完整的关闭也不能避免应用错误。复制线程停止和当前事件组处理具有具体实现规则，不能把所有节点都当成一个无外部关系的单机进程。

关闭缓慢时，需要识别等待的工作，而不是重复发送更强信号。脏页写回、事务回滚、复制线程和后台任务都可能参与。SHOW PROCESSLIST可以显示部分会话状态，InnoDB状态和错误日志补充存储层信息；没有哪一个命令能保证精准定位所有关闭阻塞。

这也改变了本章开篇事故能够推广出的结论。我会先弄清正常路径还在完成什么，再决定是否接受异常恢复的后果，而不是把“多等一会儿”本身当作安全保证。如果磁盘已经无法写入，无限等待也不会产生一个完整恢复点。

3.4 Kafka：本地恢复与集群资格同时成立

3.4.1 KRaft的元数据不等于业务分区日志

Kafka3.9.0以KRaft作为本书的主要模式。控制器集合通过元数据日志维护集群状态；Broker保存业务分区数据并参与相应复制。进程可以配置不同角色，控制器不必就是承载业务分区的同一个Broker。即使部署在一台机器，也不能据此认为两种日志具有同一职责。

启动首先需要识别集群与节点身份、检查目录和配置，准备本地日志相关资源，并接入当前元数据状态。数据目录格式化建立的身份信息不应在每次普通重启时重新生成，否则就不是把原节点恢复上线。元数据目录与业务日志目录可以有相应配置，本书不假定它们永远在同一路径。

Broker向控制器注册，并持续报告状态。追上所需元数据、本地恢复及准备工作完成以后，再进入允许承担服务职责的状态。KRaft中的fencing限制尚未获得资格的Broker，因而“已经注册”和“已经可以作为正常服务节点”不完全相同。

本地日志恢复要辨认正常关闭留下的标记、恢复位置、日志段与索引。异常退出可能要求扫描未确认完成的部分，识别损坏或不完整尾部，重建必要索引及生产者状态。并不是每次重启都从头完整扫描每个历史段，也不是仅截掉末尾不完整的记录就结束。

分区很多时，目录遍历、打开文件、恢复和索引维护会积累成本。`num.recovery.threads.per.data.dir`提供恢复并行度的配置，但提高线程数也会争用同一存储和CPU，不能保证倍增线程就等比例缩短启动。恢复速度需要在相同数据与退出方式下测量。

3.4.2 网络接收、处理与响应分开，仍会互相施加压力

Kafka网络层把接收连接、网络I/O和请求处理分到不同角色。Acceptor接收连接并分配给Processor，Processor处理连接上的数据与响应，完整请求经过队列交给请求处理线程。处理完成后，响应回到相应网络处理路径。

图3-5保留这个结构，具体线程数取决于监听器及配置，不把图上的数量当作所有部署的固定值。



图示线程职责，不规定全部监听器的固定线程数，也不表示启动只有这四个阶段。

图 3-5 Kafka网络与请求处理：接入、网络I/O与请求处理通过队列衔接。

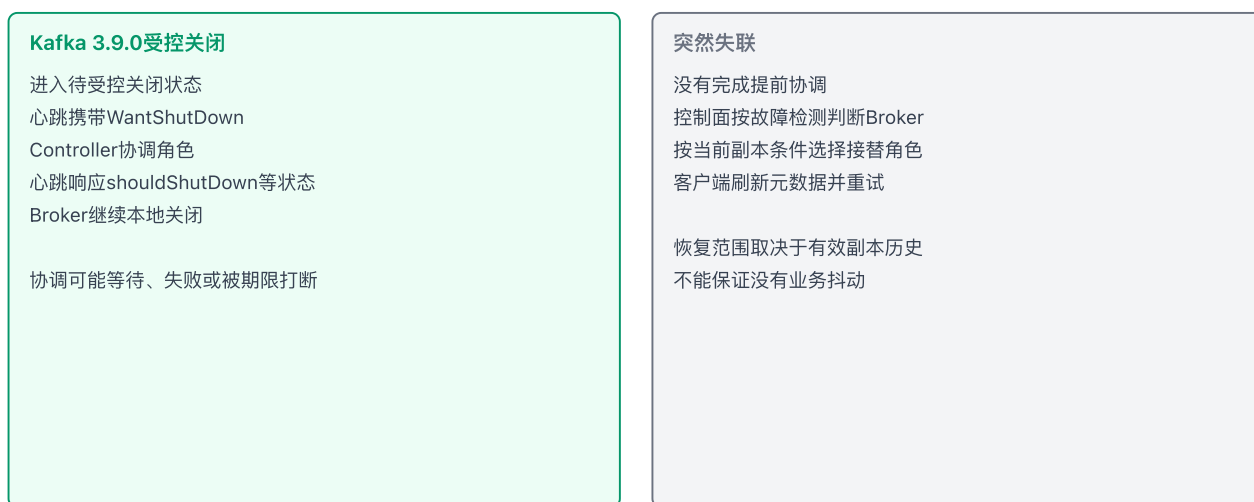
队列使工作可以独立调度，却不会让阶段之间完全不受影响。处理跟不上时，请求队列和内存压力会增加；网络发送慢时，响应也可能积压。队列上限、连接状态和背压共同决定系统怎样限制继续接收。第5章将从职责隔离解释这组线程边界，本章关注它们怎样启动以及怎样停下。

网络资源准备完成以后，仍需考虑权限和相关管理器是否就绪。过早允许处理需要这些依赖的请求，会暴露尚未准备的状态；但先建立监听资源、暂缓处理某些请求可以是合法顺序。关键是放行条件，而不是把“打开端口”固定在所有软件初始化的最后一步。

3.4.3 受控关闭把离开意图传给控制器

正常退出时，Kafka可以通过受控关闭尝试减少Broker离开造成的影响。在KRaft中，运行中的Broker进入待受控关闭状态，并通过发给控制器的心跳表达关闭意图。控制器根据当前集群状态处理相关职责变化，响应中包含是否允许继续关闭的信息。这里不能照搬ZooKeeper模式中固定的RPC时序。

图3-6比较提前协调和突然失联两种情况。



两条路径均需合适接替者；KRaft心跳状态不能替换成ZooKeeper模式固定RPC时序。

图 3-6 KRaft Broker受控关闭与突然失联：前者在退出前表达意图并等待协调，后者依赖故障检测和后续接替。

能够接替的分区副本必须满足相应条件。不是每个副本都可立即接替，也不保证每个Leader都在任意负载下迅速迁移。控制器不可达、缺少合适副本或关闭期限耗尽时，计划中的协调可能无法完成，客户端仍可能遇到重试与短暂不可用。

Broker 生命周期管理的状态、心跳关闭意图和控制器响应处理，可核对 Kafka3.9.0 BrokerLifecycleManager。源码显示的是一组条件与状态转换，不能将其概括为“通知一次，控制器保证全部迁完”。

完成或终止协调后，本地各管理器、网络 and 日志仍需按关闭路径停止，清理资源并保存相应状态。异常终止可能跳过这些工作，因此下次恢复需要更保守地检查日志。受控关闭减少部分故障窗口，并不保证业务永远感受不到Broker离开。

KRaft与ZooKeeper模式的控制通道、超时和重试配置并非可以逐项照搬。调整关闭行为时，应以实际模式和版本核对配置，而不是把历史的 `controlled.shutdown.max.retries` 及退避参数直接作为所有3.x集群的通用建议。

3.4.4 追加日志没有取消事务与附属状态

Kafka无需把每条业务记录恢复成另一套数据库行，这简化了一部分恢复工作。但分区日志还有索引、生产者幂等状态、事务标记、恢复点以及副本历史的关系。格式上的完整批次与业务上可以向某种隔离级别的消费者返回，不是同一个条件。

未提交事务中的记录可以已经存在于日志。采用 `read_committed` 的消费者需要依据事务状态和可见性边界读取，而不能把“字节已完整写入”当成事务已完成。第9章会继续讨论这些状态怎样与复制和消费进度配合。

因此恢复工作多少不能按Redis、Kafka、MySQL排一个固定顺序。一个大型Redis数据集需要大量对象重建，一个分区众多的Kafka节点可能有大量恢复工作，一个干净关闭的MySQL实例又可能很快恢复服务。比较应落到实际留下的工作量。

3.5 容器环境与横向对比：外部期限也是条件

3.5.1 探针分别控制什么

Kubernetes的startup、liveness和readiness探针承担不同职责。startup探针为启动过程提供检测安排，配置后会在成功前抑制其他两类探针；liveness失败达到条件可以触发重启；readiness失败把Pod标记为未就绪，影响匹配Service的流量分配，本身不触发容器重启。探针定义明确区分了这些行为。

如果把“数据库尚在恢复”当作存活失败，编排系统可能不断重启它，使恢复一直无法完成。相反，只用进程存在作为readiness，也可能把流量引到未完成加载的节点。探针应检查对应的条件，不是检查越多越严格就越可靠。

关闭时摘除入口也不证明所有写已停止。已有长连接、其他服务发现路径或直接连接可能仍然存在，在途请求也可能尚未结束。应用和客户端的排空策略需要与编排状态配合，不能据readiness变为false推导“此后没有新写”。

3.5.2 preStop也消耗终止宽限期

容器终止有时间预算。若配置preStop钩子，终止宽限期在钩子执行之前已开始计时；钩子完成以后再向容器发送相应停止信号。不能把宽限期理解为“preStop用多久都可以，随后才开始倒计时”。默认信号之外还可能有镜像或运行时相关约定，应按部署配置确认。容器生命周期钩子说明了这一顺序。

这会直接影响Redis的保存建议。先做一次BGSAVE再NOSAVE退出，只有在业务写入和快照时点之间的关系已经明确、后台保存确实完成且结果可用时，才可能满足目标。readiness摘流不等于写入停止，fork之后仍有写就可能触发写时复制；快照之后新增的写又可能需要其他持久化来源。不能用这组命令保证COW接近零

或保证在期限内退出。

MySQL的清理与Kafka的职责协调也一样。统一写“给120秒”不能说明这段时间对目标系统是否足够：数据量、事务、存储、控制器状态和其他副本决定实际工作。时间预算应来自代表性关闭记录与故障场景，并明确超过期限后的恢复路线，而不是在任意经验值上固定再加几十秒。

3.5.3 三家共有的问题和不同的状态

表3-2按恢复来源、服务资格和退出遗留工作比较三者，不做启动速度排名。

表 3-2 生命周期的共同问题

问题	Redis	MySQL/InnoDB	Kafka3.9.0 KRaft
重建什么	所选持久化来源表达的键空间	页、事务及相关引擎状态	本地分区日志、附属状态与元数据视图
什么不等于就绪	监听端口已存在	已可连接但后台回滚仍有影响	已注册但尚未满足服务资格
正常退出处理什么	保存选择、AOF、副本等待及连接资源	事务、日志和按档位安排的页/维护工作	受控协调、本地管理器与日志清理
集群外部关系	主从、Sentinel或Cluster及客户端	复制、切换入口及读取流量	控制器、分区副本和客户端元数据
异常退出后检查什么	恢复来源完整性与可接受的状态损失	redo/事务恢复与剩余回滚	有效日志、恢复位置、附属状态与角色

区别在于状态种类和恢复协议，不在于某两个产品只影响自己、第三个才影响集群。任何被其他节点和客户端依赖的服务，退出都会改变外部关系。是否能提前协调，要看部署是否提供对应的控制路径。

3.6 架构启示：把退出结果写成可以检查的条件

3.6.1 请求放行的状态条件

一个统一的“启动成功”信号很方便，但我更关心它承诺的范围。完成配置、不再加载数据、获得写入资格，分别排除了不同的风险。若某些恢复仍在后台进行，就应能观察它的进度和对业务的影响，不能让启动日志掩盖后续等待。

日志文本可以帮助定位阶段，自动化却不宜只搜索某句固定英文文案。日志轮转、输出目的地和版本变化都可能影响文本，旧日志还可能留下上次启动记录。服务管理状态、相应协议检查和阶段指标应共同解释当前进程，不能由一条历史“ready”消息单独决定。

3.6.2 提前准备只应覆盖确实必要的工作

Redis预建部分共享对象，MySQL可以在必要恢复后继续后台回滚，Kafka可以并行恢复本地分区。这些安排说明，并非所有状态都必须一次性准备到底，也不是懒加载天然不可靠。决定前置还是延迟，需要看未完成部分是否会破坏当前请求的语义，以及失败是否能够被明确处理。

我倾向于把会改变业务正确性的条件放在放行之前，把可以隔离且可观察的维护留到后台。这个判断依赖具体对象：一段缺失的权威数据不能冒充缓存预热，一项可以从完整来源重建的辅助索引也未必值得阻塞所有服务。划分条件比笼统要求“确定性优先”更接近实现。

3.6.3 关闭请求要有成功、失败与超时三种结果

正常关闭可能因为不能保存而失败，受控关闭可能因为没有合适接替者而等待，编排期限也可能强制结束进程。这些结果都应具有后续处理。收到终止信号并不证明持久化已经完成；进程退出码也不能独自证明所有业务请求都得到响应。

开篇事故之后，我把优雅关闭看得更重，但它不能代替写入时的保证。若业务已经返回成功，就应该按照此前定义的持久化与复制条件承担责任，而不是等计划停机时再补齐。正常关闭降低维护期间的风险，异常恢复则处理正常关闭没有机会完成的情况，两者都需要存在。

3.6.4 一次验证应覆盖具体中断点

验证生命周期时，我会把目标从“反复重启没出错”改成几项可观察结果：恢复前后哪些已确认状态必须相同，哪些未确认请求允许重试，哪些副本有资格接替，以及期限耗尽以后从什么来源恢复。这些都是验证对象，不是测试次数越多就自然获得的结论。

正常退出、进程强杀与整机断电应分别测试，因为它们保留的操作系统和存储状态不同。测试还需要区分空实例与有实际待处理工作的实例。一个没有脏页、没有活跃事务、没有落后副本的节点，退出很快，并不能代表最需要验证的阶段。

3.7 小结

启动建立服务所需的条件，正常关闭处理已经承担的责任，两者都不能由进程或端口状态代替。Redis的加载与保存、InnoDB的页和事务恢复、Kafka的本地状态与集群资格，各有明确的中断点和后续工作。探针与终止期限属于外部条件，不能凭摘流或固定等待时间承诺没有数据损失。下一章进一步检查这些责任中最容易被合并的一项：写入从内存经过日志和磁盘，究竟在哪一步获得了哪种保证。

第4章 内存与磁盘：速度与持久化的平衡

本章导读

我参与过一次选型争论：缓存到底该用Redis还是MySQL？有人提到MySQL的内存表和Query Cache。我花了一下午把两者的访问模式列成表，讨论才从软件名字转向具体工作：哪些数据频繁读取，哪些修改需要事务，命中缓存以后还要执行什么。这里提到的Query Cache是历史背景；MySQL8.0已经移除了这项查询结果缓存，它与InnoDB缓冲池不是同一机制。

那次对照有助于明确分工，但不能把结论扩大为“高频小数据都用Redis，事务数据都用MySQL”。缓存是否可丢、更新怎样传播、容量增长后怎么办，仍然影响决定。即便访问全部命中内存，事务、网络和计算也不会因此没有成本。

本章把几件容易混在一起的事分开：数据在哪里供日常访问，修改在哪里暂存，何时形成持久恢复依据，以及副本确认增加了什么。Redis、MySQL和Kafka都使用内存与磁盘，差别在这些资源分别承担哪项责任。

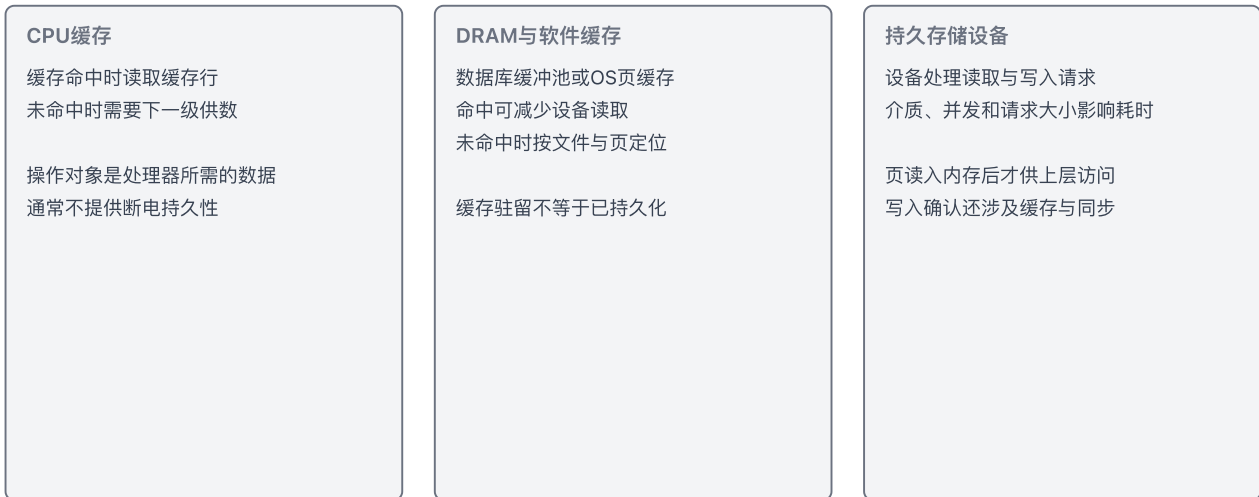
4.1 问题的本质：访问路径与恢复路径同时存在

4.1.1 延迟、吞吐与容量不能放进同一个排名

延迟描述一次操作需要多久，吞吐描述一段时间完成多少工作。一个设备可以通过并发取得较高吞吐，同时使排队中的单次请求等待更久。把SSD顺序写的GB/s与内存随机读取的微秒放在一起，不能据此判断哪一种介质“更快”。块大小、并发、读写方向和持久化要求必须相同，数字才有比较意义。

DRAM、处理器缓存和持久设备具有不同的访问和保存特征。处理器缓存未命中可能访问DRAM，数据不在内存缓存时又可能需要存储I/O。机械盘还需要寻道和旋转，SSD没有这些机械动作，却仍受控制器、闪存管理、队列和写放大影响。顺序访问的收益在两类设备上并不完全相同。

图4-1用访问路径而非无条件硬件排名表示这种关系。



延迟比较须保持操作类型、块大小和并发口径一致；不把硬件量级当成应用延迟。

图 4-1 一次读取中的缓存与持久存储：缓存命中减少后续访问，实际延迟取决于同口径的负载与设备。

缓存命中所避免的是某段工作，不一定是整个请求。数据库即使命中页，仍可能执行过滤、锁检查和排序；Redis找到键以后，还要处理命令与发送结果；Kafka命中页缓存以后，消费者的业务计算也不由存储系统完成。分析性能时，需要先识别真正占用时间的路径。

4.1.2 用一个假设计算说明命中率的影响

下面的数值仅用于算例，不是任何硬件或软件的实测。假定一次页访问命中时耗时5微秒，未命中并读入所需页时耗时500微秒，暂时忽略排队和其他计算。命中率为99%时，平均页访问时间是：

$$0.99 \times 5 + 0.01 \times 500 = 9.95 \text{ 微秒}$$

如果命中率变成90%，同一模型中的平均值变为54.5微秒。两个场景的存储设备没有变化，访问落到设备上的比例变了。这说明工作集和缓存容量可以显著影响平均成本，不能只拿设备最快的一次访问解释应用响应。

这个算例仍不能推出请求的P99。一个请求可能访问很多页，各次命中并非独立；热点锁、并发排队和结果传输也会改变尾部延迟。平均命中率很高，不表示最重要的业务对象一定命中，更不表示某次大扫描不会影响其他请求。

工作集指一段时间内被反复访问的数据，大小可以远小于总数据量，也可能接近总量。数据库能保存超过内存的数据，并不意味着它假设“大部分页在任何时刻都不在内存”。小数据库可以完全被缓存，大数据库也可能只有一小部分频繁访问。容量规划需要描述访问分布，不能只拿总数据量除以内存大小。

4.1.3 易失缓冲、持久副本与确认分别是什么

程序修改内存以后，可以把数据写入文件。通常的缓冲I/O先把内容交给操作系统页缓存，写调用返回并不等于设备已经持久保存。fsync 等接口用于请求按相应语义把数据同步到持久存储；设备和文件系统也必须兑现其约定。这是本地存储的边界。

复制把内容送到另一个节点，增加了恢复来源，但接收端可能也只是写入自己的易失缓冲。另一个节点收到、刷盘和应用修改是三种不同状态。若要抵御机器故障，还需要说明副本位于哪些故障域、接替选择什么历史，不能仅根据“有两个副本”作结论。

表4-1先固定本章使用的几个动作。

表 4-1 几个容易混淆的写入动作

动作	直接建立的事实	不能直接推出
修改进程内存	当前进程得到新状态	重启以后仍能恢复
写入操作系统缓冲	内核接受了相关文件内容	整机掉电后一定保留
按协议完成本地刷盘	有相应本地持久恢复依据	任意副本已经获得或应用
收到副本确认	达到该复制协议定义的确认位置	接收端已刷盘、业务效果已完成
向客户端返回成功	接口按约定完成了处理	客户端一定收到响应，外部系统也成功

后面的参数各控制表中的一部分。把它们排成一条“弱到强”的统一刻度，会丢失它们究竟保证什么。

4.2 Redis：主要数据集驻留内存，持久化另有路径

4.2.1 内存占用不只是键和值的业务长度

Redis的常规数据访问使用内存中的对象和结构。第2章的SDS、字典、跳表、listpack和intset不仅决定操作，也决定数据量相同的实例需要多少内存。许多很小的键会反复承担对象、指针与分配器开销；较大集合可能跨过编码转换条件，改变空间分布。

总占用还包括客户端缓冲、复制或AOF相关缓冲、后台任务和分配器保留的空间。设置 maxmemory 不等于整个进程或容器从此不可能超过这个字节数。部分缓冲不计入淘汰判断，后台保存期间也可能增加写时复制占用。容量边界需要覆盖这些工作，不能把内存全额分给业务数据。

在数据被当作缓存时，可以定义可淘汰范围；若数据是唯一的业务记录，简单淘汰会直接改变事实。因而“内存满了怎么办”是数据责任问题，不能只按命中率选择一个听起来更先进的算法。

4.2.2 近似LRU与LFU怎样选择候选

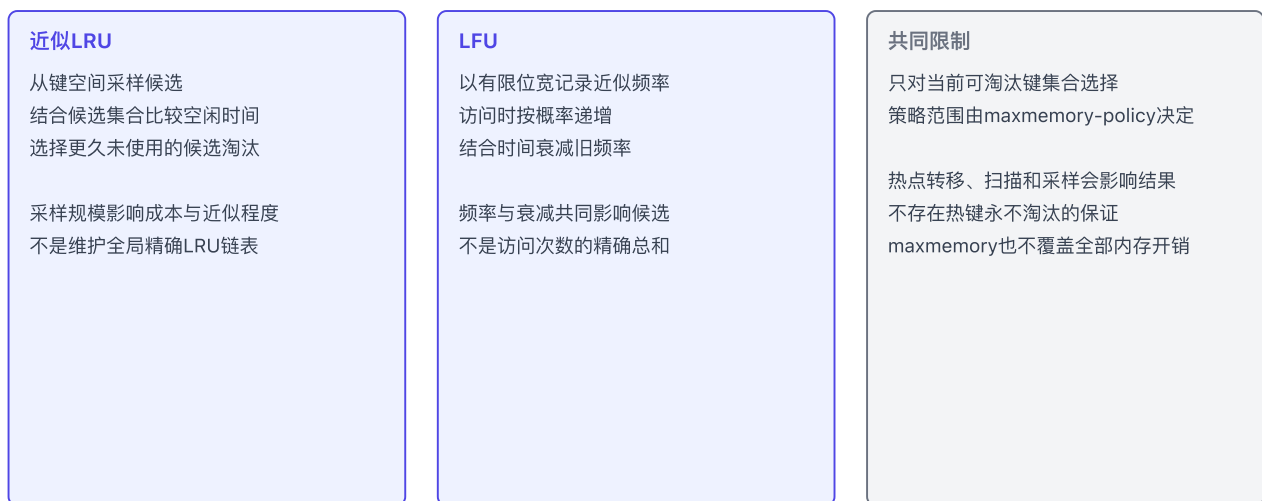
严格LRU（最近最少使用，Least Recently Used）需要维护访问顺序。Redis的近似LRU通过采样和候选集合，寻找较久未访问的键，减少持续维护完整全局顺序的成本。`maxmemory-samples` 控制采样数量，7.x常见默认值为5；算法还保留较好的淘汰候选，不是每次独立抽五个键后立刻任意淘汰一个。

增加样本可能使候选更接近全局较久未用的对象，也增加检查成本。近似算法没有保证每次都找到真正最久未用的键。它在这里追求足够有效的候选，而不是为了满足LRU定义支付全部维护成本。Redis淘汰说明解释了采样与候选集合。

LFU（最不经常使用，Least Frequently Used）考虑访问频率，同时让历史频率随时间衰减。Redis使用有限位宽的概率计数，不逐次保存精确访问次数；较高计数继续增加的概率更低，使有限空间能区分更宽的热度范围。计数的含义因此是用于比较的近似值，不是业务请求统计。

衰减避免一个过去很热、现在不再使用的键永久占据优势。`lfu-log-factor` 影响计数增长，`lfu-decay-time` 影响衰减时间尺度；相关路径在访问或评估时处理衰减，不需要为每个键配置一个定时器。热度变化比衰减更快时，旧热点仍可能占用空间，算法不能直接识别业务变化。

图4-2比较两种选择依据。



图示淘汰判断：释放对象、后台释放和请求延迟仍需分别分析。

图 4-2 Redis近似LRU与带衰减的LFU：前者比较近期使用，后者使用近似频率与衰减信息。

一次扫描把许多冷键访问一遍时，单看最近访问容易把它们视为新近使用；频率信息可能帮助区分反复访问的键与一次性访问的键。但LFU也不能保证热键绝不淘汰，采样、容量、访问分布和衰减都会影响结果。判断策略收益需要比较相同工作负载下的命中与淘汰，而不是只给出一个算法名称。

4.2.3 淘汰策略定义的是可删除范围

Redis7.x的八种常见淘汰策略可以按范围理解。`allkeys-lru`、`allkeys-lfu`、`allkeys-random`从全部键中选择；`volatile-lru`、`volatile-lfu`、`volatile-random`、`volatile-ttl`只考虑设置过期时间的键；`noeviction`不通过淘汰释放空间，对受到内存限制的写入返回错误。

`noeviction`只表达不因该策略主动淘汰，不能保证数据不因过期、删除、故障或配置变化而丢失。`volatile`策略也不是一个持久性标记：未设TTL的键不在它的候选集合，不等于这些键已刷盘。若可淘汰键不足，内存压力仍会影响新的写入。

过期和内存淘汰还应区别。TTL表达数据在时间上的有效期，淘汰表达资源不足时可以放弃什么。一个被过期删除的会话可能是业务规定的行为，一个被内存压力删除的会话则可能导致用户重新登录。两者都表现为键不存在，但应对方式和服务指标不同。

对缓存而言，删除之后的回源能力是设计的一部分。若大量键同时失效，后端是否能够承受重建流量？若原始状态早已清理，缓存是否真的可重建？仅说“缓存可丢”会忽略恢复来源和恢复速度，这个判断必须落到实际数据。

4.2.4 RDB：快照时点、fork与写时复制

RDB把某一时点的键空间序列化为快照。通过BGSAVE生成时，Redis使用fork创建子进程；父子最初共享内存页，子进程按照自己的视图写文件，父进程继续处理请求。父进程后续修改页面时，操作系统通过写时复制（COW，Copy-on-Write）隔离需要分开的内容。

fork并不在开始时复制全部数据页，却要处理页表及相关进程状态，父进程在这段工作中不能照常执行命令。后台生成也仍与前台争用CPU、内存和I/O。不能因为主要序列化发生在子进程，就把整个快照过程称为不阻塞或没有前台影响。

COW成本与快照期间实际修改的页面有关，而不只与修改的业务字节有关。多次修改同一个已复制页，与分散修改许多不同页，可能产生不同的额外占用。快照持续越久，就有更长时间累积写入；是否启用透明大页、分配器行为等也会影响表现。不能仅按实例容量推算一个固定毫秒数。

快照文件完成与原子替换帮助避免把半成品直接当作当前RDB，但不代表任何文件系统或设备故障都被覆盖。恢复的是快照代表的状态，后续写若没有其他持久化来源，就不在这份快照中。第3章正常关闭保存可以减少某些计划停机的缺口，不能替代运行期间对成功写的约定。

RDB适合需要完整状态副本和可搬运恢复文件的场景。它的价值既有恢复效率，也有备份用途；二者仍不完全相同。只有本机最新快照，无法处理设备整体丢失，也未必能回到误删除之前的时间。保留多个时间点和异地副本，需要另外安排。

4.2.5 AOF：执行记录与刷盘策略

AOF保存用于重建状态的写操作记录。通常先执行影响数据的操作，再把需要传播的写记录放入AOF相关缓冲，经过文件写入与刷盘形成恢复依据。它不等于InnoDB的WAL过程，也不是审计意义上永久保留所有原始请求：传播命令可能经过改写，AOF重写还会压缩历史。

`appendfsync` 控制AOF同步策略，只有AOF已启用才有对应作用。Redis7.x示例配置默认 `appendonly no`，因此不能看到 `appendfsync everysec` 就认定实例已经持久记录每次写。

表 4-2 AOF刷盘策略及其边界

策略	主要安排	需要理解的代价与边界
<code>always</code>	AOF写入后按每次同步策略等待刷盘，处理可以合并工作	增加持久化等待；不应机械理解成每条命令独占一次系统调用
<code>everysec</code>	通常由后台周期性执行同步	常以约一秒说明风险窗口，调度与存储异常下不是严格的一秒上限
<code>no</code>	Redis不主动为这条策略执行周期同步，依赖系统写回	本地掉电前未持久化的内容可能更多；不能据此保证吞吐一定最高

缓冲文件写入也可能受到存储压力和系统回写影响，“不主动fsync”并不等于主线程永不等待I/O。评估时要 把正常响应、后台同步与异常存储状态一同考虑。机制与策略见Redis持久化说明。

Redis7.2新增 `WAITAOF`，可以对当前连接此前的写等待规定数量的本地及副本AOF同步确认。它与等待副本处理进度的 `WAIT` 不同，也不能仅因返回了某个数量就把整个集群称为线性一致系统。超时、返回值、部署和切换条件仍要解释。`WAITAOF`定义标明了小版本和作用范围。

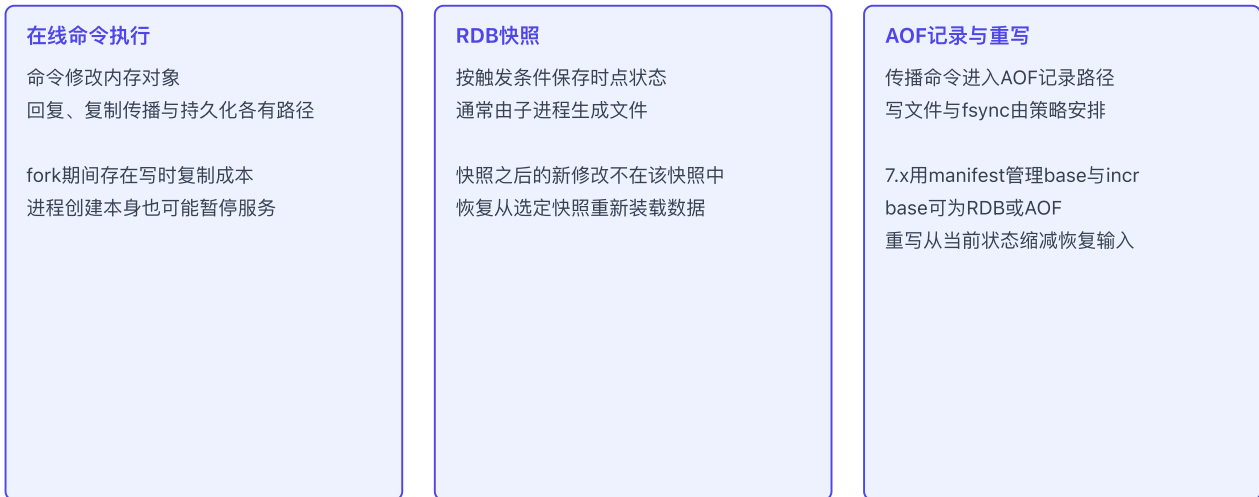
4.2.6 重写减少历史，持久化仍按完成时点判断

一个计数器增加很多次以后，恢复当前值不必再保留每次增加。AOF重写从当前内存状态生成新的基础表示，保留恢复所需的信息，减少重复历史带来的文件和重放成本。新基础文件完成持久化以后，可以成为当前状态的恢复来源；它无法追回已经丢失的修改，也不能倒推先前成功响应时就已满足同样条件。

Redis7.x使用多部分AOF，`base`表达重写得到的基础状态，`incr`保存后续变更，`manifest`确定有效文件集合。`base`默认可以采用RDB格式，也可以配置为命令格式。重写期间的新写入进入相应增量文件，新基础文件和清单准备好后再切换有效集合。具体文件布局与原子切换留到第8章。

旧单文件AOF也有维持重写期间可恢复性的设计，因此多部分AOF的价值不能描述成“以前一重写就会破坏旧文件，现在才安全”。改造主要改变增量管理、重写过程开销和文件组织；版本不同，成本发生的位置不同。

图4-3区分日常内存状态、快照和AOF恢复路径。



RDB和AOF不是每条命令必须串行执行的两步；确认与故障后保留范围取决于具体策略。

图 4-3 Redis内存状态与持久化路径：RDB保存时点状态，多部分AOF由base与增量共同表达恢复内容。

主要数据访问不按需从RDB或AOF加载单个键，但持久化I/O仍可能影响请求执行。由此得到的判断应是：Redis把常规对象访问留在内存，把恢复状态另行保存；它是否能够承担某类权威状态，还要结合确认、故障与副本语义判断，而不是预先接受一个固定秒级丢失窗口。

4.3 MySQL：页可以晚写回，不能缺少恢复依据

4.3.1 缓冲池既缓存读取，也保存尚未写回的修改

InnoDB缓冲池保存正在使用的页。查询需要某页时，先检查它是否在内存；不在时，再读取并纳入管理。修改页以后，内存中的版本与数据文件不同，成为脏页。缓冲池因此不只是一个可随时删除的只读副本集合，还保存等待写回的修改。

管理这些页需要几种相互关联的组织。Free List提供空闲页；LRU List组织已使用页的替换候选；Flush List跟踪需要写回的脏页及其修改位置。同一脏页可以同时出现在LRU和Flush相关结构中，不能把它们画成三个互斥的内存分区。

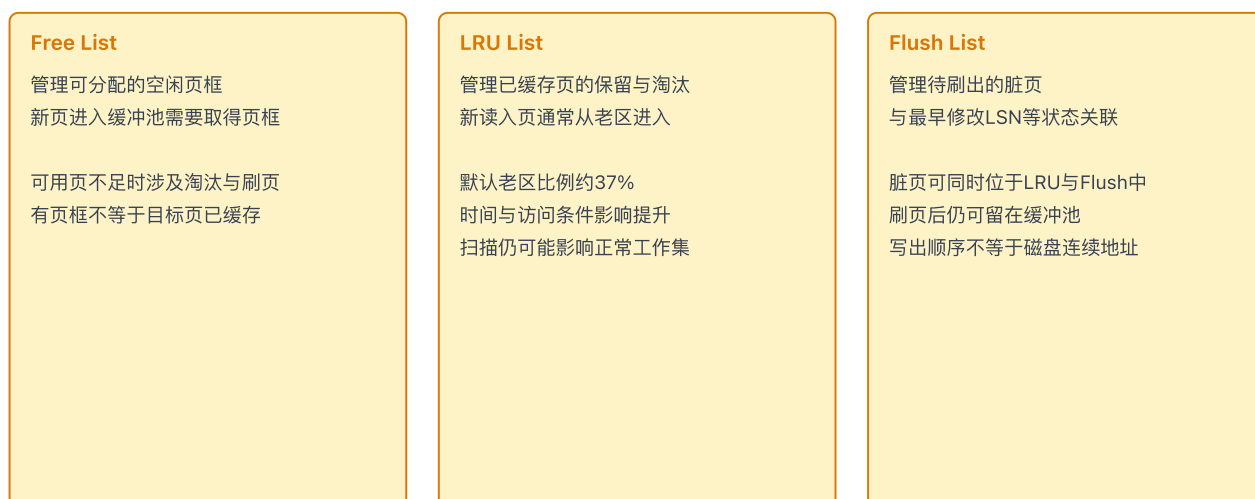
替换一个干净页，可以在后续需要时重新读取；替换脏页则先要满足写回与日志约束。若持续写入使可用干净页不足，前台可能等待页面回收，缓冲池越大也不表示永远不会出现这类等待。观察命中率之外，还需要观察脏页和刷新进度。

4.3.2 新老分区降低扫描影响，但不保证热页不受影响

朴素LRU把最新访问视为近期有用。一次性扫描会把许多页提升，可能替换原有热点。InnoDB采用中点插入：新读入的页进入旧区附近，只有满足后续访问条件才提升，给一次性访问与反复访问留下区别。

`innodb_old_blocks_pct` 控制旧区比例，8.0默认值为37；`innodb_old_blocks_time` 控制相关的提升时间条件，默认1000毫秒。时间从首次访问等规定时点参与判断，不是“页只要待满一秒就自动变热”。具体行为见缓冲池抗扫描说明。

图4-4同时表现管理链表与LRU内部的新老区域。



三条链表描述管理关系，不是把缓冲池切成三个互不相交的内存分区。

图 4-4 InnoDB缓冲池的页管理关系：LRU的新老区域与Free、Flush管理结构不是同一层分类。

中点插入降低了扫描页立即占据热区的机会，并未取消扫描的I/O和CPU成本。扫描仍可能持续很久，也可能重复访问足以满足提升条件；真正工作集超过内存时，热点之间也会竞争。减少缓存污染，并不意味着热数据完全不受影响。

缓冲池启动后的状态也不只有空和热两种。MySQL支持保存与加载部分缓冲池内容的页标识，用来预热；它不等于把整个进程内存快照恢复。重启后页是否已加载、操作系统页缓存是否还在、是否采用直接I/O，都会改变预热过程。第3章的就绪并不承诺工作集已完全进入缓冲池。

4.3.3 WAL限制的是持久化先后，不是内存修改先后

WAL（预写日志，Write-Ahead Log）的关键约束是：数据页写入持久存储之前，能够恢复对应修改的日志必须先满足持久化要求。事务可以先修改缓冲池中的页，并产生redo记录，随后按提交与刷新协议把日志和页写出去。把WAL写成“日志先落盘，然后才允许修改内存页”，会误解写入过程。

这样安排，可以不让每次提交都等待所有被修改的数据页分别写回。事务需要的恢复信息进入顺序日志，多个脏页以后按刷新安排逐步处理；同一页在此期间被多次修改，也可能合并成后续一次写回。收益是提交路径与全部数据页写回解耦，不是永久消除了随机写。

redo有有限容量，需要checkpoint推进释放可复用空间。若脏页长期无法写回，恢复就仍可能需要旧日志，日志空间压力最终影响前台。增加日志容量可以提供更大余量，却不能在存储长期跟不上时无限吸收写入。缓冲是时间上的调节，不是持续吞吐能力的替代。

图4-5把提交所依赖的日志同步与后续数据页写回分成两条相连路径。



两条路径不以对方完成为前提；WAL约束日志与页的持久化次序，undo处理需要撤销的事务修改。

图 4-5 InnoDB提交与数据页写回：内存修改产生日志，提交等待所需日志条件，脏页在满足WAL约束后写回。

日志记录怎样表达页修改，第8章会说明。本章要建立的是先后约束：恢复所需信息已经可靠保存以后，数据页可以晚一点到达最终位置；如果数据页先出现新状态而恢复日志缺失，恢复就可能无法解释这份状态。

4.3.4 redo与binlog分别控制，组提交可以摊薄同步成本

`innodb_flush_log_at_trx_commit=1` 要求事务提交时写出并同步相应redo；为0或2时，提交与周期写出、同步的安排不同。后台同步周期还受 `innodb_flush_log_at_timeout` 和调度影响，因此“约一秒”不能作为所有故障中的严格损失上限。

表 4-3 InnoDB redo提交策略

配置值	提交时与周期工作的关系	主要风险边界
1	提交要求相应日志写出并同步	依赖文件系统和设备兑现持久化语义；不自动覆盖远程副本
2	提交写出日志，同步主要由周期任务承担	内核与设备中尚未持久化内容在相应故障中可能失去

配置值	提交时与周期工作的关系	主要风险边界
0	提交不要求立即完成日志写出，主要周期性写出与同步	进程内日志缓冲也可能尚未交给内核

这里描述的是提交需要达到的条件，不代表每笔事务单独执行一次物理同步。组提交允许若干事务共享一部分日志同步工作，在保留各自提交条件的同时摊薄成本。决定刷盘策略之前，应先分清安全条件与实现如何批量满足条件，不能用“一笔事务一次fsync”的简单模型估计全部吞吐。

MySQL服务层binlog有自己的 `sync_binlog` 设置。常说的“双1”指它与 `innodb_flush_log_at_trx_commit` 均为1，用于加强相应的本地日志持久性。两份日志还要经内部提交协议协调，避免崩溃恢复时引擎提交事实与binlog事实无法对齐。官方分别定义redo提交策略和binlog同步选项。

双1不是跨节点“不丢”的同义词。主节点日志已同步，但异步从节点尚未获得该事务时发生切换，新主的历史可能不包含它。半同步增加等待条件，组复制又有不同协议；第7、9章将分别讨论。这里先保留一个明确结论：本地恢复依据与故障切换后可用的恢复依据必须分开。

4.3.5 doublewrite保护页完整性，redo解释页修改

页写入不一定具有与页大小相同的原子性。默认16KB页写到一半遇到故障，可能留下部分新内容和部分旧内容。redo不等于每次都保存整页镜像；若需要的基础页已经损坏，仅靠增量修改记录未必能够正确重建。

doublewrite先保存一份用于恢复的完整页内容，再写入最终表空间位置。恢复发现目标页不完整时，可以使用完整副本补足基础页，然后按照恢复规则处理日志。这不是让最终页从此不会撕裂，而是为可能的撕裂提供恢复来源。

MySQL8.0.20起，doublewrite使用独立文件组织相关内容，早期安排位于系统表空间。它增加写入量和同步工作，是否便宜取决于设备与负载，不能仅因部分顺序写就视为几乎零成本。不同小版本还有相关模式和配置，具体范围见doublewrite说明。

页完整性、事务持久性与副本接替再次分开了。doublewrite不能保存从未写出的redo，也不能让落后从节点自动拥有新事务；redo刷盘也不能替代一切完整页恢复手段。每项机制处理一个特定缺口，它们共同出现不是重复建设。

4.3.6 change buffer把部分二级索引工作推迟

修改一个当前不在缓冲池的二级索引页，可能先触发随机读取。change buffer记录符合条件的修改，在页随后读入或后台合并时应用，减少立即把目标页读取进来的要求。它有适用限制，例如唯一性检查和某些降序索引条件会影响是否能够使用，不能说所有二级索引修改都进入这里。

这份缓冲也不是只存在于进程内存。在内存中它占用缓冲池的一部分，在磁盘上有系统表空间中的对应组织，使未合并工作能够跨越事务和重启继续存在。相关持久化与恢复机制必须保护这些延迟修改，否则“先缓一缓”就会成为漏更新。change buffer说明列出了范围及合并时机。

合并可以聚合某些访问，却也可能在后续引入I/O。持续产生而无法及时合并的工作，会转化为新的资源压力。第3章快速关闭允许跳过完整合并，并不是认定这些工作没有必要，而是允许它们按恢复协议继续存在。

第2章的自适应哈希索引则是另一类辅助状态：它减少部分热点搜索的重复路径，可以配置是否启用，并有维护和争用成本。change buffer改变写入安排，辅助哈希改变读取定位，doublewrite应对页损坏。三者不宜因为都称“缓冲”或“索引”就按同一种缓存理解。

4.4 Kafka：日志访问利用页缓存，确认仍由协议定义

4.4.1 顺序追加有局部范围，不代表整块设备只有顺序I/O

Kafka把生产记录追加到分区活跃日志段。日志段按大小、时间等条件滚动，旧段根据保留或清理策略处理。一个分区的主要写入方向因此是向末尾追加，而不是按业务键频繁原地修改任意位置。

同一Broker却可能同时承载很多分区，还有索引、检查点、清理、复制读取等工作。多个顺序文件在同一设备上交错，不等于设备从头到尾只接收一条连续写流。compaction也会读取与重写内容。顺序追加减少了一类访问成本，不能推出写路径上完全没有其他I/O。

消费者常沿offset前进，但不同消费者的位置可以相距很远，也可以主动从历史位置读取。最近追加的数据可能仍在页缓存，较旧数据可能需要磁盘或远程存储。Kafka没有按任意业务字段提供通用数据库索引，不等于完全不能跳到某个offset附近读取。

4.4.2 页缓存让文件内容在内核中复用

Kafka的数据文件访问利用操作系统页缓存。写入先形成内核中的脏页，读取可以命中相应文件页；当数据已经在缓存中，消费者不必每次从设备重新读取。相同文件内容也不必为了常规读取再在JVM中维护一份完整数据库式缓冲池。

这不表示Kafka没有内存管理。生产者批次缓冲、网络接收发送、请求队列、元数据、索引和各种管理状态都需要内存。JVM堆与页缓存还会竞争机器资源，容器内存限制也可能改变可用空间。把堆调得很大，不一定为日志读取留下更多可用缓存。

进程重启而操作系统未重启时，原文件的某些页可能继续留在内核缓存，这能减少重新读取。但它们随时可能因为内存压力被回收；文件重新建立、其他负载或整机重启都会改变结果。因此“Kafka重启后缓存必在，立即按内存速度服务”不是保证。第3章还说明了独立于缓存冷热的日志与集群状态恢复工作。

4.4.3 回写阈值限制脏数据，不是为Kafka预留固定缓冲区

Linux根据脏页数量和相关阈值推进后台写回，在压力较大时也使产生写入的进程参与或等待写回。`vm.dirty_background_ratio`、`vm.dirty_ratio`与对应的字节阈值是系统配置，不是Kafka协议参数，实际值应从运行环境读取。

比例的分母也不能直接当作物理内存总量。内核文档说明，它与包含空闲页和可回收页的可用内存相关，并不等于整机内存。因此“64GB机器乘10%就有6.4GB属于Kafka的异步缓冲”既省略了共享负载，也使用了错误分母。Linux虚拟内存参数定义了这些口径。

缓冲允许短时间内以高于设备持续写入能力的速度接受内容，脏页逐渐累积后，最终仍要受实际写回能力约束。一次很短的吞吐测试可能主要测到写入缓冲的能力，而不是持续落盘能力。延长测试、观察脏页和设备写入，是为了判断这部分差异，不是为了从某个峰值推出整机容量。

4.4.4 零拷贝减少的是部分数据经过用户态的次数

常规文件发送可能先把文件内容读入用户态缓冲，再交给套接字发送。使用 `sendfile` 或 Java NIO的 `transferTo` 等适用路径，可以让文件内容在内核管理下发送，减少用户态中转和相应复制。

图4-6比较普通中转与适用的文件发送路径。



Kafka 3.9.0使用的发送路径受连接和数据格式影响；页缓存命中不是sendfile成立的必要条件。

图 4-6 Kafka文件发送与用户态中转：适用路径避免内容进入用户态再复制发送，实际复制和DMA步骤依实现而异。

“零拷贝”是这类优化的通用名称，不意味着物理上没有任何数据移动，也不能把上下文切换数固定成所有平台都相同的四次或两次。若数据尚未在页缓存，仍需取得文件内容；加密或格式转换需要处理内容时，路径也可能变化。优化降低了某段成本，没有免除读取冷数据或执行加密所需的工作。

Kafka批次布局使多条记录能够作为连续内容传递，进一步减少逐条处理固定开销。但为减少复制而安排格式，也要考虑校验、版本和消费者需求。第2章已经说明消费者API可以逐条交付记录，网络批量与业务处理单位无需相同。

4.4.5 本地刷新与acks分别回答不同问题

Kafka通常不要求每次生产写入都先执行本地 `fsync` 才返回，常规路径利用操作系统写回，并通过副本机制增加可恢复性。日志刷新参数可以改变本地同步安排，但不会替代副本协议。开启更频繁刷新可能增加等待，影响幅度需要实测，缺少测量依据，就无法给出固定的下降比例。

`acks=0` 不等待Broker确认，生产者甚至不能由成功发送推断服务端已经保存；`acks=1` 等待Leader本地日志接受相应记录，不等待完整ISR确认；`acks=all` 要求按ISR协议等待相应副本确认，并与 `min.insync.replicas` 配合约束可接受写入的副本条件。Kafka3.9生产者默认 `acks=all`，配置冲突还会影响幂等设置。生产者配置给出了精确定义。

这里的日志接受不等于各节点强制刷盘。多节点仍存活时，可以利用它们的记录恢复一个失败节点；若相关节点同时掉电且记录均未持久化，复制确认不能使未持久化的数据保留在磁盘上。`acks` 因而不与Redis的 `appendfsync` 或MySQL的redo刷盘参数逐档对应。

反过来，某一个节点的文件已经刷盘，也不说明故障切换会选择它的历史。选主规则和日志对齐决定哪些记录被继续保留。第7章解释资格，第9章解释历史；本章只确认本地持久化与分布式确认的职责区别。

4.4.6 日志之外还有需要恢复的辅助状态

一个日志段周围有offset索引、时间索引，以及某些与生产者、事务和恢复有关的文件。offset索引是稀疏映射，用来先定位附近，再扫描到目标；时间索引帮助按时间取得适当的起始位置。它们不需要为每条记录都建立数据库式索引，也不意味着二者必定在完全相同的时刻追加。

表 4-4 Kafka日志及辅助状态的职责

状态	作用	与恢复的关系
分区日志记录	保存按offset组织的内容	是读取和副本对齐的重要依据
offset与时间索引	缩小文件定位范围	需要与对应日志一致，部分损坏可按机制重建
生产者状态	支持幂等与相关检查	可由快照配合保留日志恢复，不能忽略保留边界
事务和恢复位置等状态	限定可见性与恢复扫描	不能仅凭日志文件存在就认定全部可用

本表只划分职责，不重复第8章的字节布局。是否能够重建某项状态，取决于所需记录和辅助信息是否还在；日志保留策略与恢复要求因此有关。所谓“数据就是日志”，并不表示日志以外没有需要管理的事实。

4.5 横向对比：不同故障需要不同恢复来源

4.5.1 先比较访问和保存责任

表4-5以同样的问题比较三者。磁盘文件、内存缓存和副本都可以同时存在，不能只归为“谁以谁为主”。

表 4-5 访问路径与持久化责任

问题	Redis	MySQL/InnoDB	Kafka
常规数据访问	内存对象与结构操作	缓冲池页，未命中时读取	日志读取利用内核页缓存
内存不足时主要约束	淘汰范围、拒绝写入、对象与缓冲占用	页替换、脏页刷新、工作集中	页缓存回收、堆与缓存竞争、冷读
本地恢复依据	所选RDB/AOF文件集合	数据页、redo、undo及提交相关状态	日志、索引与相关状态文件
日常写入的延迟安排	AOF策略和后台任务影响请求	提交等待日志条件，页可以后写	批量追加与系统回写，副本确认另定义
不能省略的外部条件	数据是否可重建、副本与切换	binlog、复制模式、读取和切换	ISR、选主、保留和消费状态

这张表不足以直接选型，却能指出比较时需要补哪种事实。若缺少本地恢复来源，增加缓存命中率没有帮助；若瓶颈是热点锁，增加设备顺序吞吐也未必解决。指标和机制要对准同一个问题。

4.5.2 按故障区分“已写入”

考虑一个明确的假设：系统已经向客户端返回成功，但最新修改还只在某一层易失缓冲中。若只结束应用进程，操作系统仍然运行，内核页缓存可能继续存在；如果整机掉电，内核状态也失去。二者对尚未同步的文件内容产生的风险不同，因此不能用进程强杀测试代替掉电测试。

再考虑主节点失联而从节点继续服务。即便原主本地日志完整，接替者也可能还没有这次修改；反过来，原主本地尚未刷盘，存活副本可能仍保有足够记录。恢复结果取决于选主与复制协议，而不是单看原主的磁盘。

最后是逻辑删除或错误写入。正常复制和持久化可能忠实保存并传播这次错误。副本不是历史备份，刷盘也不是误操作撤销机制。要回到更早时点，需要相应备份、保留日志与恢复流程。这些需求不能全部归入“数据不丢”四个字。

表 4-6 故障与需要检查的证据

故障	主要检查	容易误用的结论
进程异常结束	哪些内容已交给内核，恢复文件与状态是否完整	强杀不丢，所以掉电也不丢

故障	主要检查	容易误用的结论
整机掉电	哪些内容已达到持久存储，设备是否兑现同步语义	文件写返回，所以数据已永久保存
主节点切换	哪些副本拥有哪段历史，谁有资格接替	原主双1，所以新主一定不缺事务
错误写入被复制	历史备份与保留记录能否恢复正确时点	副本多，所以可以回到任意过去

这些条件对应选择架构时实际要作的决定。不同业务可以接受不同结果，前提是成功语义和恢复责任事先清楚。

4.5.3 回到那次缓存争论

查询命中InnoDB缓冲池，减少了页读取，但仍执行数据库语义；Redis缓存结果，可能减少整个查询的计算，却增加结果如何更新的问题。两种缓存对象不同，不能仅根据都用内存就认为功能重叠。

如果某结果很难失效或更新成本很高，单纯减少读取延迟可能不足以抵偿一致性维护。若结果可以从明确正本重新计算，且允许有限更新延迟，缓存就可能合适。MySQL8.0移除旧Query Cache，不等于所有查询结果缓存都不应该存在；那项历史机制的局限需要在具体并发与失效安排下解释，第5章会继续讨论。

我现在更看重这类比较最后是否说明了职责。缓存中存什么，错了或没了从哪里恢复，重新生成期间请求怎样处理，才决定引入第二个系统是否值得。访问模式是起点，但还不能单独决定完整方案。

4.6 架构启示：把性能收益与恢复责任一起说明

4.6.1 我会先确定被加速的对象

缓存一页文件、缓存一条业务记录、缓存一次查询结果，失效条件和恢复方式不同。InnoDB需要检查页是否脏，Redis淘汰需要确定哪些键可以删除，Kafka页缓存由内核按文件页管理。它们的区别不只是缓存位于哪一层，而是这一层掌握哪些语义。

因此，我在决定自己管理缓存之前，会先问它必须掌握什么信息。如果只需要复用文件内容，已有页缓存可能足够；如果要控制业务对象失效、隔离脏状态或避免特定访问污染，就需要相应机制。多一份用户态缓存还会占用空间和增加同步责任，不能仅因可定制就视为收益。

4.6.2 我不会把刷盘和复制确认放在一条强弱轴上

这两项机制可以共同提高某些故障下的恢复能力，但保护的状态不同。本地日志刷盘不能保证接替者已有日志，副本收到内容也不能保证相关机器同时掉电后的保留。选择参数时，我会先写明不能接受哪种失败，再确定需要哪些完成条件。

条件写清以后，仍可以对代价有明确判断。例如，一项业务不能丢失已确认的权威记录，就不能为了平均响应更低而悄悄改成未满足所需持久化条件即返回成功。若选择允许窗口，窗口应成为业务约定和恢复设计的一部分，不是藏在默认配置里。

4.6.3 批量和顺序写是优化手段，不是通用保证

Redis AOF集中保存写操作，InnoDB redo把提交与全部页写回分开，Kafka让事件主要沿分区追加。三者都利用了追加或批量，但没有消除所有后续工作。AOF要重写，InnoDB要刷新页，Kafka要管理索引、保留和清理。

我会同时检查前台节省的工作与后台积累的工作。如果正常流量下后台长期追不上，短时低延迟只是缓冲尚未耗尽。把这种状态解释为架构高效，容易在容量增长或维护时才暴露问题。持续运行和恢复时间应当进入同一份评估。

4.6.4 重建能力必须包含来源、时间和副作用

“有正本就能重建缓存”只说明内容可能可计算，尚未说明正本能否承受全量读取、重建需要多久，以及期间业务是否继续写入。重建期间如果读到不同时点的数据，还需要界定目标视图。派生状态可以不做同等级持久化，但不能缺少重建协议。

我会把权威数据、派生数据和处理进度分别标记。原始消息存在，不一定能推断哪些外部动作已经完成；数据库行在，也不一定包含丢失的消费进度。第9章讨论同步位置，第10章会把这种区别落实到重试和事件消费。

4.7 小结

内存与磁盘的分工同时影响访问成本和故障恢复，不能以“全内存”“磁盘为主”替代具体路径。Redis为内存对象另行保存恢复状态，InnoDB通过日志约束允许页延迟写回，Kafka利用页缓存和批量日志传递内容；每种安排都有需要继续处理的后台工作。本地刷盘、复制确认与业务成功分别建立不同事实，必须按故障条件组合。下一章讨论这些职责怎样在软件内部划分，以及增加一层究竟隔离了什么变化。

第 5 章 分层架构设计 — 存储层 / 逻辑层 / 交互层

本章导读

我接手过一个运营后台，需求只是加一个导出按钮。我改了 Controller 三行代码，结果部署后首页打不开了。原来订单查询的 SQL 和导出逻辑混在同一个 DAO 里，我动的那几行不小心改了 SQL 拼接的顺序。我只动了三行，但影响的边界根本不在这三行里。

这个项目已经有 Controller 和 DAO。名称上的分层存在，查询和导出之间的依赖却没有因此消失。仅凭这次事故，还不能断定多加一层就能解决问题：两项功能也许确实需要共享查询条件，问题可能出在共享了不该共享的可变状态，或者把不同用途的 SQL 拼装合在了一处。分层能不能改善修改风险，要看它隔离的究竟是什么。

这一章从请求进入服务端开始，分别观察 Redis 的命令与对象、MySQL 的 SQL 与存储引擎、Kafka 的请求与分区日志。它们都有接收输入、执行操作、访问数据的职责，但职责边界未必是线程边界，更未必是网络边界。我关心的是一次变化到哪里为止，以及为了限制这次变化，系统又增加了哪些接口、状态和等待。

5.1 问题的本质：一项变化会穿过哪些边界

职责、执行和部署是三件不同的事

一个服务可以把协议解析、业务判断和数据访问分成三个模块，再把三个模块放在同一个线程中执行。它也可以不改变模块职责，只把网络读写交给另一个线程。两种情况下，代码都分了层，线程安排却不同。如果再把数据访问移到独立进程，就出现了进程间通信；放到另一台机器，才进一步涉及网络超时、连接中断和独立故障。

因此，“多了一层”本身不能告诉我们多了多少延迟。一次方法调用、一次队列交接、一次远程请求产生的成本不同。方法调用可能被编译器优化，也可能触发昂贵的数据复制；队列交接可能只增加很短的等待，也可能在负载上升后成为主要延迟来源；远程请求需要处理序列化与失败，但服务拆开以后也可能获得独立扩容的收益。只有把边界上的动作写清楚，讨论代价才有对象。

本章用交互、逻辑、存储三个名称标识职责。交互处理连接和报文，逻辑解释操作的含义，存储维护数据及其访问方法。这是观察时采用的分类，不声称三个项目的源码目录都严格按这个结构组织。认证、事务上下文、复制和资源限制还可能同时影响几项职责，不能仅把它们归入某个模块，就认为依赖已经处理完了。

接口要隐藏实现，也要暴露必要的限制

设想上层只需要“根据订单号找订单”。底层从内存哈希表换成数据库索引，只要返回的字段、找不到时的含义、错误处理和并发约定不变，上层就有机会保持不动。但是，原来一次调用不会等待，后来却可能超时；原来对象返回后可直接修改，后来修改需要单独提交。即使函数名和参数表没变，契约也已经变了。

接口不是函数签名的同义词。它还包括谁拥有返回的数据，调用失败后是否产生了副作用，调用能否重试，结果在什么时间点有效。把这些条件藏起来，上层表面上少知道一些细节，却不得不在出错时猜测下层行为。能够隐藏的通常是实现方法，不能随便隐藏的是影响调用者正确性的语义。

这种区别也是评价可替换性的起点。两个实现都能响应同一个方法，不等于替换后系统行为相同。内存与磁盘、事务与非事务、同步与异步都有可能被包装成相近的接口，但包装不能自动补齐语义。一个可插拔边界的价值，在于明确哪些差异允许存在，哪些能力必须由所有实现兑现。

判断分层，要落到具体修改

我看分层方案时，会先找几项已经可以指认的变化。例如，协议要增加一个字段，存储表示要随数据量转换，某个调用要由同步改为异步，或者一种实现要退出使用。然后沿着这项变化检查：哪个模块首先需要知道，哪些调用者必须修改，哪些测试会因此改变。

如果上层必须知道下层每一种内部表示，接口并没有隐藏这项变化。反过来，如果为了替换一个很少变化的部件，所有正常请求都要维护复杂的适配状态，也需要解释这笔成本为什么合理。分层不是越多越稳，也不是越少越快；判断的对象是具体的依赖及其代价。

还有一类边界不直接服务替换，却仍然有价值。测试替身让业务逻辑脱离真实网络运行；资源隔离让慢任务只占用指定队列；兼容层使旧客户端可以逐步退出。这些收益不一定表现为一种新的领域概念。只用“有没有新抽象”作为硬门槛，会误删有明确工程用途的层。

5.2 Redis：同一线程内也有需要维护的边界

5.2.1 交互层先把不完整的字节变成完整命令

客户端通过 TCP 发送数据时，一次写入和一次读取并不一一对应。服务端可能只读到命令的一部分，也可能一次读到多条命令。Redis 的查询缓冲区保存已经收到但还没消费的字节，RESP 解析器根据类型标记、长度和元素个数判断报文是否完整。这里复用的是第 2 章介绍的协议边界，而不是依赖网络恰好按命令交付。

解析可以随着字节到达逐步推进，但命令不能因为读到一行就立即执行。以数组形式发送的命令，需要收齐对应参数后才能形成一次完整调用。若把“增量解析”误当成“参数尚未到齐即可执行业务”，就会混淆报文处理与命令执行两个阶段。解析器负责保留进度，执行器接受的是已经形成的命令参数。

Redis 7.x 的普通命令执行主要在主线程串行进行。事件循环把就绪的连接交给处理函数，处理函数读入数据、推进解析，再进入命令分派。网络 I/O 可以按配置使用额外线程，部分后台工作也有自己的线程或子进程；因此“Redis 是单线程”适合说明主要命令执行路径，不适合说明整个进程只有一个执行单元。

这种安排减轻了键空间并发修改时的协调负担，但没有消除所有等待。某条命令执行很久，其他命令仍需等待主线程；一次响应很大，即使数据查询很快，输出缓冲和网络发送也会占用资源。性能问题不能仅由层数解释，操作复杂度、响应规模和后台任务同样会影响延迟。

5.2.2 输出缓冲区把执行完成和发送完成分开

命令完成后，客户端未必能够立即收完结果。Redis 把待发送内容放入客户端输出缓冲，随后在连接可写时继续发送。这样，命令处理不必一直等着慢客户端；但尚未送出的数据仍占内存，解耦的代价并没有消失，只是从等待变成了缓冲。

`client-output-buffer-limit` 按客户端类型设置限制，硬限制针对当前占用，软限制还结合持续时间。Redis 7.x 示例配置中普通客户端的该项默认值为零，含义是没有由这项配置施加的输出缓冲上限；不能概括为所有客户端默认都会在同一阈值被关闭。实际配置与客户端类别要一起看。R5-01

这个边界值得保留，因为命令执行速度与客户端接收速度本来就可能不同。但边界的正确性还包括资源归属：待发送结果算谁的内存，积压何时结束，超限以后是断开连接还是限制生产。除了缓冲区，还需要明确积压受什么约束。

在有管理权限的测试实例上，`CLIENT LIST` 中的输出缓冲相关字段，可以帮助区分“命令还没执行完”与“结果已产生但发送滞后”。这项观察不需要先改线程模型。若主要开销来自响应数据量，增加 I/O 线程与缩小单次响应解决的是不同部分的问题，不能互相代替。

5.2.3 命令表把共同行为从处理函数中分离出来

完整命令进入分派阶段后，Redis 根据命令名找到命令描述，再进行必要的参数、权限和执行条件检查，最后调用相应处理函数。命令描述不仅包含函数入口，还包括参数数量规则、命令属性和键位置等元数据。Redis 7.x 的命令描述由相应数据文件参与生成；查看具体实现时应使用对应版本源码，不能继续把早期版本的表位置当作固定目录契约。

命令表的好处，是把“怎样找到处理函数”和“怎样检查一类命令”集中起来。权限系统不必为每条命令独立发明识别方式，集群路由也能够利用键位置信息。但新增命令并不等于只加一行声明：如果命令访问多个键、有阻塞行为，或者需要传播到 AOF 和副本，处理函数与描述必须共同满足相关约定。



AOF、RDB与复制有各自触发条件；不能统一画成所有命令执行后的同步存储步骤。

图 5-1 Redis 命令处理与会话上下文：收到完整参数后进入分派与校验，结果先进入输出缓冲，再等待发送。

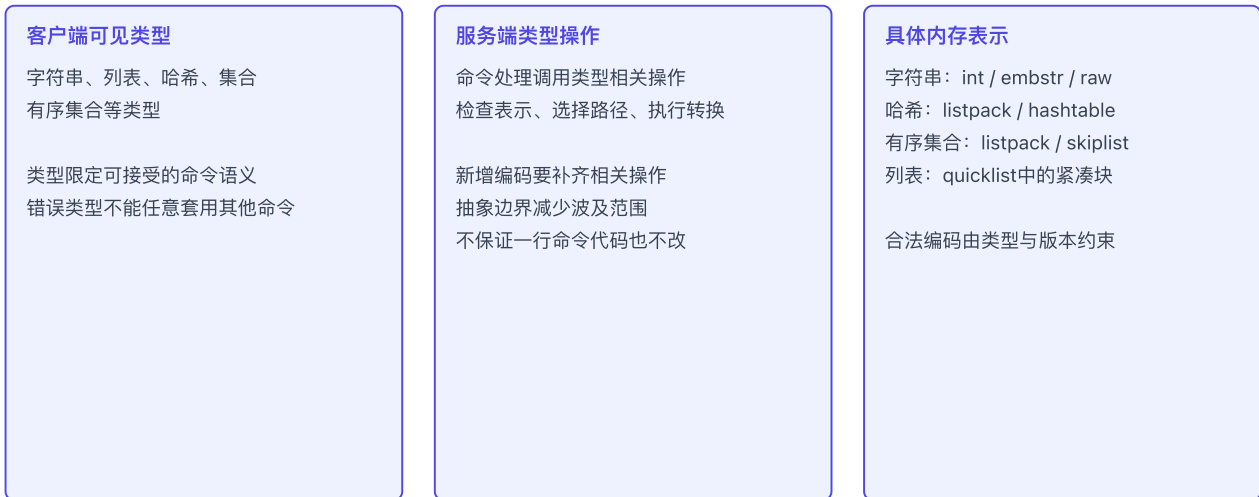
贯穿这条路径的是 `client` 对象。它保存连接状态、查询缓冲、当前命令参数、所选数据库、用户身份及输出状态。跨越几个函数时，共享这个对象比反复复制所有字段直接；但共享对象本身并不证明职责已经解耦。哪些字段可以在哪个阶段修改，谁负责清理一次命令遗留的状态，仍需由实现约束。

例如，参数数组属于当前命令，而身份与数据库选择可以跨命令存在。如果处理函数把本应属于单次调用的状态留在会话里，下一次请求就可能受影响。共享上下文减少了传参负担，也把状态生命周期变成了需要持续维护的契约。Redis 的 `client` 和稍后要看的 MySQL THD，共同点首先在这里。

5.2.4 type 与 encoding 隔离的是表示变化

`redisObject` 区分对象的类型与编码。类型说明对象提供什么操作语义，编码说明当前使用哪一种内存表示。例如，一个哈希对象可以在数据较小时采用 `listpack`，在超出适用条件后转换为哈希表。用户仍调用同一组哈希命令，但内部访问和存储的方法改变了。

第 2 章已经解释 `listpack` 怎样减少紧凑结构的更新依赖。本章关心另一件事：这种替换影响哪些代码。对客户端而言，命令名称、字段和值的语义可以保持；对服务端而言，转换、迭代、查找和释放等类型操作必须处理相应编码。不能据此说“逻辑层只看 `type`，所有命令代码都不关心 `encoding`”。边界减少了编码知识的传播，但没有让编码知识从实现中消失。



type与encoding不是两个互不接触的层；quicklist组织节点，listpack保存块内紧凑数据。

图 5-2 类型语义与内部编码：客户端命令保持类型语义，服务端类型操作负责选择和转换具体表示。

这个区别影响我们对升级成本的判断。将一种紧凑结构换成另一种，需要检查正常读写，还需要检查 RDB 装载、AOF 重放、复制、模块接口和旧格式兼容。上层接口稳定，是升级可以逐步进行的条件；它不是“所有上层一行不改”的证明。某个版本是否修改了具体调用者，只能从该版本变更与测试中确认。

在 Redis 7.x 的隔离测试实例中，可以用下列命令观察一个已有键的编码；这里不建立或删除业务数据：

```
TYPE example:key
OBJECT ENCODING example:key
```

这两行通过 `redis-cli` 执行，`example:key` 应替换成测试键。键不存在会得到相应空结果；键存在时，类型说明操作语义，编码说明内部表示。输出不能直接作为性能排名：同一种紧凑编码是否合适，仍取决于元素数量、值长度和操作分布。为了观察编码转换而调整阈值，也应另设测试数据和恢复步骤，不能把生产键当作演示材料。

5.2.5 内存、释放和持久化不必合成一个可插拔引擎

Redis 的对象存储围绕内存数据结构组织。内存分配器负责申请与归还空间，对象生命周期决定何时释放，淘汰策略在容量约束下选择哪些键退出，持久化则保存恢复所需的数据。这些职责彼此有关，却不是一件事。更换分配器不能给对象增加磁盘访问语义，开启 AOF 也不会把键空间自动变成可以按需读取任意规模数据的磁盘引擎。

大对象释放展示了职责与执行边界怎样分开。某些删除路径可以先把键从可见键空间移除，再将昂贵的内存释放工作交给后台线程；具体是否异步，取决于命令、配置和对象释放成本。`UNLINK` 与 `DEL` 的差别因此不是删除后能否再查到键，而是释放工作怎样安排。逻辑删除完成与内存全部归还之间，允许有一个时间差。

R5-02

持久化也有自己的选择。RDB 和 AOF 可以按部署目标配置，关闭两者会改变重启后的恢复来源；开启 AOF 则还要决定刷盘策略。`everysec` 讨论的是特定故障下未持久化写入的窗口，不是“整个实例一秒钟就能恢复”。恢复耗时取决于数据量、文件形态和设备性能，这部分在第 3、4 章处理。

Redis 没有提供 MySQL Handler 那样的通用存储引擎接口，这是其现有边界的事实。把原因归结为“内存只有一种，所以无需抽象”却过于简单：同一介质可以承载多种语义，不同介质也能实现相近接口。这里更具体的选择，是围绕既定键类型、内存表示及操作路径维持实现，而把模块扩展、持久化和复制放在各自的约定中。

5.2.6 直接调用节省了什么，没有节省什么

在主要命令路径上，许多职责之间通过函数、函数指针和对象指针交接，不需要把同一个请求再次编码成网络报文。这样减少的是中间序列化与跨执行单元交接，并不是“零抽象”。命令表和类型操作本身就是抽象；函数指针分派也是真实的间接调用。

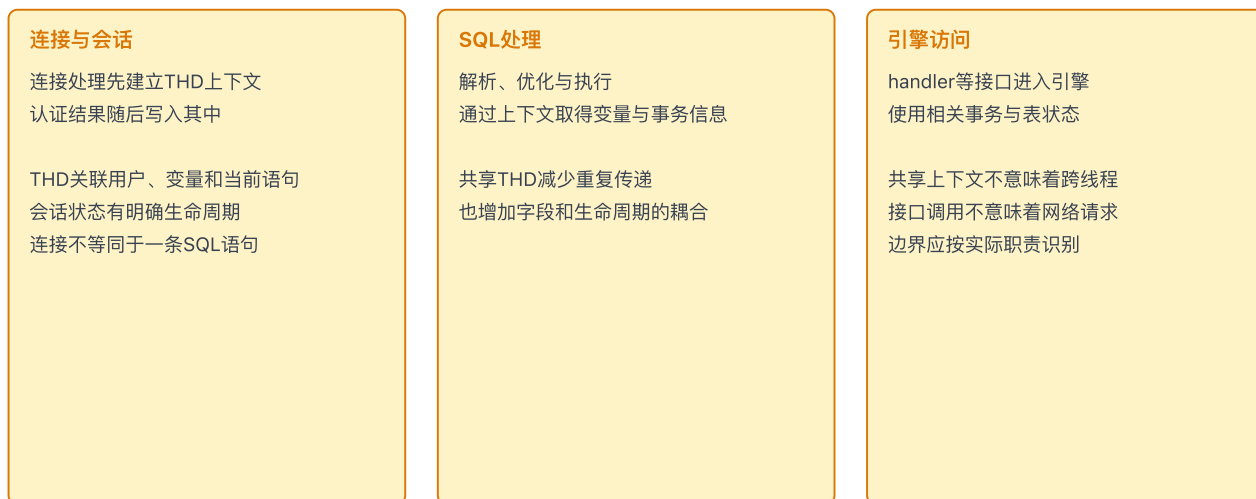
低延迟不能反过来证明每个间接调用都应该删除。若一次命令主要时间消耗在遍历大集合，减少一次函数分派可能并不改变总体表现；若某条极短路径在高负载下反复执行，分派成本才可能值得单独量化。我更愿意把直接调用看成一种执行选择，再通过性能证据判断其重要性，而不把它升格为所有内存服务都应遵守的层数规则。

5.3 MySQL：可替换性要连同能力差异一起说明

5.3.1 连接上下文与工作线程不是同一个概念

MySQL 服务端接收连接、完成协议握手和认证，再处理会话中的 SQL。常见的默认执行模型为连接分配工作线程；线程缓存和可选线程池机制又会改变线程的复用方式。因此，连接有独立状态，不等于它必须永久占有一个新建的操作系统线程。讨论线程池时还需要区分产品发行版和组件，不能把企业版或其他分支的能力写成所有 MySQL 8.0 安装的默认能力。

`THD` 是观察会话状态的重要对象。当前数据库、字符集、会话变量、语句诊断、事务相关上下文和执行状态，都可能经它到达后续处理路径。连接处理使用客户端身份，SQL 层使用当前执行信息，引擎操作又需要相应事务信息。共享上下文使这些信息能够被使用，而不必在每个函数之间展开一份完整的参数清单。



THD是会话执行所需的重要上下文：本图不把每个职责框映射成一个独立线程。

图 5-3 MySQL 会话上下文与职责边界：THD 关联连接、语句与事务信息；共享上下文不意味着每一项操作都跨线程。

但共享一个对象也会扩大修改影响。一个字段究竟属于会话、语句还是事务，清理时点不同。如果执行失败后保留了不该延续的状态，后续语句就可能继承错误条件。这里需要讨论的是状态生命周期，而不是把共享对象简单评为“边界是虚的”。在同进程系统中，职责边界完全可以通过受约束的对象访问维持，未必需要靠复制或序列化证明存在。

连接池还会让这个问题到达应用侧。物理连接被多个业务请求轮流复用，上一请求改变的会话设置若未恢复，下一请求就可能受到影响。这不是 MySQL 必然会出的问题，而是连接复用必须遵守的接口条件。服务端的会话抽象与应用的请求抽象并不天然一致，适配层需要明确如何归还连接。

5.3.2 SQL 的中间表示让不同阶段处理不同问题

SQL 文本首先需要被解析成可以处理的结构，再经过语义准备、优化和执行。解析与准备关心语法、对象和表达式能否成立；优化阶段选择可行的访问路径与算子安排；执行阶段依据计划取数并产生结果。这几项职责可以在内部交织，但不能因为都在服务层，就当成同一种工作。

中间表示的价值，是让后续阶段不再反复解释原始字符串。优化器处理的是表、谓词、连接关系与访问候选，执行器处理的是已经选择的计划。这样，语法增加一种写法时，有机会在准备阶段转换成既有表达式；引擎增加一种访问能力时，也可以通过能力和代价信息参与计划选择。是否真能局部修改，还取决于该特性有没有改变语义。

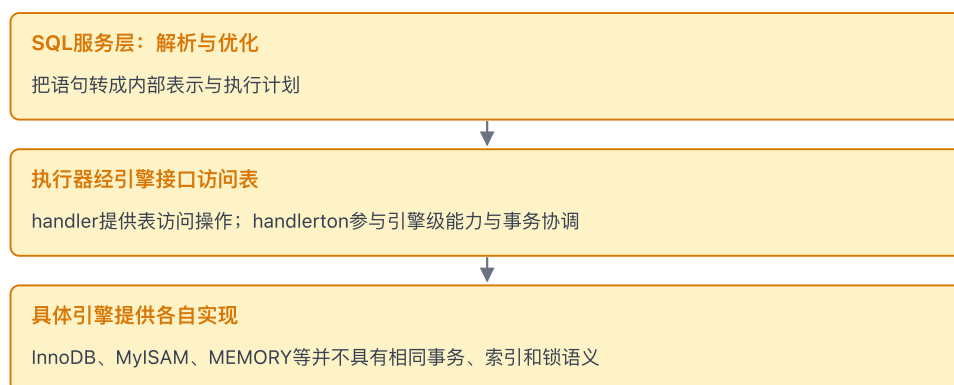
这与 Redis 命令表形成了有用的差异。Redis 的一条命令往往已经指出具体操作，MySQL 的查询需要从关系描述推导执行方法。差异在于输入保留了多少执行选择，并不意味着 SQL 天生只能逐行、永远无法批量处理。MySQL 8.0 的不同算子和优化存在版本演进，不能从某个典型的逐行调用循环推出声明式语言的性能上限。

在排查中，计划与实际执行之间的差距同样重要。估计行数、访问索引和连接顺序可以帮助解释为什么一次查询产生大量行读取；仅看到 Handler 接口有虚函数，不能把查询慢的原因直接归给多态。若访问行数相差几个数量级，先查计划通常比争论一个调用指令更有解释力。这里的判断依据是工作量，而不是语言特征。

5.3.3 Handler 隐藏页布局，但不统一全部语义

MySQL 的存储引擎接口让服务层能够以相对统一的方式打开表、请求索引或扫描访问、读写记录，并参与事务协调。InnoDB 在内部处理页、索引、锁、MVCC 和 redo log；服务层不需要在每条 SQL 中直接实现这些细节。不同表使用不同引擎，是这一边界支持的能力之一。R5-03

这里有两个层次需要分开。`handler` 对象提供表和行访问相关操作，`handler_ton` 则提供引擎级注册及相关回调。把所有方法都列为某一个对象的成员，容易使人误解实现。本书保留这些名称用于对应官方接口，不列出一份看似完整但随版本演进的函数清单。



统一调用接口便于替换实现；能力差异仍必须由服务层和使用者处理。

图 5-4 MySQL 服务层与存储引擎接口：统一入口下仍存在事务、索引和锁语义等能力差异。

InnoDB 支持事务、崩溃恢复和行级并发控制；MyISAM、MEMORY 等引擎具有不同约束。接口允许它们接入，不代表把表换成任意引擎之后，原来的事务、锁或恢复承诺还能自动成立。应用若依赖回滚，就不能只依据“SQL 能执行”判断替换成功。

引擎差异还会向优化器传播。哪些索引访问可用，统计信息能提供到什么程度，某些条件能否下推，都会影响计划。好的抽象不是完全屏蔽差异，而是让差异通过明确的能力和约束表达。否则服务层只能假定所有引擎相同，再在执行失败时补救。

因此，可插拔的代价不仅是调用开销。接口要维护兼容性，优化要尊重能力组合，测试要覆盖不同实现，用户还要理解替换后的行为。只有把这些成本纳入，才有依据评价一个扩展点值不值得。现有多种实现确实需要接入时，这些成本可能合理；只有一个实现，也仍可能为了测试或隔离频繁变化而保留接口。

5.3.4 Query Cache 说明协调成本，不证明缓存只能放一层

MySQL 5.7 及更早版本曾提供 Query Cache，MySQL 8.0 已移除。它缓存的是查询结果：符合缓存条件的相同查询可以复用已有结果，而不必重新完成整个执行过程。相关表发生变化后，依赖这些表的缓存结果需要失效。该机制与全局同步相结合，在并发和写入频繁的负载中会产生明显维护成本。R5-04 R5-05

这个案例不能推出“结果缓存本应放在引擎内部”。InnoDB 缓冲池缓存的是页，命中后仍可能需要执行查询、判断可见性和构造结果；结果缓存试图复用的是另一个层次的工作。两者的失效条件和收益不同，不能把一种缓存的存在当成另一种缓存应该移到哪里的证明。

更有解释力的问题是：缓存项依赖哪些可变数据，变化怎样通知它，失效粒度有多大，读命中节省的工作能否抵消写维护和同步成本。如果结果很少变化、相同查询大量重复，缓存仍可能有收益；如果每次更新都会使大量结果失效，命中率再高也不能独立说明整体收益。

Query Cache 被移除，是一个功能在产品演进中退出的事实，不是整个服务层被删除。它提示我们重新评价一项机制，而不是把“删层”作为预定答案。实现可以退出，相关问题却还在：应用缓存、代理缓存和物化结果同样需要明确数据依赖与失效条件，只是把责任放到了其他地方。

5.3.5 上下文共享与引擎接口各自限制不同的变化

THD 与 Handler 不能合并成一个“标准化程度”排名。前者让一次会话及语句所需的信息可被相关执行路径访问，后者使服务层能够与不同引擎交互。共享上下文主要处理状态传递，可插拔接口主要处理实现差异。它们都在减少某些重复，但也分别引入状态耦合和能力组合的复杂度。

当一个系统要支持第二种存储实现时，我会先检查服务层有没有直接使用页格式、锁类型或引擎私有返回值。如果已经依赖这些细节，补一个接口名称不足以完成替换。需要决定哪些依赖确属业务要求，哪些只是实现便利；前者应进入契约，后者才有机会被隐藏。

同样，若问题只是单元测试难以运行，未必需要建设一个完整引擎插件体系。一个范围很小的访问接口，可能已经能够隔离真实数据库依赖。MySQL 的经验说明复杂接口能够支持什么，也说明为了这种能力需要长期维护什么，不能只取“支持可插拔”这一项好处。

5.4 Kafka：把线程交接与分布式协调分开观察

5.4.1 网络线程与请求处理线程之间有队列

Kafka 3.9.0 Broker 的网络服务由不同角色协作。Acceptor 接收新连接，Processor 管理连接的网络事件，请求通过 RequestChannel 交给请求处理线程，再进入相应 API 处理。一个角色可能对应多个线程，队列本身则不是线程。不能把这些名称机械相加，再与另一个系统的源码模块数比较层数。

RequestChannel 使网络事件处理与业务处理可以采用不同数量的线程。网络侧处理大量等待 I/O 的连接，业务侧执行校验、追加、读取等工作；两侧不必以同一个比例扩容。但这并不保证它们互不影响。请求队列有容量，业务侧持续处理不过来时，入队会受到限制，影响网络侧继续接收请求。队列把短时速度差转换成

排队，不能把长期超载转换成额外处理能力。



排队容量只能限制积压，不能增加处理能力；普通函数调用、线程交接和网络RPC要分开标注。

图 5-5 Kafka 网络事件与请求处理线程：RequestChannel 是同一进程内的交接点，有界队列限制积压，并不增加下游处理能力。

一个 Broker 可以同时承载多个分区和多类请求，多线程使这些工作有机会利用多个处理器核心。但不能据此说 Kafka 的每次请求延迟一定高于 Redis，或吞吐必然高一个数量级。请求大小、确认条件、访问介质和批量方式不同，直接比较数字没有意义。本章只确认一项结构差异：这里显式安排了网络与业务线程间的交接，而 Redis 的普通命令路径主要围绕主线程组织。

队列还能帮助拆分延迟来源。请求在队列里等待的时间，与处理线程实际执行的时间不是同一个指标。若队列持续增长但执行时间没有改变，可能是到达速度超过了处理能力；若执行本身变慢，则还需检查磁盘、锁、配额或等待副本等条件。只增加队列长度往往把拒绝推迟，却让请求在系统中停留更久。

5.4.2 API 适配不等于没有业务判断

Kafka 协议用 API 标识和版本描述不同请求。KafkaApis 根据请求类型进入对应处理路径，并与分区、副本、消费组等组件协作。协议版本的不同字段要转换成内部能够处理的输入，身份与资源权限需要核对，配额和错误要转换成相应响应。这些工作并不等于完整的领域逻辑，但也不是毫无判断的转发。R5-06

把协议适配与领域操作分开，有助于让多个协议版本使用相近的内部能力。然而，并非所有协议变化都能在适配层内处理。如果新请求增加了新的事务或一致性要求，内部状态和执行规则也必须改变。接口隔离的是可被表达转换消除的差异，无法隔离语义本身的增加。

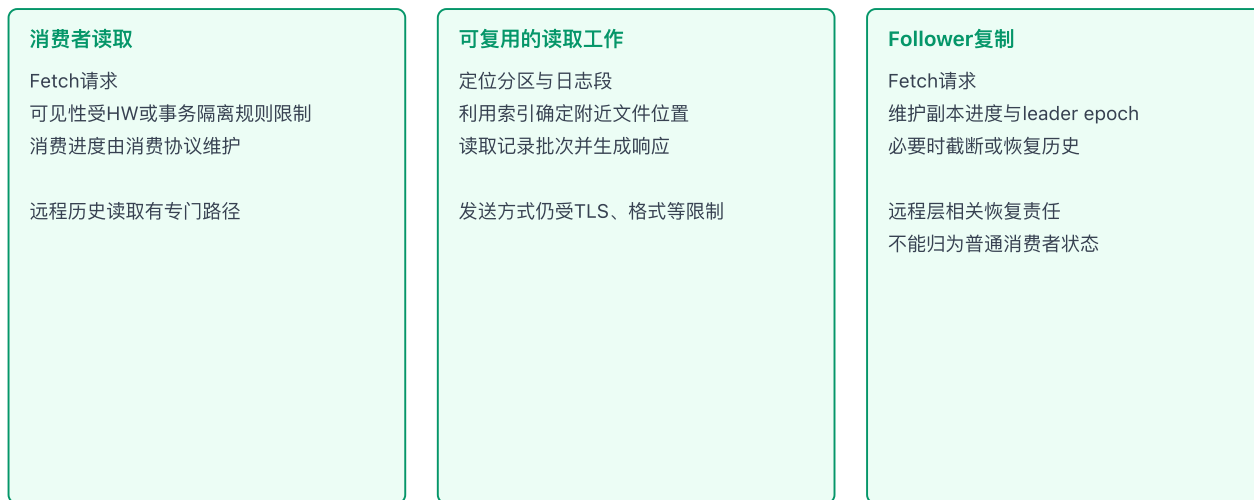
请求还会带着上下文继续处理：来自哪个连接、认证后的主体是谁、使用哪个 API 版本，以及怎样关联响应。这与 Redis 的 `client`、MySQL 的 THD 有相似问题，但生命周期并不完全相同。跨线程以后，谁能修改什么、对象何时可以释放，需要比普通同步调用更明确；不能因为都叫上下文，就假定它们具有相同共享方式。

5.4.3 分区日志提供稳定的操作语义

Kafka 的本地日志按分区组织，并拆成日志段。3.9.0 源码中的日志实现又区分统一日志视角与本地文件管理。上层需要的是按 offset 追加、定位、读取、截断和清理等操作，并不应该在每次请求中重新管理所有文件细节。这里的抽象价值，在于把分区日志的行为与具体文件操作分开。

追加路径仍有校验、索引和状态更新，后台 compaction 还可能重写数据，所以“追加日志”不等于系统没有写放大，也不等于所有读写都是常数成本。它说明常见写入按日志次序进行；保留、压实、事务状态和异常恢复则增加了其他工作。用一个简洁的数据模型描述主路径，与忽略其余工作是两回事。

消费者 Fetch 和副本 Fetch 可以复用读取分区日志的基础能力。它们都需要根据 offset 定位记录，都可能利用页缓存和文件传输路径；但是，可见范围、调用身份、复制进度和异常处理不同。普通消费者的读取还要结合隔离级别，副本则需要推进自身日志并处理主节点历史变化。共享读取组件不能推出“复制就是消费者换一个标志”或“无需独立状态机”。



共享底层读取能力，不表示身份、可见性规则、进度状态与故障处理完全相同。

图 5-6 Kafka 消费与复制共享的日志读取能力：底层定位和读取可以复用，可见性规则、进度状态与恢复责任分别维护。

这种复用的收益是局部而清楚的。定位文件和读取记录的优化，不必在两套完全独立的基础组件中各做一次；但某项改动是否同时改善消费者和副本，要看它们是否经过相同分支。测试不能因为“代码共享”就只验证其中一种调用者，尤其是边界 offset、事务可见性和主节点切换时的行为。

5.4.4 零拷贝路径有前提

对于适合直接从本地文件发送的记录，Kafka 可以利用文件到网络的传输能力，减少数据在用户态缓冲中的往返复制。第 4 章已经解释页缓存，这里还涉及接口条件：若后续处理需要改变字节，或者网络层必须在用户态完成加密，就不能把所有数据都当作原样转发。R5-07

Kafka 3.9 文档明确说明，启用 SSL 时不使用所描述的 `sendfile` 路径。记录格式转换等工作也可能影响具体读取路径。因此，“日志中的消息不再修改”不足以保证端到端始终零拷贝。不可变记录有利于复用和传输优化，但链路中的协议和安全要求同样参与决定。

这个例子说明，性能优化往往依赖多个层共同保持一个条件。存储层保持适合传输的布局，读取层避免不必要的转换，网络层提供合适的发送能力。把这些条件写成接口约束，比笼统评价“存储层很薄，所以快”更有用。条件变化时，优化可能退出，业务语义仍然必须正确。

5.4.5 远程存储扩大实现边界，不取消日志语义

Kafka 3.9.0 的分层存储允许在满足配置和实现条件时，把符合条件的历史段保存到远程存储。相关组件负责远程段与元数据管理，本地与远程读取需要协调。这个变化保留了客户端按分区和 offset 访问记录的主要使用方式，却不是“整个上层和复制层完全无感”。恢复与异常处理确实需要确定数据位于哪里。R5-08

本章不展开远程段状态与追赶流程，它们分别属于第 8、9 章。这里可以得到一个较窄的观察：稳定的日志语义允许存储实现演进，但为了守住这种语义，内部接口和状态管理可以发生显著变化。

这也有助于区分接口层次。保持外部协议兼容、保持内部方法签名、保持文件格式，是三种不同的稳定性。某项功能做到第一种，不证明后两种都没有变化。面向使用者的兼容承诺越明确，内部才越有依据决定哪些部分可以重构。

5.4.6 协调职责不等于又多出一层网络

Broker 需要获取分区归属与副本状态，消费组需要获取成员与分配信息，Controller 则处理集群元数据。KRaft 使元数据通过控制器仲裁组管理；本书以 3.9.0 的 KRaft 部署为主，ZooKeeper 只作为演进对照。不同协调职责有不同状态，不能合成一个“保证一致性”的黑盒。

在 Broker 内部，API 处理与本地日志、副本管理之间存在对象调用；网络线程与请求线程之间存在队列；跨 Broker 或独立 Controller 的通信才涉及网络。独立部署使相应服务可以单独发生故障，并不意味着每个逻辑模块都必须拆成独立进程。Redis Cluster 和 MySQL Group Replication 也有协调问题，只是不能据此把它们的源码组织硬套成相同层数。

5.5 横向对比：同一项变化，由谁承担

三个系统的共同之处，不是都能被数成三层或四层，而是都要把外部请求转换成受约束的数据操作。真正可比的是：输入变化到哪里结束，状态怎样传递，实现差异怎样表达，以及异步交接后谁承担积压。

观察样本	职责之间怎样协作	没有被接口自动解决的事
Redis	解析 → 命令分派 → 类型操作 主要命令路径可直接函数调用	表示转换、会话状态与输出积压 仍需明确的所有权和生命周期
MySQL	连接与SQL处理共享上下文 经引擎接口进入具体存储实现	不同引擎的能力与事务语义 无法由同名接口完全统一
Kafka	网络线程经队列交接 业务层调用日志与协调组件	进程内交接和分布式RPC不同 队列、配额与角色仍会相互约束

同一职责划分可以有不同执行结构；框的数量不是架构优劣指标。

图 5-7 职责边界与执行边界对照：职责通过函数调用、共享上下文或队列交接协作，接口之外仍需处理状态和能力差异。

表 5-1 三种边界安排分别承担什么变化

比较问题	Redis 7.x	MySQL 8.0.x	Kafka 3.9.0
外部输入怎样形成内部操作	RESP 参数进入命令分派	SQL 经过解析、准备、优化与执行	API 请求经适配进入领域处理
共享状态在哪里	client 及当前命令状态	THD 关联会话、语句、事务状态	连接上下文、请求对象及领域状态
表示差异怎样局部化	类型操作管理不同 encoding	引擎接口隐藏页布局等实现细节	日志操作隔离本地文件和部分存储实现细节
哪些差异仍需暴露	编码适用条件、命令语义和资源限制	引擎事务、锁、索引等能力	可见性、确认、保留和恢复条件
主要交接形式	普通命令路径多为主线程内调用	同进程上下文与引擎接口调用	同进程方法、跨线程队列、跨节点协议并存
不能从边界直接推出什么	不能推出无抽象成本或任意命令低延迟	不能推出换引擎后全部语义相同	不能推出所有层都跨网络或所有路径都零拷贝

这些替换能力没有统一的优劣排名。Redis 把同一种类型的表示变化局部化，MySQL 支持具有不同能力的引擎，Kafka 维持日志访问语义并演进其内部实现。三者替换的对象不同，维护的契约也不同。只有先指定变化，再比较影响范围，才不会把某一种扩展能力当成所有系统的必选项。

这里还存在一个容易被忽略的反向成本：接口过于通用，会迫使实现暴露大量能力开关；接口过于专用，则会让新增场景不断绕过它。哪一种更合适，需要看实际变化分布。一个稳定而频繁使用的共同操作值得抽象，一批只有名称相似、语义不同的操作未必适合合并。

5.6 架构启示

5.6.1 先判断要隔离的变化，再选择交接方式

Redis 的类型与编码说明，表示变化可以被限制在一组相关操作里；MySQL 的引擎接口说明，统一入口仍要表达能力差异；Kafka 的队列说明，执行速度不同可以通过有限积压协调。三个机制解决的问题不同，不能都缩成“增加一层”。

回到导出功能，如果共享的是同一套查询条件，那么完整复制查询代码可能增加维护成本；如果共享的是会被导出逻辑改写的状态，那么增加一个只做转发的 Service 也不会自动消除风险。需要先区分共同规则与特定处理，再决定用函数、对象还是独立任务交接。

5.6.2 稳定的是契约，不必是每一份内部实现

接口稳定对调用者有价值，但“十年没变”不是可以在设计当下验收的条件。更具体的检查，是列出已经确定的变化，观察它们能否在约定范围内完成。新增一种编码会不会改变命令语义，替换引擎会不会失去回滚能力，改为异步执行会不会使超时含义变化，这些问题现在就能讨论。

有些稳定需要通过版本演进实现。旧调用者继续使用原行为，新能力通过显式版本或新增操作提供；也有些变更必须承认不兼容，并给出迁移路径。为了保持方法名不变而让同一个参数在不同情境下偷偷改变含义，反而会使接口更难依赖。

5.6.3 共享上下文要有字段和生命周期的约束

`client`、THD 和 Kafka 请求上下文都减少了重复传递，但共享对象会扩大可见范围。评价它们时，不能只数省掉多少参数，还要看一次操作结束后哪些状态仍然有效。会话、事务、请求三个生命周期重叠时，清理职责尤其重要。

如果一个模块为了使用两个字段，必须理解几十个无关状态，这个共享对象可能已经超过了当前边界所需的范围。改进未必是复制整个对象，可以是提供更窄的访问接口、分离可变状态，或明确只读视图。跨线程后还需说明发布和回收规则；靠约定“大家不要乱改”不足以支撑并发正确性。

5.6.4 异步边界必须带上容量、取消与失败语义

有界队列能够限制一处积压，但端到端还可能存在客户端缓冲、内核 socket 缓冲和输出缓冲。某个队列满了以后，上游是等待、拒绝还是重试，直接影响系统负载。如果拒绝触发无退避重试，局部保护也可能变成新的流量来源。

取消同样需要沿着路径追踪。客户端已经超时，不代表服务端尚未执行；请求仍在队列中与已经完成数据修改，是不同状态。接口如果只返回一个笼统超时，上层就需要额外的方法确认结果，或者用幂等标识容纳重复。这些问题会在第 10 章的组合案例中再次出现，分层不能自动确定这些业务行为。

5.6.5 删除不合算的机制，要先说明它承担过什么

我仍会用“这一层挡住了什么变化”追问分层，但不会因为答不出一个新领域名词，就立刻删除它。兼容、测试、限流、故障隔离都可能是明确职责。只有确认这些职责不存在、已经移交，或者成本超过收益，删除才有依据。

Query Cache 提供了一个具体提醒：被移除的是结果缓存机制，缓存依赖与失效问题并没有一起消失。删除之后由谁承担原来的工作，调用者会失去什么，回退怎样进行，都属于修改的一部分。把“去掉一层”当作简化的证明，和把“加一层”当作解耦的证明一样不充分。

5.7 小结

导出故事里的问题，不是缺少几个标准层名，而是修改依赖没有被限制在预期范围。三个软件展示了不同的限制方法：Redis 用命令元数据和类型操作组织处理，MySQL 用中间表示与引擎接口容纳实现差异，Kafka 则让请求适配、线程交接、日志操作与集群协调各自承担明确工作。

这些边界都有成本。共享对象需要约束状态，接口需要维护能力契约，队列需要限制积压，远程调用需要处理不确定的结果。分层的收益要与这些成本放在同一次判断中。一个设计能把某类修改局部化，已经是具体价值，不必再证明它具有最少层数或最大的可插拔程度。

下一章继续检查访问边界。职责分开以后，还要知道谁能调用、允许改哪些数据，以及一次越界操作能否被阻止和追踪。第 7 章再把执行范围扩到多台机器，讨论分片、复制与控制面分别解决什么问题。

第 6 章 安全机制 — 权限、加密、审计

本章导读

早年在测试环境跑 Redis 没设密码，某天发现 `KEYS *` 返回来一堆不认识的东西：一个爬虫脚本扫到了公网 IP 上的 6379 端口，把我们的实例当成了免费缓存。好在只是测试环境。

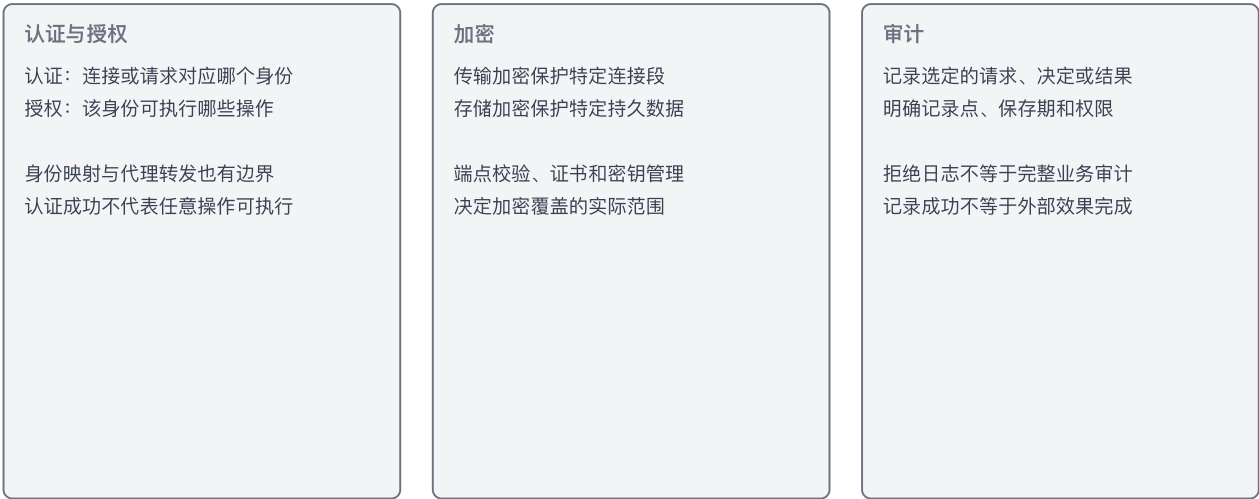
这件事能证明的是，那次部署把可以执行命令的接口暴露给了没有获得授权的人。加上一个密码也不意味着完成全部保护。后来再看数据系统的安全配置，我更关心一个具体请求：它从哪里进入，以什么身份执行，能触及哪些状态，失败和成功又留下什么记录。

Redis、MySQL 和 Kafka 都能提供认证、授权和传输保护，但资源模型与部署组合不同。Redis 的键权限、MySQL 的表列权限、Kafka 的主题与消费组权限，各自限制不同的操作。把它们排成“简单、企业级、分布式”三档，反而容易跳过最关键的判断：当前配置究竟阻止了哪一种越界。

6.1 问题的本质：同一请求涉及不同的信任关系

四项工作不能互相替代

认证确认连接或请求代表哪个身份；授权判断该身份能否执行相应操作；加密保护特定介质或传输范围内的数据；审计为事后追踪提供记录。这四项工作可以协作，但不能互相替代。已经通过密码认证的账号仍可能权限过大；已经加密的连接仍可能连向错误的服务端；完整记录一项越权操作，也不等于事前阻止了它。



这些是可交叉的责任，不是一条按认证、授权、加密、审计固定串行执行的流水线。

图 6-1 一次请求涉及的安全责任：传输保护覆盖连接，认证建立身份，授权限制操作，审计记录选定事件；存储加密另有介质与密钥范围。

图中各项不是一个固定的四步算法。TLS 可以在应用认证之前建立，也可以参与客户端身份校验；授权可能发生多次；审计记录可来自多个阶段。把它们画在一张图上，是为了标出责任，不是宣称每种系统都按完全相同的顺序执行。

先说明防护对象，再评价配置

对公网连接，窃听与冒充服务端是需要处理的风险；对已经取得应用凭证的攻击者，操作范围更重要；对被复制走的磁盘或备份，加密及密钥分离才有针对性。一个拥有操作系统管理权限的人，与一个只能调用 GET 的应用账号，不处于同一信任位置。不给出这些前提，就很难判断“已经足够安全”意味着什么。

部署环境也不能只分为可信内网与不可信公网。应用主机可能被入侵，测试凭证可能误用于生产，维护人员可能使用了过大的管理权限。网络隔离能够减少可到达的入口，却不能自动把所有内网调用都变成授权操作。内部连接是否需要加密，要根据数据与访问风险决定，不能仅凭地址属于私网就省略判断。

本章比较服务本身暴露的能力，同时明确与操作系统、身份平台、密钥服务之间的分工。这样既不把某个产品没有内置的功能说成无法实现，也不把“可以外挂”说成已经具备。一个扩展接口与一套运行可靠、能撤销凭证并保留审计的系统之间，还隔着具体的集成工作。

长连接减少认证次数，不取消身份变化

数据服务经常使用长连接，身份验证的成本可以由多次操作分担。但授权必须针对当前操作及资源，不能因为握手时通过认证，就放行会话中任何命令。密码轮换、账号禁用和权限撤销还会提出一个额外问题：已有会话何时受这些变化影响。

“认证一次、授权每次”可以描述一种常见安排，却不是永久不变的会话契约。有的操作可以重新认证，有的系统支持连接上的再认证；修改账号状态后，已经建立的连接也未必立即终止。这些差异在应急撤权时很实际。撤销新连接资格与使所有旧会话失效，需要分别验证。

6.2 认证：身份从哪里来，凭证如何更换

6.2.1 Redis 的用户与会话

Redis 6.0 引入 ACL 用户体系，7.x 延续这套模型。与早期共享 `requirepass` 相比，命名用户使不同应用能够使用不同凭证与权限。`AUTH username password` 或带认证信息的 HELLO 可以建立用户身份。兼容形式的共享密码仍然存在，因此迁移到较新版本不会自动替部署者建立按应用区分的身份。

Redis ACL 用户可以配置多个密码，这为轮换提供了过渡空间：先让新旧凭证都可用，迁移客户端，再撤旧凭证。但允许重叠与完成轮换是两回事。仍在使用旧密码的客户端、长期未重连的连接，以及配置文件中遗留的副本，都需要有观察和退出办法。只把新密码加入服务端，旧凭证并没有因此失效。

密码摘要、运行时用户状态和持久化规则也应区分。配置了 `aclfile` 时，ACL 可以按相应命令保存和加载；修改运行时规则，不等于所有磁盘配置已经同步更新。启动时从哪里恢复用户，需要与运维管理方式一致。否则重启可能重新启用旧规则，或者让已迁移客户端无法登录。R6-01

Redis 使用密码摘要并不意味着可以选择容易猜测的密码。用于校验的摘要不是可逆加密，但弱口令仍可能被离线尝试恢复。这里应依赖高熵凭证、访问控制和妥善保存，而不是只看到文件中没有明文就认为风险已经消失。

6.2.2 一个 ACL 示例还需要解释初始状态

下面的命令用于 Redis 7.x 隔离测试实例，通过已获授权的管理连接进入 `redis-cli` 后执行。它会建立或重置演示用户，因此不对已有应用用户照抄。`REPLACE_WITH_RANDOM_SECRET` 是必须替换的占位符，不是可使用的密码。

```
ACL SETUSER book_demo reset on >REPLACE_WITH_RANDOM_SECRET ~book:demo:* +get +set
```

`reset` 使示例从明确的权限状态开始，避免已有授权与新规则累积后仍然保留额外权限。`on` 允许该用户认证，密码前的 `>` 是 Redis ACL 语法，不是 shell 的输出重定向。键模式只覆盖 `book:demo:` 前缀，命令只授予 GET 与 SET。该用户没有由这条命令获得管理命令、任意键访问或订阅通道权限。

如果在 shell 中调用 `redis-cli`，带 `>` 的参数还需要正确引用；这也是示例特意说明执行位置的原因。凭证还可能进入命令历史，实际管理应使用符合环境要求的凭证注入方式。本例只展示规则含义，不能代替生产凭证发放流程。

Redis 7.0 提供 `ACL DRYRUN`，可由管理用户检查某个命令对目标用户是否允许，而不执行数据操作：

```
ACL DRYRUN book_demo GET book:demo:1
ACL DRYRUN book_demo GET other:1
ACL DRYRUN book_demo FLUSHALL
```

第一项应符合演示授权，后两项应因键范围或命令权限不符而被拒绝。这个检查直接验证规则边界，比只查看用户是否创建成功更有意义。测试结束后，管理用户可用 `ACL DELUSER book_demo` 删除演示用户；若测试期间选择将规则持久化，还应同步恢复对应配置。R6-02

6.2.3 MySQL 账号匹配与认证插件

MySQL 账号以用户与来源部分共同标识。`'app'@'localhost'` 与另一个来源规则下的 `'app'` 可以对应不同账号配置。连接建立时先匹配账号，再使用该账号指定的认证方式。客户端声称的用户名，与最终匹配到的账号并不总是同一个文本表示；排查时可以对照 `USER()` 与 `CURRENT_USER()`，前者反映连接信息，后者反映权限检查采用的账号。

MySQL 8.0 的默认认证插件为 `caching_sha2_password`，但升级后的既有账号仍可能采用其他插件。新安装、升级和发行版初始化方式必须分开看，不能概括成“装好就一定有强密码”。初始化选项、密码策略组件和账号创建语句共同影响结果。

`caching_sha2_password` 区分完整认证与缓存可用时的快速认证。完整认证需要安全连接，或在相应非加密连接上使用 RSA 密钥交换来保护密码传送；RSA 保护密码交换不等于给此后的 SQL 和结果加密。缓存路径减少后续认证的工作，但不是把现有长连接上的每条 SQL 都重新做一次密码校验。R6-03

插件接口能够接入不同身份验证方式，但客户端也必须支持对应协议，服务端还需部署正确组件。有些企业身份集成与发行版或授权有关，不能把插件名称列表当作所有安装都可直接使用的能力。更换认证方式的验收，应包含新连接、缓存失效后的连接和旧客户端，不能只在一个已连通会话里执行查询。

6.2.4 Kafka 的认证机制与监听器

Kafka 3.9.0 可以使用 SASL 机制或 TLS 客户端证书等方式建立身份。SASL 是认证框架，PLAIN、SCRAM、GSSAPI、OAUTHBEARER 是不同机制；`SASL_SSL` 与 `SASL_PLAINTEXT` 则还表达了传输层是否使用 TLS。名字中都有 SASL，不代表凭证及业务数据受到相同保护。

PLAIN 需要通过受保护的传输来避免暴露密码。SCRAM 通过带盐的派生信息和挑战响应证明持有密码，但也不能据此省去服务端身份验证与连接保护。支持某种算法，只说明产品具有一种选择；部署还要检查凭证来源、轮换、客户端兼容和运维能力，不能单凭 SHA-512 的数字较大就完成机制选型。R6-04

同一 Broker 可以暴露多个监听器，分别服务应用、节点间通信或控制面相关连接。监听器名称映射到安全协议，再关联相应认证与证书配置。它们是同一进程中的不同入口，不是三套彼此独立的数据服务。某个入口配置严格，不能抵消另一个可到达入口的宽松权限。

连接路径	示例配置职责	需要核对的范围
应用 → Broker	INTERNAL / EXTERNAL各有地址 示例采用SASL_SSL	SASL确定身份，TLS保护连接 advertised地址必须可达且可校验
Broker → Broker	副本流量使用指定监听器 单独配置认证和传输保护	应用连接安全不自动覆盖副本连接 证书、身份与授权需要匹配
Broker / Controller 控制面	本例选择TLS保护KRaft连接 Controller配置与角色单独核对	这只是一个部署选择 不据此排除其他受支持配置

监听器不是信任等级排名；每段保护范围取决于部署、身份映射与实际网络路径。

图 6-2 Kafka 监听器与连接身份：应用、Broker 间和控制面连接分别选择协议与凭证；图中是显式配置示例，不是产品默认值。

`listeners` 指定服务端监听位置，`advertised.listeners` 为客户端提供后续连接地址。`bootstrap` 地址能连通，只证明客户端找到了初始入口；元数据返回的 Broker 地址若无法从客户端网络访问，后续生产或消费仍会失败。TLS 还需要证书中的身份与访问地址匹配。把地址问题误当作认证失败，常会导致无依据地放宽安全设置。

6.2.5 控制面也需要能够启动的凭证来源

KRaft 的元数据与应用消息不同，控制器和 Broker 对凭证、授权信息及元数据状态具有依赖。设计内部认证时，需要说明首次启动时凭证从哪里来，哪些信息可以预置，哪些必须等集群建立后管理。不能仅因为凭证存放在元数据中，就推导某个监听器在所有版本都绝对不能采用某种机制。

本章图示选择控制面使用预置证书的 TLS 连接，应用和 Broker 间使用各自明确配置的认证方式。这是一个便于说明信任关系的部署示例，不是排他结论。采用其他组合时，应按 3.9.0 的具体支持与引导步骤核验，尤其不能把为数据连接准备的配置直接搬到控制器仲裁连接。

Kafka 的委托令牌提供另一种凭证生命周期：通过已认证通道申请，后续使用有效期受限的令牌，并处理续期与撤销。它可以减少分发长期凭证的需要，却不自动保存每次业务调用的最终用户身份。令牌代表谁、谁可续期、服务端怎样保护验证所需的秘密信息，仍属于完整设计的一部分。

6.3 授权：允许一项操作，需要哪些资源权限

6.3.1 Redis 的命令、键与通道

Redis ACL 可以限制命令、键模式与 Pub/Sub 通道。命令类别便于管理一组相关操作，但类别是产品提供的集合，不能直接代替业务权限。“只读”也不等于“可以读取所有数据”，某些不修改状态的命令仍可能暴露敏感信息或产生大量资源开销。资源范围与命令范围需要同时限制。



ACL LOG记录特定拒绝事件；它不是所有成功命令及业务效果的完整审计账本。

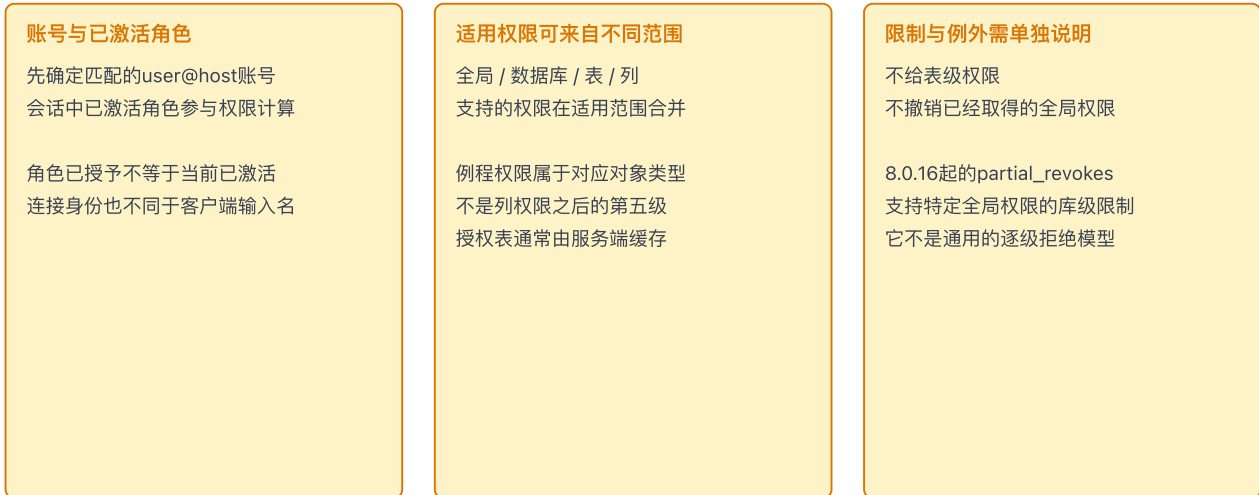
图 6-3 Redis 身份与操作权限检查：会话关联用户，具体命令仍按其操作及资源检查；键权限不自动授予 Pub/Sub 通道权限。

多键命令使这个边界更明显。用户能够访问一个键，不代表能够执行涉及另一个命名空间的操作；模块命令和脚本也需要遵守相应的键与权限约定。Redis 7.x 的 ACL 又包含读写键模式及选择器等演进能力，使用这些规则时应核对客户端和实际小版本，不能把较早示例中的两个条件当成全部 ACL 表达能力。

另一个常见误解是把关闭 `default` 用户当成撤销所有既有会话。用户 `off` 影响后续认证，但既有连接的处理需要按实际规则另行检查。相应地，`protected mode` 是对某些缺少显式防护的部署状态的保护，不是通用的身份与权限系统。它与监听地址、用户状态和网络访问策略共同作用，不能只凭一个开关宣称公网部署已经安全。R6-05

6.3.2 MySQL 的不同作用域不能画成逐级拒绝

MySQL 可以在全局、数据库、表、列与存储程序等作用域授予适用权限。对一般对象访问而言，相关授权可以组合；并非细粒度规则总能覆盖掉粗粒度授权。账号已经具有全局 `SELECT` 时，只在某张表上少授几列，不会自动把全局权限收窄到这些列。R6-06



授权作用域不是五道逐渐收窄的过滤器；权限类型、角色激活与例外机制共同决定结果。

图 6-4 MySQL 权限作用域与有效授权：适用授权共同影响结果，存储程序是不同对象分支；不能理解为从全局到列逐级设置拒绝。

MySQL 8.0.16 起的 partial revokes 可以在启用相关变量后，对部分全局权限施加数据库级限制。这是有范围、有条件的功能，不能扩展成任意表列上的通用拒绝规则。使用它还会影响相关授权语义，需要在实际配置中明确，而不是靠一张“越细优先”的图略过。R6-07

MySQL 8.0 的角色把一组权限命名，并可授予账号；角色授予与角色激活仍需区分。账号拥有某个角色，不保证当前会话已经使用它。排查权限时要结合账号匹配、直接授权、角色和激活状态，而不是只检查某条 GRANT 是否存在。

管理权限也不能仅按“比写权限更危险”排列。FILE 涉及服务端文件访问，但受系统权限和 `secure_file_priv` 等条件限制，并非可以读写任意文件。复制、账户管理和运行状态查看各自暴露不同能力。应用通常不需要管理整个实例，然而迁移或维护账号可能确实需要较大范围；把它们与日常业务账号分开，才能分别设置使用时限与审计。

6.3.3 Kafka 的主体、操作与资源必须同时匹配

Kafka ACL 将主体、操作、资源以及适用的主机条件和允许或拒绝规则组合起来。Topic、Group、Cluster、TransactionalId 等是不同资源类型。可以写某个主题，不等于可以管理消费组；消费者能够读主题，也还需要满足消费组相关操作的权限要求。事务生产者又会涉及不同资源，不能从最简单的生产示例直接推导所有应用所需 ACL。

前缀资源模式便于管理同一应用的一组主题，但也把命名约定变成授权的一部分。若所有 `app-a.` 开头的主题都被授予某服务，未来创建的新主题也可能落入该范围。前缀降低逐项管理成本，同时扩大了命名错误的影响。因此创建权限、命名审核与前缀授权应一起考虑。

对 Apache Kafka 3.9 内置授权器，启用授权器而资源没有匹配 ACL 时，默认只允许超级用户；显式设置 `allow.everyone.if.no.acl.found=true` 才改变这项行为。ZooKeeper 模式的 `AclAuthorizer` 并非默认放行所有人，`StandardAuthorizer` 的默认拒绝也可以通过配置改变。未启用授权器与启用了但没有 ACL，是两个不同状态。R6-08

授权变更后出现拒绝时，实际主体、资源、操作和有效规则共同决定缺少哪项权限。临时授予全资源 ALL 也许能让请求通过，却掩盖了究竟缺少哪项权限，并可能把这个临时状态长期留下。成功请求只能证明当前权限足够，不能证明权限最小；还需要验证应该失败的操作确实失败。

6.4 加密：保护范围不能只写成一个开关

6.4.1 TLS 同时涉及传输保护与对端身份

TLS 可以保护连接上传输的数据，但客户端必须正确验证服务端证书及身份。如果只是加密而不确认连接到谁，客户端仍可能把数据交给不应接收它的对端。证书链可信、访问主机名匹配、证书在有效期内，是相互有关但不同的检查。

客户端证书又增加了另一方向的认证。服务端可以要求客户端持有受信任的证书，不过证书身份如何映射到应用权限，仍由产品机制和部署配置决定。持有有效证书不必然等于拥有某个 Redis ACL 用户的所有权限，也不等于可以执行任意 SQL。连接认证与资源授权不能在概念上合并。

Redis 7.x 的 TLS 能覆盖客户端、复制和 Cluster bus 等连接，具体通道由相应配置启用。可以保留明文与 TLS 端口以支持迁移，也可以关闭明文入口。两种安排各有目的：迁移期需要旧客户端继续工作，最终保护目标却可能要求不再存在可到达的明文路径。只验证 TLS 端口能连接，不能证明所有客户端已经迁移。R6-09

6.4.2 MySQL 的连接要求与客户端验证

MySQL 可以通过服务端及账号条件要求加密连接，也支持不同客户端验证模式。`REQUIRE SSL` 表达账号使用加密连接的要求，`REQUIRE X509` 等条件还可要求客户端证书。但服务端对客户端提出要求，不会替客户端完成对服务端身份的正确验证。两端配置都属于同一条连接的安全条件。

这在代理场景中尤其重要。应用到代理的一段使用 TLS，不等于代理到数据库的另一段也受保护；如果 TLS 在代理终止，代理就是能够看到明文的受信任组件。架构图应把终止点标出来，才能知道证书、私钥和明文分别出现在哪里。把多段连接统一画成一根“已加密”箭头，会遗漏真正的责任边界。

6.4.3 存储加密取决于文件与密钥的管理

InnoDB 的静态数据加密针对其支持的表空间和日志范围。密钥由相应 keyring 组件或插件管理，MySQL 8.0 的不同小版本、发行版与组件具有差异。社区版并非只有一种永远不变的旧插件形态；具体安装应按对应版本确认组件支持。企业版还提供更多密钥管理集成。R6-10

这里最重要的边界是，文件加密主要防止没有解密能力的人读取被取得的文件。已经通过数据库授权执行 SELECT 的用户，通常仍会得到正常明文结果；主机上能够访问数据和密钥的高权限主体，也不能仅靠表空间加密隔离。加密保护的是明确的获取路径，不是对任何攻击者都不可读。

数据文件、redo log、undo log、binlog、relay log 和备份又有各自的配置与支持范围。只给表空间加密，不能自然推出其他记录已加密。恢复时还需要与数据匹配的密钥及其管理配置；拥有备份文件而失去必要密钥，可能无法恢复。备份验证因此必须包含解密与实际装载，而不是只确认文件复制成功。

Redis 和 Apache Kafka 的常见开源部署可以通过文件系统、磁盘或外部存储服务提供静态加密，这与应用内置的表空间加密属于不同实施层次。远程存储对象的加密、访问凭证和副本数据也需要纳入范围。是否必须在应用内实现，取决于防护对象与运维要求，不能据此直接给产品排安全高低。

6.4.4 Kafka 要逐条说明通信路径

Kafka 客户端、Broker、副本和控制器之间存在不同连接。某个监听器使用 TLS，不会自动给所有连接加密；节点间选择哪个监听器与安全协议，需要显式配置。集群内部流量也可能经过不同主机或网络设备，数据敏感性不会因为流量被称为“内部”而改变。

第 5 章提到 Kafka 3.9 的 SSL 路径不能使用所述 `sendfile` 优化。这是加密成本的一项具体来源，此外还有握手、数据加解密、缓冲管理等工作。证书算法主要影响认证与握手部分，长连接的大量数据则主要经过记录保护路径。实际占比受连接寿命、消息量、硬件和实现影响，不能用一个固定的“吞吐只剩六到八成”代表所有场景。

评估性能时，应把握握手密集与长连接稳态分开，把相同消息大小、确认方式和硬件下的结果放在一起。若必须满足传输保护要求，性能预算要在这个前提下测量，再调整批量、连接复用或资源。不能先以无保护配置取得一个目标数字，再把保护要求描述成可随意舍弃的附加成本。

6.4.5 轮换不只是生成新证书

证书更新需要考虑信任方的更新顺序。新证书由新 CA 签发时，客户端如果还未信任新 CA，服务端切换后就会拒绝连接；反过来，长期同时信任已经应当退出的 CA，也会保留旧风险。一次轮换的完成条件应该包含旧信任退出，而不只是新连接成功。

密钥轮换与数据重新加密也不是同义词。某些方案更换用于保护数据密钥的上层密钥，实际数据页不必立即全部改写；具体机制要按实现确认。对运维而言，无论采用哪种方式，都要能证明旧数据仍可恢复、新数据使用正确配置，以及回退不会重新依赖已经销毁的密钥。

6.5 审计：记录了什么，仍然不知道什么

6.5.1 拒绝日志、慢日志和完整审计有不同目的

Redis 的 ACL LOG 记录与权限相关的拒绝事件，SLOWLOG 关注超过执行时间阈值的命令。二者都有排障价值，但都不等于所有成功读写的完整审计。没有看到拒绝记录，只能说明相应记录中没有捕获到拒绝，不能证明没有发生越权或凭证滥用。

记录字段同样有边界。审计需要关联身份、操作、目标、时间及结果，但不同日志未必同时提供这些信息。若应用共用一个用户，日志即使完整记录该用户的行为，也无法直接区分是哪一个应用实例或最终用户发起。第 6.2 节的身份划分在这里影响可追踪性，审计端无法凭空还原上游已经丢掉的身份。

MONITOR 可以观察命令流，却不是无条件适合长期生产审计的替代品。它会增加处理负担，并可能暴露敏感操作信息。是否采集某种日志，需要同时考虑覆盖范围、性能和敏感数据，而不能把“能看到更多”直接等同于“更安全”。

6.5.2 MySQL 日志要按事件含义选择

MySQL General Log 记录连接和收到的语句，慢查询日志关注特定执行条件，binlog 面向复制与恢复。它们记录不同事件，收到语句不证明事务最终提交，binlog 中的变化也不直接包含所有查询行为。把这些文件拼在一起，仍需理解它们的时点与标识才能重建过程。

MySQL Enterprise Audit 提供专门的审计能力，其他发行版或外部组件也可能提供不同方案。选择时需要确认实际产品、插件和过滤策略，而不是按“社区版只能开 General Log”概括全部可能性。本书不把某一个商业组件的存在当作合规完成；事件覆盖、保留、权限及取证流程仍需落实。

如果需要证明谁修改了某项业务状态，数据库账号可能只对应共享连接池。此时还需要应用请求标识、业务主体和事务结果的关联记录。应用记录与数据库记录各自不能替代另一方：前者保存业务上下文，后者更接近实际执行。组合之后仍要处理时钟、重试和失败时的记录缺口。

6.5.3 Kafka 的授权记录不包含完整消息处理历史

Kafka Authorizer 的日志可以提供授权决策信息，但开启哪个级别、记录哪些事件，要以实际日志配置为准。某个写请求被允许，只说明权限检查通过，不直接证明每条消息最终获得何种确认；某个消费请求被允许，也不能证明应用已处理结果或完成外部副作用。

客户端拦截器可以补充应用侧记录，但无法被当作不可绕过的服务端控制。一个未安装拦截器的客户端，或者已经被修改的应用代码，可能不产生预期记录。如果审计目标要求所有访问都有可验证痕迹，记录点必须位于不能被相应调用者绕过的位置，或者明确承认这一覆盖限制。

把日志输出到独立系统，可以减少业务节点故障导致的丢失风险，但也增加了传输、积压和收集端故障问题。异步审计需要说明最多允许丢失多少记录，以及队列满了以后怎样处理；同步审计则把记录服务的可用性加入业务请求路径。这是业务与审计要求共同决定的取舍，不能默认采样永远足够。

6.5.4 审计数据本身也需要保护

日志可能包含账号、对象名、查询条件或其他敏感信息。为了审计而复制这些信息，会形成新的数据副本和访问范围。记录哪些字段、哪些字段需要遮蔽、谁能读取和删除，应该与业务数据一样有明确约定。

留存时间也不是越长越好。太短会缺少调查所需证据，长期无差别保留则增加暴露范围和存储负担。若需要证明记录没有被修改，还要考虑追加写存储、完整性校验及管理权限分离。普通文本日志存在，并不自动具有不可抵赖或防篡改性质。

我更愿意从一次具体事件倒推审计需求：若应用账号被误用，哪些记录能确认影响的资源和时间，哪些信息只能估计。这样可以看清当前证据是否足够，而不是先把所有日志开到最大，再希望事后总能查出来。

6.6 横向对比：能力要与部署状态一起记录

表 6-1 三种系统的安全责任与边界

问题	Redis 7.x	MySQL 8.0.x	Kafka 3.9.0
应用身份怎样建立	ACL 用户与密码；TLS 可校验连接证书	账号匹配及认证插件	SASL 机制或 TLS 身份等配置
资源怎样划分	命令、键模式、通道及相关规则	全局与对象作用域、角色和适用限制	主体、操作、资源模式及允许/拒绝规则
默认状态要检查什么	default 用户、密码、监听与 protected mode	初始化方式、现有账号插件与权限	监听协议、是否启用授权器、无 ACL 行为
传输保护怎样确定	客户端、复制、Cluster bus 分别配置	客户端、代理、复制等连接分别确认	应用、Broker 间、控制面连接分别确认
静态数据怎样保护	操作系统或外部存储等方案	支持范围内的 InnoDB 与日志加密、keyring	本地介质与远程存储等方案
原生日志不能证明什么	ACL LOG/SLOWLOG 不覆盖全部成功访问	收到 SQL 不等于事务提交或最终用户身份	授权通过不等于消息已处理或外部效果完成

表中的差异不能形成一个统一的权限粒度排名。列权限和键模式限制不同对象，主题权限也不能直接与列数比较。真正有意义的是当前业务的授权单位：应用是否需要读取某些字段，是否需要共享主题，是否需要在同一实例隔离键空间。资源模型不合适时，强行叠加复杂规则可能比拆分数据更难维护。

同样，某项能力可选，并不证明产品故意把安全排在性能后面。要解释历史设计动机，需要相应记录；本章依据当前可观察的机制与代价作判断。默认配置、可配置能力和具体部署状态应分别记载，避免把一份演示配置写成产品永恒属性。

6.7 架构启示

6.7.1 默认值需要与完整安装路径一起检查

从源代码默认值、发行版配置、容器镜像初始化到实际启动参数，最终状态可能经过多次覆盖。只引用产品文档的一项默认值，不能证明当前实例就是那个状态。第 3 章讨论配置加载，本章进一步说明它为什么会改变信任边界。

开篇事件中，关键是未授权者能够访问并写入测试实例。整改首先要阻断这条路径，再区分应用身份与操作范围。用一条“默认安全”的产品标签代替实际连接测试，仍然可能漏掉另一个监听入口或旧账号。可观察的拒绝行为，比配置名更接近需要验证的目标。

6.7.2 最小权限要验证允许与拒绝两侧

应用正常运行，只证明它拥有足够权限，不证明没有多余权限。一个只需要写自己主题的服务，也许同时拥有删其他主题的能力；一个查询用户，也许继承了全局权限。权限评审需要从实际业务操作推导允许集合，再对越界操作做受控验证。

Redis 的 ACL DRYRUN 提供了不实际执行命令的检查方式，MySQL 与 Kafka 也能通过查看有效账号、授权规则以及隔离环境中的请求验证来检查边界。方法不必完全相同，目标是一致的：被允许的必要操作能够完成，被禁止的操作不会因另一条广泛授权而放行。

6.7.3 纵深防御要有不同的失效条件

网络限制、认证、授权和加密可以降低不同路径的风险，但不能据此说攻击者必须逐层攻破所有措施才能获得数据。若攻击者已经控制了有权解密并读取数据的应用进程，某些保护并不会阻止其正常权限内的访问。多个措施是否独立，要根据攻击者已经取得的能力判断。

因此，我不会只数安全图上有几个框。更有用的是分别设想一个条件失效：密码泄露后权限能否限制影响，应用主机失陷后其他服务是否仍隔离，磁盘被复制后密钥是否同时暴露。这样才知道措施之间有没有真正提供额外保护。

6.7.4 身份跨过服务边界时，需要明确保留还是转换

应用连接数据库时可能使用服务账号，原始用户身份只存在于应用内；Broker 处理客户端请求与节点间复制，也使用不同连接身份。下游服务接收到的主体，不一定还是最初的用户。这个问题同样存在于 Redis、MySQL 的代理与应用链路中，不是 Kafka 独有。

若只保留服务身份，权限和审计就只能到服务这一层；若传递原始用户标识，下游还需要判断谁有资格声明这个标识。一个普通请求字段不能自动成为可信身份。委托令牌、受信代理和应用审计各自提供不同能力，选择时应明确需要保留的身份粒度。

6.7.5 轮换、撤销和恢复决定控制能否持续有效

一套认证配置刚启用时能工作，只是生命周期的开始。人员离职或服务下线后如何撤销相关账号，证书到期前怎样更换，密钥服务故障时数据能否恢复，都会影响控制的长期有效性。把这些动作交给外部平台可以减少重复实现，但依赖关系也要进入部署与恢复说明。

安全检查需要明确当前请求从入口到数据、再到日志记录的责任，而非脱离这些责任比较哪个产品更安全。任何一项能力若只有接口、没有实际配置与验证，都还不能算当前部署已经具备。

6.8 小结

那次测试实例被当成免费缓存，并不复杂，却把身份与访问范围缺失的结果直接暴露了出来。更复杂的部署并不会改变这个基本问题，只会增加账号、连接、复制和记录之间的关系。

Redis 的 ACL、MySQL 的账号与对象权限、Kafka 的监听器与资源 ACL，都可以限制具体访问。TLS 与存储加密保护不同范围，审计又提供另一种证据。把这些能力准确组合起来，需要区分产品支持、默认配置和实际部署，而不是凭产品定位替安全方案下结论。

下一章把这些连接放进集群拓扑。节点失联后谁还能服务，副本能否替代主节点，控制面怎样更新所有权，既影响数据保证，也影响本章讨论的身份与权限应落在哪里。

第 7 章 集群架构 — 从单点到分布式

本章导读

有个同事问我：“Redis 内存快爆了，加个从节点是不是就行了？”

这句话把扩容和复制放在了一起。如果新节点保存的是同一份完整数据，原主节点的内存压力不会因此消失。副本可以为故障后的接管提供数据，也可能分担符合条件的读取，但要容纳更多互不重复的数据，通常需要把数据分开放置。

集群中的很多误判，都来自把一种能力当成另一种能力。自动选主不等于已确认写入不会丢失，数据有多个副本不等于任意节点都能返回最新结果，新增机器也不等于热点自动被摊开。本章沿着数据放置、复制、故障转移和元数据管理四个问题，观察 Redis Cluster、MySQL 复制与组复制、Kafka 分区副本的具体安排。

第 9 章还会深入复制位点与断线恢复。本章先把节点放进拓扑，明确谁保存什么、谁能接受请求，以及一段网络中断会改变哪些行为。

7.1 问题的本质：扩展、冗余和协调分别解决什么

7.1.1 先确定单机遇到了哪一种上限

内存容纳不了数据，写吞吐达到设备上限，一个热键占满执行线程，以及机器故障后没有接替者，是四种不同问题。分片可以分摊部分容量与访问负担，副本可以保留冗余，但并不保证每一种压力都能被新增节点解决。

如果全部流量集中在一个不可拆分的键或分区上，把其他数据搬走只能释放周边资源，不能让同一个串行操作自动并行。若主要压力来自大范围聚合，分片后还可能增加跨节点读取与结果合并。容量规划不能只画节点数，还要标出一个操作触及多少分片，以及最忙的那一份数据有多大。

副本与备份也不同。副本持续接收变化，因此一次误删除可能很快被复制；备份保存某个可恢复时点，需要独立保留与验证。副本主要改善节点故障时的数据可获得性，备份还用于回退到过去。把同一错误复制到多台机器，并没有形成回到错误之前的能力。

7.1.2 分片边界也是操作边界

按范围分片，邻近键容易一起访问，但某个持续增长的范围可能成为热点。按哈希分片，键分布可能更均匀，范围查询却往往需要访问多份数据。这里的均匀性还取决于键及访问分布：键数相等，不代表每个分片收到相同流量。

分片映射可以再加一层中间单位。Redis 的槽把键映射与节点分配分开；Kafka 的分区既限定日志顺序，也是并行处理的单位；关系数据的分片方案则还要处理关联查询与事务范围。选择粒度时，需要同时考虑迁移成本与跨分片操作，不是单纯把单位分得越细越好。

复制是在同一份分片数据之外再放置冗余副本。若有三个分片，每个分片保存三份，逻辑数据仍是三份互不重复的内容，物理上却有九份副本。这个数量示例只用于区分容量口径；实际可用空间还要扣除日志、索引、恢复和迁移所需余量，不能直接把磁盘总容量除以副本数当作可写满的上限。

7.1.3 故障检测不能仅凭超时确定原因

一台节点长时间没有回应，可能是进程退出，也可能是网络中断、暂停或过载。观察者通常无法仅凭超时区分这些原因。超时越短，发现真实故障可能越快，但误判和不必要的切换也可能增加；超时越长，则延长部分请求的等待。这是一项具体的检测取舍。

多个观察者达成一致，可以减少单个观察点的偶然影响，但不能保证被怀疑的节点真的已经停止执行。故障转移还需要限制旧主继续作为当前主节点产生影响：例如通过任期、配置版本、资格检查和拒绝旧请求等机制。只选出一个新主，而不处理旧主，仍可能留下两个执行路径。

多数派有交集，是共识协议可以利用的条件，但不是独立的正确性证明。成员如何持久保存投票与日志，新任期如何继承已确定历史，旧节点如何被约束，都决定了协议能否维持保证。不能把“过半投票”推广为任何集群中绝不会出现冲突。

7.1.4 CAP 讨论的是特定保证，不是产品等级

CAP 中的一致性通常指线性一致性：操作应当表现得像按一个满足实时先后关系的顺序执行；可用性要求非故障节点收到的请求能够得到相应响应。在网络分区条件下，系统无法同时无条件满足这两项要求。它不是把缓存、数据库和消息系统排成三个固定位置的公式。R7-01

不能仅凭异步复制中的丢写窗口给系统贴上 AP 标签。丢失已确认写入、读取旧值、拒绝少数派写入和跨节点事务，是相关但不同的问题。一个系统可以在分区后停止某些写入，同时仍因此前的异步复制窗口丢失已确认数据；这正是不能仅靠两个字母理解拓扑的原因。

比较时先给出操作和故障：写已返回成功，随后哪个节点失联，剩余节点保存到哪里，谁有资格接管，客户端又被路由到哪里。这个顺序比先决定产品属于哪一类，更容易发现一项保证的实际边界。

观察样本	容量与数据副本	角色协调
Redis	Cluster按槽分片 主从复制保有同一分片的副本	Sentinel管理非Cluster故障转移 Cluster有自己的探测与选举机制
MySQL	普通复制不把一张表自动分片 副本可能承受限制的读取	半同步改变等待点 Group Replication管理成员与事务协调
Kafka	分区划分日志与并行任务 同一分区副本保存重复记录	分区Leader由控制面管理 KRaft协调元数据日志

这是可组合能力的分工，不是三个产品必须依次经过的演进阶梯，也不是CAP排名。

图 7-1 扩展与容错能力的组合：分片、副本、故障转移和元数据管理可以组合，图中不是所有部署必须依次经过的升级路线。

7.2 Redis：槽、复制与故障转移分别承担责任

7.2.1 主从复制保留数据副本，不拆分键空间

Redis 副本通过复制协议跟随主节点。首次建立完整状态与断线后续传，涉及快照、复制流和 backlog，第 9 章会展开。本章先保留一个事实：普通主从复制下，副本保存与主节点对应的同一份数据，主要写入仍由主节点处理。

从副本读取可以分担部分读负载，但要接受相应延迟和读取语义。如果应用刚写完就需要读取自己的结果，任意选一个尚未追平的副本可能返回旧值。增加副本提供了更多可选读取位置，也增加了需要跟踪的状态，不能只把读吞吐相加。

默认异步复制不会让每次普通写入都等待副本确认。客户端已经收到成功时，相关变化可能还没有进入可接管的副本。如果主节点在这时永久不可用，或者故障转移选择了缺少变化的副本，已确认写入可能丢失。**WAIT** 等机制能够让调用者额外等待复制确认，但其结果与故障转移选择、持久化配置仍需一起判断，不能把它直接称为线性一致性开关。R7-02

7.2.2 Sentinel 有两道不同的门槛

Sentinel 监控一组主从节点，并协调故障转移。单个 Sentinel 根据自身观测形成主观下线判断；达到配置的 quorum 后，可以形成客观下线判断。之后还需要足够 Sentinel 授权某一轮故障转移，由其中一个执行者推进。这两件事不能都简单称作同一次多数派投票。R7-03

quorum 是配置的故障判断门槛，授权故障转移又要求满足多数派等条件。把 quorum 调小，不代表少量孤立 Sentinel 就可以不受约束地接管。相反，把 quorum 设得过大，也可能让真实故障发生后无法及时达到判定条件。部署时需要按实际 Sentinel 数量和网络故障域共同检查。

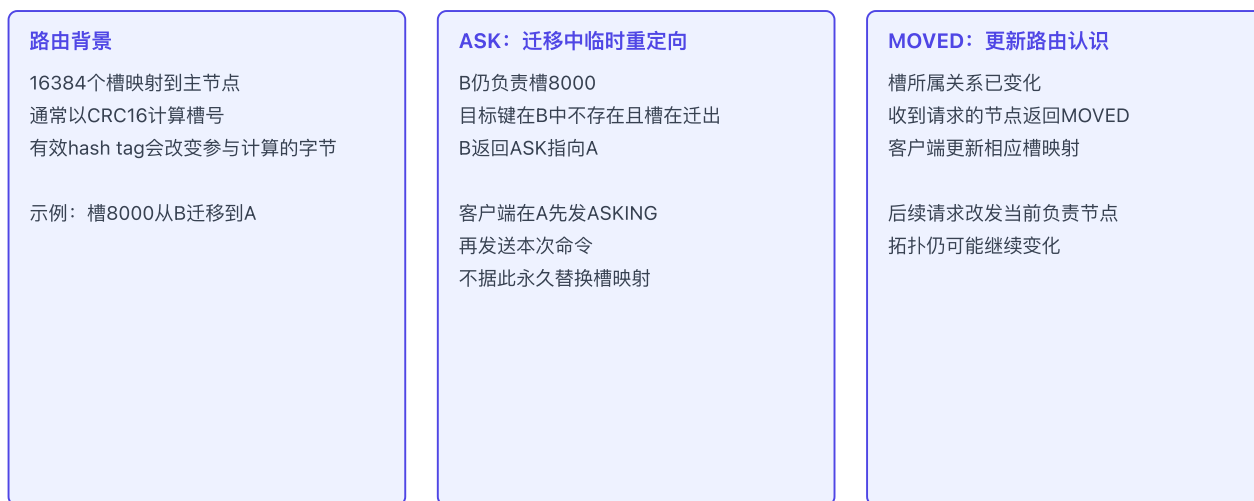
选取新主还需要排除不适合接管的副本，再依据优先级、复制进度及标识等规则选择。把优先级与进度画成一张简单排序表，不能省略候选资格与断连状态。自动化可以按既定条件重复执行转移，但不能据此证明哪个副本具有所有刚刚确认的写入。

Sentinel 的协调与 Redis 数据复制分属两项责任。监控者同意转移，不证明原主真的停止，也不证明新主拥有全部数据。客户端需要发现新主、重新建立连接，并处理进行到一半的请求。对于超时写入，客户端可能不知道原主是否已经执行；新主地址正确，并不能消除这个结果不确定性。

7.2.3 槽把键映射与节点数量分开

Redis Cluster 将键空间分成 16384 个槽，通常用 `CRC16(key) mod 16384` 计算键所属槽；哈希标签会改变参与计算的键内容。槽再分配给主节点，因此增加节点时可以迁移一部分槽，而不必因为节点总数变化就重新定义每个键的哈希结果。R7-04

16384 位的槽位图对应 2048 字节，这是位数与字节之间的直接换算。它不代表整条集群消息只有 2KB，也不代表任意规模集群的通信成本固定。节点身份、消息头、状态传播以及连接数量都还会增加开销。数量在这里用于解释表示方法，不是用来承诺集群可以无条件扩到多少节点。



图示单键迁移路径：多键请求与迁移中状态还可能受到额外约束。

图 7-2 Redis Cluster 槽映射与重定向：键先映射到槽，槽再关关节点；迁移中需要区分长期映射更新与单次重定向。

MOVED 表示请求所对应槽的服务位置需要按返回地址处理，客户端通常会更新或刷新映射；**ASK** 则用于迁移过程中的单次重定向，客户端在目标连接上发送 **ASKING** 后执行本次请求，不据此永久替换整个槽映射。**ASK** 不意味着迁移完成后槽一定回到旧节点，它只是避免把过渡状态误当最终拓扑。

哈希标签可以让相关键落在同一个槽。例如，采用相同非空标签的库存键与订单辅助键可以被放在一起，以满足某些多键操作的槽约束。但这个选择同时集中数据与访问：如果所有业务都使用相同标签，就可能把整个集群重新变成一个热点槽。局部原子性与分布式并行度，在这里通过数据放置产生了联系。

7.2.4 迁移要同时处理数据和路由

槽迁移不是只改映射表。数据从源节点移动到目标节点期间，部分键可能已经过去，其他键还在原处，客户端也可能保存旧视图。重定向协议用于协调这一阶段，但多键操作还可能遇到暂时无法按原方式完成的情况。所谓在线迁移，应该解释哪些操作继续可用、哪些需要重试，而不是理解为完全没有可见影响。

迁移本身会占用源与目标的 CPU、内存和网络。一个已经接近容量上限的节点，未必有足够余量完成数据移动和复制；迁移速度越快，也可能越明显地影响正常请求。扩容应当留出执行扩容所需的资源，而不能等到最后一点容量用尽才开始。

7.2.5 状态传播与选主资格不是同一个机制

Redis Cluster 通过集群总线传播成员、槽和故障等信息。Gossip 帮助节点交换观察，但故障标记、配置版本和副本晋升还有各自的规则。不能因为状态通过 Gossip 传播，就把整个系统概括成“所有节点对等、只需最终一致”。持有槽的主节点与副本在故障转移中的职责不同。

候选副本晋升需要获得相关主节点的授权，并结合配置 epoch 等信息更新槽所有权。这里的多数派按主节点等协议条件判断，不是按谁持有最多槽数判断。一个节点拥有很多槽，并不会因此获得更多选票。把槽数量与投票节点数量混在一起，会误判某种故障分区是否有接管资格。

集群是否继续为其余槽服务，还受到完整槽覆盖等配置影响。即使一侧具有多数主节点，如果缺少某个不可达主节点的可用副本，也不能假定所有数据立即恢复。选举资格、数据所在位置和对不完整覆盖的服务策略，需要同时满足相应条件。

7.2.6 从库存事故看已确认写入的边界

我做过一个库存服务，使用 Redis Cluster。大促那天发生网络分区，旧主在失去联系后的短暂窗口里仍接受写入，随后发生主从切换；网络恢复后，那些未保留在新主上的扣减被覆盖，订单与库存出现了矛盾。

这段过程不能简化为“为了快，所以选择丢数据”。真正起作用的是异步复制、故障检测窗口、新主的数据状态，以及应用把成功响应解释成了什么保证。即使没有发生这次网络分区，主节点确认后、复制完成前的故障，也需要单独考虑。

分区持续足够久时，少数派一侧的旧主会依据集群失联规则停止相应服务；这不代表此前窗口内的写入自动进入新主。多数派一侧能否恢复，还要有合适副本等条件。因而“会停写”和“可能丢确认写”可以同时成立。了解这条时间线，比争论一个固定 AP 或 CP 标签更接近事故原因。

第 10 章会把库存系统作为设计推演的起点。本章先保留结论的边界：自动故障转移缩短了部分故障的恢复流程，却没有替业务完成不可丢失状态的设计。

7.3 MySQL：复制、提交与读取位置需要共同配置

7.3.1 基于 binlog 的复制把接收与应用分开

MySQL 复制源输出 binlog，副本接收相关事件并写入 relay log，再由应用线程推进本地状态。MySQL 8.0 可以配置并行应用，不应把全部 8.0 副本都画成只有一个 SQL 线程逐条执行。网络接收与事务应用分开以后，接收快于应用时，relay log 可以保存尚未处理的工作，但积压仍占用空间并增加读取延迟。



多个副本仍共享Source的网络、日志保留和发送资源，不能理解成完全互不影响。

图 7-3 MySQL 接收、持久化与应用的职责：源写 binlog，副本接收进入 relay log，再由一个或多个应用工作线程执行；接收进度不等于应用进度。

基础异步复制不等待副本应用完成才向客户端确认。主库本地提交与副本上的可见结果，存在时间差。若需要写后立即读到结果，读请求应选择满足相应条件的位置，或者显式等待进度；任意读写分离并不自动提供这一语义。

GTID 标识事务，已执行 GTID 集合记录已经执行的事务，有助于发现不同来源之间缺少哪些事务。它不让所有节点拥有一个可直接比较大小的全局序号，也不保证对应 binlog 仍然保留。换主时能够识别差异，与能够从当前来源补齐差异，是两个条件；第 9 章将把身份、进度和历史可获得性分开说明。

7.3.2 半同步增加了一种确认，不等于所有副本可读

半同步复制让源在相应等待点等待指定数量副本确认。副本的 ACK 包含将事件写入并刷入 relay log 的语义，不等于应用事务完成。于是，确认成功可以增加故障后的数据可获得性，却仍不能直接证明对该副本发起的查询已经看到新值。R7-05

等待点、等待数量和超时后的行为共同决定效果。尤其是超时降级后继续异步写入，保证会发生变化；故障转移选了没有相应确认数据的副本，也可能失去原本希望保留的事务。选主工具需要检查候选节点的状态，不能把“开启半同步”当作其余转移流程的替代。

已经写入 relay log 但尚未应用的数据，不应一概称为丢失。它可能仍可通过正确恢复和应用保留。这里需要区分的是尚不可见、尚未应用与不可恢复，而不是把三个状态都写成“弱一致”。这种区分也影响故障处置：急于开放新主，可能比多等待一段应用过程承担更大业务风险。

7.3.3 Group Replication 对事务排序与认证进行协调

MySQL Group Replication（组复制，简称 MGR）将成员组织成组。事务的写集等信息参与传播与认证，组通信为相关事务及成员变化提供一致的顺序基础；成员再根据规则判断事务能否提交，并推进本地应用。XCom 是其组通信实现的重要部分。把整个过程压成“一轮 Paxos，多数节点落盘，所以所有读取强一致”，会丢掉认证、应用和本地持久化的区别。R7-06

单主模式将普通应用写入集中到当前主成员，其他成员承担副本角色；多主模式允许多个成员接收写入，但仍需要认证冲突及遵守功能限制。多个节点都能收写，不等于对同一热点行的冲突消失，也不等于吞吐按成员数增长。事务被认证拒绝后，应用仍要处理回滚和重试。

成员保存对应的完整组数据，因此 MGR 不是自动分片方案。新增成员增加冗余与可选择的读取位置，也增加组内传输、应用和管理工作。只要每个成员都要存储并应用同一份业务变化，逻辑数据容量就没有因副本数增加而线性扩大。

7.3.4 组的一致性选项影响读取与切换

MySQL 8.0 提供 `group_replication_consistency` 等配置，用于指定事务在相关场景中需要等待哪些进度。默认行为、主切换后的处理，以及事务前后等待可以采用不同安排。应用从任何成员读取时是否能看到先前已提交的写，必须与这些配置及路由策略一起分析。R7-07

这解释了为什么一个组已经具有可靠的事务排序，读请求仍可能遇到成员应用进度差异。协调层确定顺序，与每个成员已把所有事务应用到可查询状态，不是同一时刻。等待更多应用进度可以提供更强的读取条件，但也会让慢成员或故障影响等待时间。

本地刷盘同样是另一项工作。共识和组通信说明节点如何接受历史，不自动替代设备、日志配置和备份。讨论“不会丢”时必须说明容忍的是单节点故障、多少成员同时故障，还是整个故障域失去存储。脱离失败模型的数学保证，无法指导实际部署。

7.3.5 多数派失联后，不应把读写都概括成停服

组复制在失去多数派通信能力时，不能继续像正常组那样推进需要组协调的事务。故障检测、驱逐超时、失联超时与退出动作等配置，会影响成员等待、退出以及后续可服务状态。某些读取仍可能在相应条件下发生，所以“少数派立即停止所有服务”不是准确的通用描述。

在固定成员集合、依赖过半成员继续决定的简化条件下，三成员组失去一个成员仍能保持多数派，失去两个就不能；五成员组可以失去两个。这个例子说明多数派算术，不证明任意成员故障组合都具有相同恢复时间，也不涵盖强制重组、磁盘损坏等额外操作。

成员分别位于不同故障域才有意义。如果所有副本都在同一台宿主机或同一个会同时失效的存储系统上，进程数并没有变成独立冗余。跨地域放置又会增加通信距离并改变可用多数派分布，不能用“多放几台更安全”代替拓扑分析。

7.3.6 Router 与 Shell 把组件组织成可使用的方案

InnoDB Cluster 将 Group Replication、MySQL Router 与 MySQL Shell 的管理能力组合起来。Router 根据元数据和角色选择后端，Shell 提供创建、检查与维护等管理入口。它们减少了应用与运维需要直接处理的细节，但没有改变“全量副本不是分片”的基本事实。

主节点切换后，新连接可以被路由到新的合适节点；正在执行的事务却不能直接迁移过去。旧连接可能中断，应用仍要判断事务结果并决定是否重试。代理提供地址和角色适配，不会使超时结果从不确定变成确定。这与第 5 章讨论的接口失败语义相同，只是现在发生在多节点拓扑中。

如果需要水平分割关系数据，可以另外采用合适的分片层与数据模型设计。但这会增加跨分片查询、事务和迁移问题，不应简单写成“加上中间件即可同时拥有所有能力”。副本组可以成为一个分片的基础，分片之间仍需新的协议与业务边界。

7.4 Kafka：分区顺序、副本资格与元数据仲裁

7.4.1 分区既划分数据，也限定顺序

Kafka 主题由分区组成，每个分区保存一份有序日志，其副本可以分布在不同 Broker。把不同分区的主副本分散到多台 Broker，可以让它们并行承载请求。但一个单 Broker 部署仍然是可以运行的配置，只是没有跨 Broker 的故障冗余；不能因为产品支持分布式，就说单节点形态在概念上不存在。

分区	Broker 1	Broker 2	Broker 3
P0	Leader / ISR	Follower / ISR	Follower / ISR
P1	Follower / ISR	Leader / ISR	Follower / 已移出ISR
P2	Follower / ISR	Follower / ISR	Leader / ISR

写入经Leader；普通读取默认经Leader，也存在可配置的Follower读取。ISR随进度与条件变化。

图 7-4 Kafka 分区与副本放置：同一分区的副本保存相同日志，不同分区提供独立顺序与并行空间。

生产者决定消息进入哪个分区。对常见的键分区策略，相同序列化键、相同分区数和相同策略有助于保持路由稳定；一旦增加分区、改变序列化方式或使用自定义策略，就不能继续承诺“相同业务键永远在同一个分区”。没有键的消息又可以采用利于批量的分配方式，3.9.0 的具体默认行为应按对应客户端实现与配置理解。R7-08

分区内的日志次序不等于所有业务事件天然按因果关系到达。多个生产者之间的协调、重试和业务时钟仍然影响写入先后。消费者并行处理后，完成次序也可能与取出次序不同。因此，日志提供了一项有价值的顺序依据，但应用仍需决定哪些事件必须串行处理、哪些可以并行。

增加分区可以提供更多独立处理单位，却也可能改变后续同键消息的落点，旧消息并不会自动重新分布到符合新哈希映射的位置。若业务依赖按键顺序，扩分区就是语义迁移，不能只视为容量操作。可以选择新主题迁移、固定路由策略或其他明确方案，但都需要处理过渡期。

7.4.2 分区数量不是吞吐保证

分区增加文件、索引、状态和管理工作，Broker 的网络与磁盘仍有总上限。消费者也要有足够执行能力，且经典消费组中的分区分配限制了同一组可同时有效工作的消费者数量。把消费者数增加到远超分区数，不会自动得到相同比例的处理并行度。

反过来，分区很多也可能只有少数分区很忙。生产者键分布、批量大小和下游处理耗时都影响热点。如果瓶颈集中在一个业务键上，均匀分配其他键并不能消除它。容量评估至少需要观察最忙分区，而不只是集群平均吞吐。

缺少负载条件的单分区消息条数，不能直接用于推算其他系统的分区数。同样的条数，可以对应截然不同的消息字节数、压缩、复制网络和处理成本。更可靠的规划是使用实际负载形态测出单个资源的约束，再为故障转移、重建和增长保留空间。数据暂时均匀时的峰值，不是失去一台 Broker 后仍能维持的保证。

7.4.3 ISR 描述同步资格，不等于所有副本永远一样快

Kafka 为每个分区维护 ISR，即满足相应同步条件的副本集合。Follower 从 Leader 拉取日志，Leader 根据副本追赶情况维护其资格。ISR 可以收缩和扩大，并不要求每一时刻所有成员的日志末端字节完全相等。同步集合描述的是协议所认可的状态，而不是肉眼看到 offset 一样大的瞬间。

正常的干净选举使用符合相应资格的副本，避免直接把缺少已确定历史的落后副本提升为 Leader。若显式允许 unclean leader election，就可能选择不在 ISR 的副本，并承担数据丢失风险。因而“新主只能来自 ISR”需要保留这个配置前提，不能说成任何部署都不可改变的事实。R7-09

ISR 收缩不会免费消除慢副本影响。如果集合缩到写入要求以下，相关请求会失败；如果仍能写，系统的冗余条件也已经改变。恢复中的副本还要传输和写入历史数据，占用其他节点的资源。移出集合是协议处理当前状态的一种方式，不意味着集群因此恢复了原来的容量与容错余量。

7.4.4 acks 与最小 ISR 分别控制等待和准入

`acks=0` 不等待 Broker 返回生产确认，客户端不能据此判断服务端是否接收成功。`acks=1` 等待 Leader 相应追加结果，但不等待所要求的全部同步副本条件。`acks=all` 则按分区同步集合及提交推进规则等待确认；这与本地每条记录都执行 fsync 是不同的机制。

`min.insync.replicas` 与 `acks=all` 共同约束写入能够接受的同步副本条件。例如，一个副本因子为 3、最小 ISR 为 2 的分区，在 ISR 只剩 1 个成员时，不应把写成功当作仍然符合原来的两副本条件。配置关系的意义是让不足以兑现预期冗余的写显式失败，而不是在表中只标一个“强一致”级别。

这里也不能把 `min.insync.replicas=2` 理解成每次只需任意两个副本参与。最小 ISR 是准入与相关校验条件，`acks=all` 的等待依据还涉及当前 ISR 和提交推进。集合可以在请求过程中变化，错误返回还可能出现在已经追加以后；应用不能只凭失败就断言消息绝未进入日志。精确轨迹与第 9 章的 HW、LEO 一起分析。

生产者重试又引入重复问题。幂等生产配置可以约束相应生产会话中的重复追加，事务机制可以扩展某些原子性范围，但它们都不能自动替外部数据库完成一次业务扣款。复制确认、重复控制和外部副作用是三件事，不能用 `acks` 一个参数统一解释。

7.4.5 消费读取与副本复制的用途不同

Follower 保存副本主要服务于冗余和接管，并不意味着应用可以不加条件地把读取平均发往任意 Follower。Kafka 的通常读取路径与配置的副本选择能力有关，基于机架等条件的副本读取又有明确适用方式。本章以常见 Leader 读写路径解释拓扑，不把这个常见安排写成不可变化的协议限制。

消费者还要区分取到消息、处理完成和提交进度。Broker 存在多份日志副本，不能替应用知道外部处理已经完成；读到的消息也可能因为重试再次处理。第 9、10 章会把这些状态放进恢复与业务流程，避免只从 Broker 拓扑推导端到端保证。

7.4.6 KRaft 管理元数据，不替业务分区运行同一套选举

Kafka 3.9.0 可使用 KRaft 管理元数据。本书以该模式为主：控制器仲裁组维护元数据日志，活跃 Controller 负责相应集群管理，Broker 根据元数据变化维护自身视图。Controller 可以与 Broker 合并部署，也可以使用独立进程与资源；不同安排的隔离和运维成本不同。R7-10

元数据日志在实现中有特定标识与存储布局，但不是应用随意生产消费的普通业务主题。它保存成员、主题、分区及相关配置的状态，访问与恢复方式也不同。把它简称为“元数据也放进 Kafka 主题”，会让读者误以为可以用普通生产者绕过控制面直接维护集群。

这里有两种 Leader。控制器之间通过 Raft 确定活跃 Controller；业务分区的 Leader 则由相应控制逻辑依据副本状态和配置选择。Controller 有共识日志，不等于每个业务分区也使用完全相同的 Raft 提交规则。ISR 和 `acks` 仍属于业务日志的责任范围。

ZooKeeper 模式使用外部 ZooKeeper 与 Kafka Controller 协同管理元数据。KRaft 减少了这类跨系统依赖，并改变了元数据传播和管理方式，但不能据此推导任何规模下故障切换都更快。故障检测参数、元数据量和硬件都会影响恢复时间。旧模式具备接管能力，也不应被简化成没有保护的单点。

7.4.7 控制面失效与数据面失效可以不同步发生

控制器仲裁暂时不可用时，一些已经建立且不需要元数据变更的工作可能继续，而创建主题、变更副本、处理需要选主的故障等能力会受影响。具体行为要看故障持续时间和 Broker 状态，不能把“控制器失联”直接等价为“所有已有分区立即停止读写”。

这种分离也不是数据面永远不依赖控制面。既有 Leader 失效以后，需要新的有效决策；客户端拿到过期地址以后，需要更新视图。如果控制面迟迟无法恢复，数据服务能力可能逐步下降。评估高可用时，应分别测控制面故障、数据节点故障以及两者同时发生，而不是只删除一个 Broker 验证一次生产成功。

7.5 横向对比：把同一故障放进不同拓扑

表 7-1 选定部署形态的职责对照

问题	Redis Cluster 7.x	MySQL 8.0 组复制	Kafka 3.9.0 KRaft
数据如何分开	键到槽、槽到主节点	单个组保存完整对应数据，不自动分片	主题分区分布到 Broker
冗余单位	每个主节点数据的副本	组成员的完整数据副本	每个分区的副本
谁处理主要写入	槽的当前主节点	依单主或多主模式	分区当前 Leader
角色变化依据	故障状态、资格、授权与配置 epoch	组成员视图、模式和相关选举规则	控制面依据分区副本资格及配置
控制协调与数据复制	不同职责，选举不等于复制完成	组通信、事务认证与应用需区分	Controller Raft 与分区 ISR 需区分

问题	Redis Cluster 7.x	MySQL 8.0 组复制	Kafka 3.9.0 KRaft
应用仍要处理什么	重定向、连接变化、确认写损失风险	连接中断、事务结果不确定、读取进度	元数据更新、生产结果不确定、重复处理

表格刻意固定了部署形态。若把 Redis Sentinel、MySQL 默认异步复制和 Kafka 单 Broker 混在其中，结论会改变。产品名称本身没有提供足够前提，部署选择必须进入比较。



探测失联不等于证明进程已停止。角色选择、历史保护和客户端恢复各有条件。

图 7-5 故障检测、接管资格与业务恢复：检测到失联以后，各系统仍要检查接管条件；新角色产生后，客户端与数据进度还需恢复。

一个主节点失联后，还剩哪些判断

在 Redis Cluster 中，先检查其余主节点的可达性与候选副本，再看槽能否被接管及完整覆盖策略；原主确认过的写，可能未进入新主。在组复制中，先看组是否仍能形成有效多数派，再看新主与应用进度、读取策略；原连接上的未决事务仍需要应用处理。在 Kafka 中，先看相应分区是否有可用合格副本，以及控制面是否能够更新 Leader，再看生产确认条件是否仍满足。

这三段过程不能排成简单的快、中、慢。它们检查的状态不同，故障检测参数也不同。比较恢复时间，应从相同起点算到相同终点：是发现失联、产生新主、接受新连接，还是业务重新达到目标吞吐。只给一个“秒级恢复”，可能把其中几段等待漏掉。

从指标确认当前状态，而不是只看节点在线

在 Redis Cluster 测试集群中，`CLUSTER INFO` 和 `CLUSTER NODES` 可以帮助检查槽与节点视图；在 MySQL 组复制环境中，可以查看 Performance Schema 的 `replication_group_members`；Kafka 则可通过对应管理工具查看分区 Leader、Replicas 与 ISR。它们报告的是不同层面的状态，不能互相替代。

例如，Broker 在线而 ISR 缩小，意味着进程存活但复制条件改变；MySQL 成员在线而应用积压，意味着可连接不代表数据已追平；Redis 节点在线而槽不可服务，也需要检查更具体的状态。监控“进程还在”是必要的基础，却不是业务保证的完整指标。

7.6 架构启示

7.6.1 新增节点之前，先写清它将分担什么

开篇“内存满了加副本”的问题，说明节点数量容易遮住数据放置。一个新增节点是保存同一份数据，还是接走一部分数据，是否参与写入，是否仅承担读取，要在扩容之前说清楚。

即使采用分片，也应单独处理热点。某个不能拆开的业务对象可能继续受单节点限制，跨分片事务也可能把原来局部的操作变成分布式协调。扩展收益需要按操作验证，不能以集群总容量增加代替最忙路径已经改善。

7.6.2 失败之后的选择，比成功时的吞吐更能区分方案

ISR 不足时拒写、半同步超时后降级、旧主在检测窗口内尚未停写，是会改变业务结果的具体行为。评审这些选择时，我更关心应用能否识别保证变化，以及能否承受相应后果。吞吐相近的两个配置，在故障时可能需要完全不同的后续处理。

有些业务可以暂缓确认，有些可以排队等待，有些允许经过明确流程补偿。不能先把软件设成最宽松行为，再假定业务一定能够修复。故障后的处理必须与成功响应的含义一同设计；如果二者冲突，需要修改确认流程或部署，而不是只增加一个重试循环。

7.6.3 元数据是有负载和恢复成本的状态

槽映射、成员视图、分区 Leader 和配置看起来比业务数据小，却决定了数据在哪里可访问。规模增长以后，元数据传播、缓存、变更和恢复也会占用资源。一个集群能容纳多少分区或成员，需要考虑这些工作，不能只依据数据磁盘容量。

但元数据也不是集群上限的唯一原因。网络、磁盘、最忙分片和客户端行为都可能先成为限制。KRaft 改善一种管理方式，不等于任何硬件都能支持“百万分区”。容量数字必须和版本、配置、工作负载及恢复目标一起出现。

7.6.4 控制决策与业务数据要分别验证

Sentinel 获得转移授权，不证明新主具有全部写入；Controller 正常选出分区 Leader，不证明客户端先前的失败请求没有进入日志；组复制成员关系正确，也不证明任意读请求都已满足业务进度要求。控制面正确是重要条件，却不是业务端到端正确的全部证明。

一份故障演练记录应同时保留角色变化和数据结果。只看到新主产生，就宣布演练成功，可能漏掉确认写、重复请求和读取旧值。反过来，只核一条记录还在，也不能证明元数据恢复和所有受影响分区已经正常。验证粒度需要对应实际承诺。

7.6.5 可配置参数需要明确依赖与失败行为

Redis 有 WAIT 与副本相关写入限制，MySQL 有半同步与组复制一致性选项，Kafka 有 acks 与最小 ISR。三家都在不同位置暴露了可选择行为。

参数并非越多越灵活。使用者必须理解相互依赖以及条件不足时的结果：等待多久，超时后是否降级，错误能否代表未执行，角色变化后规则是否仍成立。好的配置说明不止列取值范围，还解释这些取值怎样改变一个真实请求。第 9 章将沿着复制时间线继续做这件事。

7.6.6 迁移与恢复需要预留能力

节点故障以后，剩余节点承担更多请求，还可能同时向新副本传送数据。若平时已经把所有资源用满，故障时的冗余数据虽然存在，服务仍可能因重建负载而持续恶化。副本数量表达了数据冗余，却没有表达资源余量。

恢复目标应进入正常容量规划：允许多久恢复冗余，期间是否降低服务，哪些分片优先重建，是否限制迁移速率。这些不是主路径之外可以永久推迟的附加工作。系统长期运行总会经历成员变化，能够完成变化本身，也是容量的一部分。

7.7 小结

分片改变数据放置，副本增加冗余，故障转移改变服务角色，元数据管理使这些变化可以被系统识别和处理。它们可以组合，却不能互相代替。集群比单机多出的能力，需要由这些机制在相同故障条件下共同兑现。

Redis 的槽与重定向、MySQL 的复制与组通信、Kafka 的分区与 ISR，都提供了具体的安排。理解它们，不能停在产品标签或节点数量。更重要的是写清成功响应代表什么、失联后谁有资格继续、剩余数据是否足够，以及应用怎样处理未决请求。

第 8 章把这些状态落到字节布局，检查文件能否定位、校验和恢复；第 9 章再追踪日志历史与复制进度。集群拓扑说明数据应放在哪里，后面两章解释数据实际上怎样保存与追上。

第 8 章 磁盘存储格式：数据写下以后

本章导读

我第一次自己定义文件格式时，在文件头放了版本号，却没有想清楚旧程序怎样处理新字段。三个月后增加字段，旧解析器直接报错，迁移脚本写了整整一周。版本号没有缺，缺的是版本之间的处理约定。后来再看一个格式，我会同时看它怎样写、怎样读，以及换一版程序后还能做什么。

磁盘格式容易被当成实现细节：对象已经定义好了，找个序列化工具存下来即可。但数据一旦离开当前进程，指针、对象生命周期和编译器布局就不再是可靠的解释依据。文件要说明一条记录在哪里结束，哪个字段决定后面的解码方式，损坏后还能信任哪一部分。升级时，读写双方甚至不一定同时变化。

本章比较三个具体对象：Redis 用于重建内存状态的 RDB 与 AOF，InnoDB 支持按页访问的表空间，以及 Kafka 按分区组织的日志段。它们不是三种互斥的存储定律。Redis 同时使用快照与追加记录，InnoDB 在数据页之外也使用日志，Kafka 的历史段还会经历删除、compaction 和远程迁移。真正值得追问的是：每一种结构支持什么操作，省下哪一步，又把工作交给了哪里。

第 4 章已经讨论刷盘与持久化的职责。本章从文件里的字节出发，关注定位、损坏粒度和格式演进。涉及 Redis 的实现细节以 7.x 为范围，具体源码示例取 7.2；MySQL 为 8.0.x 的 InnoDB；Kafka 为 3.9.0。

8.1 文件格式需要明确的内容

8.1.1 字节存在，不等于记录可读

设想一个很小的文件，依次写入用户名和积分。第一条是 `alice` 与 `12`，第二条是 `bob` 与 `300`。如果只是把字符串连接起来，读到 `alice12bob300` 时，程序必须借助外部约定才能分清字段。给字段加分隔符，还要规定字段内容本身出现分隔符怎么办。给字段加长度，则要规定长度占几字节、使用哪种字节序、表示的是字符数还是字节数。

这是格式定义最初的工作：让读取者知道接下来有多少字节、这些字节怎样解释。整数既可以定长保存，也可以根据数值大小采用变长编码。前者定位和解析直接，后者能减少小数值的空间占用。选择何种编码，需要知道数值分布和读取频率；“二进制比文本好”不能代替这两项信息。

文件还需要区分正常结束与意外截断。如果某条记录标明后面还有一百字节，实际只剩二十字节，读取者可以识别不完整。若文件正好停在一条完整记录之后，是否说明这是合法结尾，取决于格式有没有结束标记、记录总数或其他完整性证据。长度检查、校验和、结束标记各自回答不同问题，不能互相顶替。

这些问题并非只有磁盘才存在。网络消息也有边界和版本，内存对象也可能损坏。磁盘格式的特殊之处是时间跨度：写入它的程序已经退出，机器可能更换，原来的依赖可能不再可用，而文件仍然需要解释。

8.1.2 定位单位与更新单位可以不同

业务操作通常很小：查询一个键，修改一行，读取一条消息。底层不会因此每次只处理业务所需的那几个字节。文件系统、存储设备、数据库缓存和压缩器各有处理粒度。

InnoDB 可能为了读取一行而加载一个页；Kafka 可以把多条记录组成一个 RecordBatch；Redis 从 RDB 恢复时需要遍历文件中的对象，而恢复后的查询再通过内存索引执行。这里至少有业务记录、缓存单元、编码单元和存储读写单元，未必相等。

一个单位变大，固定开销可以被更多记录分摊，但读取少量数据时可能多读、解压或校验一部分不需要的内容。一个单位变小，局部访问更细，却需要更多头部、索引和管理状态。判断单位大小时，至少要同时看连续访问的长度、随机访问比例和可接受的损坏范围。

例如，归档文件主要按小时整批读出，把每条记录都建立独立索引未必划算；一个需要按订单号点查的存储，把整天记录压成不可分割的一块则会增加点查代价。这不是“批量与随机只能二选一”：可以分块压缩、块间建索引，再在块内扫描。代价落在块大小、索引体积和解码次数之间。

8.1.3 当前状态、历史变化和派生索引

格式分析还需要区分三种内容。当前状态说明某个时刻数据是什么；变化记录说明如何从先前状态走到后来状态；派生索引帮助定位内容。三者可能放在同一个文件，也可能分开保存。

快照可以缩短恢复所需的历史，但无法单靠自身说明快照之后发生了什么。变化记录可以重放，但必须知道起点、顺序和哪些记录属于有效历史。索引通常可以重建，不过重建要花时间，且它引用的原始内容必须仍然存在。把“可重建”理解成“没有成本、无需保护”，会把启动时的工作遗漏在设计之外。

分析对象	正常服务主要使用什么	文件在什么时刻被读取	本章重点
Redis RDB/AOF	内存中的对象与索引	启动恢复、备份检查及相关持久化流程	整体重建怎样表达对象和后续变化
InnoDB 表空间	缓冲池中的页，未命中时读数据文件	正常查询、页调入、恢复	按页定位与页内记录如何协作
Kafka 日志段	本地或远程的分区日志，借助缓存与索引	消费、复制、恢复、维护	有序记录怎样支持定位与分段管理

观察样本	正常读写主要路径	恢复输入
Redis	命令访问内存数据结构	按配置由RDB或AOF重建内存
InnoDB	请求访问缓冲池中的页 缺页时读取对应数据文件	数据页与redo等恢复状态配合 不能把最新脏页视为文件中已有
Kafka	追加与读取利用OS页缓存 日志段及索引组织记录	恢复日志边界与相关状态 远程存储启用后还有远程来源

本图不展开复制副本；框的大小不表示数据权重或唯一正本。

图 8-1 正常访问路径与恢复路径。图中省略了各系统的复制拓扑，文件位置不直接表示业务权威归属。

Redis 的普通键查询不需要在 RDB 中查找某个键，因而 RDB 不必提供在线点查索引。InnoDB 不能在每次查询前重新加载整张表，因此需要页和树的定位结构。Kafka 的典型读取从一个分区位置向后进行，索引可以先缩小范围，再让顺序扫描找到确切记录。操作路径不同，才使这些格式差异有了解释。

8.2 Redis：让状态可以重建

8.2.1 RDB 的入口、对象与结尾

RDB 是 Redis 的快照格式。文件开头包含 REDIS 魔数和四位 ASCII 格式版本，随后是辅助信息、数据库选择信息及键值对象，结束位置带有结束标记和校验字段。实际顺序由具体类型和操作码决定，不能把示意图理解成每种文件都有同样长度的固定分区。

魔数帮助识别格式，版本决定加载器能否解析后续内容。文件扩展名只是操作系统中的名称，不能替代文件内容检查。反过来，魔数正确也不证明文件有效；后面的类型码、长度、对象内容和校验仍然需要检查。

R8-01：Redis 7.2 RDB 类型及操作码

对象记录要提供足够的信息重建逻辑值。字符串不能只保存一段无边界的内存；集合、列表、哈希等对象还需要表达成员或内部编码。过期时间、数据库编号也会影响重建后的行为。若只把键和值保存下来，却漏掉过期语义，恢复的就不是同一个数据集。

RDB 的对象类型与内存实现之间存在转换空间，但也不是彻底无关。某些紧凑编码有对应的持久化类型，加载器需要识别它们；另一些对象会以元素序列保存，再由加载过程构造当前实现。因而“内存结构升级完全不需要改变磁盘格式”过强。更准确的边界是：文件不保存进程指针，加载器可以转换旧表示，但新增类型是否需要提高格式版本，取决于旧加载器能否解析。R8-02：RDB 保存与加载实现

魔数 REDIS	版本 4位ASCII	可变内容 辅助信息 / 数据库选择 / 键值对象	结束 EOF	校验 CRC64
-------------	---------------	-----------------------------	-----------	-------------

对象解析示意：类型决定后续解释方式，长度按具体编码读取

可选对象元数据 过期 / LRU / LFU	对象类型 类型码	键长度与内容 变长长度 + 字节	值表示 元素、紧凑编码或其他类型内容
---------------------------	-------------	---------------------	-----------------------

长度编码 00 / 01表示短长度；10后可用32或64位	特殊编码 11可用于整数或压缩字符串等表示
----------------------------------	--------------------------

文件条仅示意职责次序，不表示对象均定长；尾部校验可边解析边累计，不必先完整读两遍。

图 8-2 RDB 布局示意。长度、类型和结束标记协同确定对象边界；尾部校验用于发现内容损坏。

尾部校验适合顺序生成文件：计算过程可以随写入推进，最后保存累计结果。读取也可以一边解析一边累计校验，不必先完整读一遍验证、再读第二遍装载。校验失败后不能把已解析成功的局部内容自动认作完整快照；错误处理和是否允许忽略校验是另一个显式选择。

这也解释了备份验证为什么不能只看文件大小。两个文件都占几百兆字节，一个可能是完整快照，另一个可能有对象长度错误或损坏。格式检查能发现其中一部分问题；真正的恢复演练还要检查加载后的对象数量、关键数据以及应用能否继续工作。

8.2.2 紧凑编码省下什么

RDB 的长度编码让常见的小长度少占空间。小值可以直接放进一个字节的`有效位`，更大的值再进入更宽的编码；特殊前缀还能表示整数形式或压缩字符串。7.2 的定义包含多种宽度，不能把某一前缀的所有情况都概括成“接下来固定四字节”。精确解码规则应以对应版本的 `rdbSaveLen` 和 `rdbLoadLen` 为准。

假设一份数据有大量很短的键，如果每个键都用八字节保存长度，长度信息自身可能占去明显空间。变长编码使常见情况更小，但读取者需要判断前缀后才能知道字段宽度。对于顺序加载，这是可接受的额外分支；若每次都要直接跳到第十万个对象，变长对象还需要额外索引，单靠长度编码不能做到。

数字字符串也有类似机会。逻辑值是字符串，并不意味着文件中必须保存同样的十进制字符序列。保存器可以在满足条件时采用整数编码，加载器再恢复其逻辑值。这里节省的是表示空间，不是改变 Redis 对外的数据类型。

压缩的效果同样取决于数据内容。重复的文本与已压缩图片的可压缩性不同，键名、对象元数据和内存分配开销也会影响“内存大小与 RDB 大小”的比例。因此本章不给“1GB 内存必然变成多少兆文件”的通用比例。比较备份体积时，应对同一份数据同时记录逻辑内容、实例内存统计和实际文件大小。

另一个常被忽略的量是加载后的内存峰值。文件紧凑，只说明磁盘表示较小。加载时还要创建对象、索引和分配器管理结构；当旧实例与新实例并存或加载伴随其他工作时，机器需要的内存也不等于 RDB 文件大小。文件格式节约的空间，不能直接当成运行容量。

8.2.3 AOF 保存传播结果，不保存任意客户端对话

AOF 使用 Redis 协议的命令表示记录可重放的写入。RESP 的字符串可以包含任意字节，因此 AOF 命令部分并不是只能包含人类可读字符的“纯文本”。长度和边界仍然必须按协议解释，不能因为用编辑器能看到命令名，就随意按行删除内容。R8-03：RESP 规范

AOF 也不是网络抓包。客户端原始请求与持久化传播命令可以不同：时间相关操作需要避免重放时重新计算相对时间，某些命令会被转换为具有确定效果的传播形式。持久化层需要保存的是重建状态所需的结果，而不只是当时用户输入的字符。具体改写由命令实现及传播流程决定，不能把一条示例规则外推到所有命令。

例如，一个键当时设置为十秒后到期。如果数分钟后恢复时仍执行“从现在起再保留十秒”，这个键的有效期就被延长了。以绝对时间表达过期边界，恢复时才有依据判断它是否已经过期。这个例子说明重放格式需要保留时间语义，但不意味着所有系统都必须采用命令日志；行变化、事件或物理变化也可以成为重放单位。

协议复用减少了一套独立语法的维护，却没有消除恢复的专用工作。启动时要选择文件、按顺序读取、处理尾部不完整记录，并在完成后才进入可服务状态。把文件直接通过普通客户端发送给正在运行的实例，还涉及已有数据、权限、数据库选择和错误处理，不能与正常恢复程序等同。

追加记录还会保留多次被覆盖的中间变化。一个计数器被增加一万次，恢复当前结果不一定需要重放一万条命令；从当前内存生成等价的基础状态，可以缩短文件和恢复过程。这个等价只针对重建的数据状态，不承诺保留所有历史操作供审计。需要审计事件的系统，不能把 AOF 重写当成没有信息损失的归档。

8.2.4 多部分 AOF：先确定有效组合，再回收旧文件

Redis 7.0 起采用多部分 AOF。目录内至多有一个当前基础文件，以及一个或多个增量文件，清单文件 manifest 记录恢复应使用的组合。基础文件可以是 RDB 格式，也可以是命令格式；`aof-use-rdb-preamble` 控制重写基础文件的形式。增量部分继续记录基础状态之后的变化。R8-04：Redis 持久化与多部分 AOF

重写过程涉及三个时刻。重写开始前，已有基础文件和增量文件组成当前恢复序列。开始重写时，父进程启用新的增量文件承接后续写入，子进程从其看到的数据状态生成新基础文件。新基础文件准备好后，新的 manifest 将它与必要增量文件组成新的有效序列，旧文件随后进入可回收范围。



新版本不使用旧单文件方案中将大段增量缓冲拼回临时AOF的完整流程。

图 8-3 新基础文件与增量文件先形成可用组合，再发布新的清单。清单决定恢复集合，旧文件清理是后续工作。

这里改变的是切换对象。重写结果不是孤立的一个文件，它必须与重写期间新增的变化配套。只有新基础状态而没有对应增量，会漏掉后来的写入；只有增量而缺少它依赖的基础状态，也不能重建完整数据。manifest 把这个关系显式保存下来。

发布和清理必须有次序。新组合尚未成为有效恢复入口时，旧组合不能提前删掉；新组合已经生效后，旧文件仍然暂时存在通常只是占空间。两种中断后果不同。实现还需处理写文件失败、清单保存失败、重写失败重试和进程崩溃，不能用一句“改名是原子的”覆盖全部持久化条件。R8-05：Redis 7.2 AOF 清单及重写实现

这种安排与第 4 章的 `fsync` 策略分属两件事。清单正确解决“恢复要读哪些文件”；数据与清单是否及时持久化，影响故障后这些文件是否真的存在且完整。文件组织合理不会自动改变已确认写入的丢失边界。

恢复时间也由组合决定。基础状态较新，通常需要重放的增量较少；重写太频繁又会增加生成基础文件的资源消耗。由此得到的调节依据应是恢复预算、增量增长和重写成本，而不是越小的 AOF 越好。容量评估还要给重写期间的新旧文件共存留空间。

8.2.5 一次格式变化不只影响写入程序

Redis 从旧的紧凑编码演进到 listpack，可以让新的内存实现避免一些旧结构的维护成本；读取旧文件时仍需要识别旧类型并转换。新程序能加载旧表示，不代表旧程序也能加载包含新类型的文件。这两个方向必须分开验证。

回滚问题由此出现：服务升级成功，写出了一份新格式快照，后来因其他原因想回退程序。旧程序是否能读新文件，不取决于“API 没变”，而取决于文件中的格式版本和对象类型。若不能读，回滚就需要保留旧格式数据、执行转换或使用其他恢复来源。

因此格式演进的验收不能只有“新程序能够启动”。至少还要覆盖：新程序读旧文件、新程序写新文件后重启、旧程序遇到新文件时的可识别失败，以及升级中断后的恢复。这个范围来自数据跨版本存活的事实，与是否采用 Redis 无关。

8.3 InnoDB：在页内组织可变记录

8.3.1 页号让定位直接，树结构才让键可查

InnoDB 把表空间组织为页，并通过区、段等结构管理空间。默认页大小是 16KB，`innodb_page_size` 还支持其他规定大小，属于初始化时的重要选择。不同页大小影响行、索引键及空间管理的约束，不能把默认大小当成所有实例的事实。R8-06：InnoDB 页大小配置

固定大小提供一个直接的定位关系：已知页属于哪个文件、页号如何映射到该文件，就能计算它在文件中的位置。这个关系解决的是“第 N 页在哪”。用户提出“订单号 1001 在哪”，仍然需要 B+ 树等索引把键转换成目标页。页号定位的常数成本，不等于任意 SQL 查询都能常数时间完成。

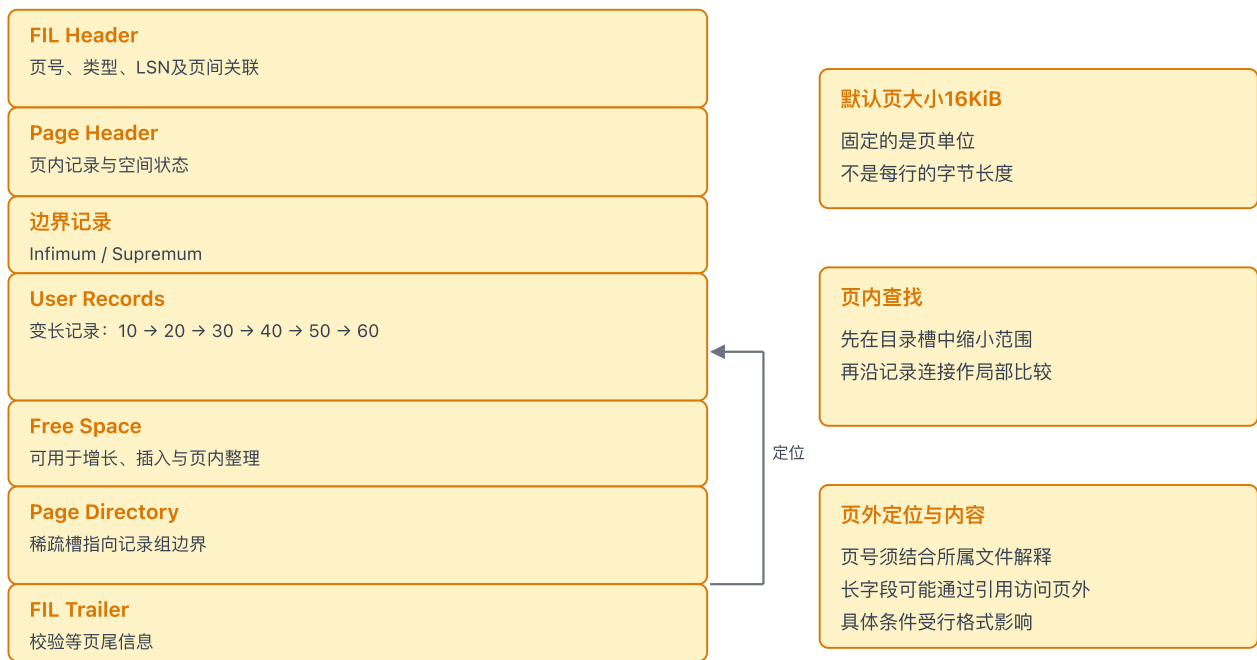
在默认页大小下，一个区由 64 个页组成。区和段为页的分配与回收提供管理单位，索引中的叶子页与非叶子页可以由不同的段管理。空间分配尽量利用相邻位置有利于连续访问，但树中的逻辑相邻页不能据此假定永远物理连续；分裂、删除和重用都会改变实际布局。R8-07：InnoDB 文件空间管理

同一层页之间的连接支持遍历。范围查询到达第一个相关叶子页后，可以继续读取后续叶子页，不必为每一条记录从根重新查起。然而跨页访问是否命中缓存、能否预读、实际读了多少文件块，仍然决定成本。逻辑上连续，不必然意味着设备上的一次连续读取。

页大小还影响缓存和修改的单位。读取少量字段可能带入整页，更新一条很小的记录也可能最终使整页进入刷出流程。更大的页可以容纳更多记录或索引项，也可能增加点查时的无效数据读取。没有负载条件就推断“大页顺扫一定快，小页命中率一定高”，遗漏了树高、记录分布和缓冲池使用方式。

8.3.2 固定页里容纳变长记录

InnoDB 索引页包含页头、页内管理信息、记录区域、空闲空间、页目录和尾部信息。索引记录按照键的逻辑顺序连接，但记录字节不必按同样顺序紧密排列。删除和插入可以留下可重用空间，页内整理则可以重排字节位置。因此“记录按主键有序”主要描述查找和遍历关系，不能直接画成文件里永远没有空隙的一段排序数组。



记录字节位置不必与键序连续一致；这里的数字只是逻辑顺序示例，省略真实槽管理与记录头细节。

图 8-4 固定大小的页包含可变记录及页内定位信息。图示为结构关系，具体字节布局与行格式有关。

页目录为页内查找减少比较范围。目录槽指向记录组的边界，查找先确定目标所属范围，再沿记录连接检查少量记录。它保存的不是每条记录的一份完整副本。页目录的管理规则为组大小设置约束，使局部扫描有明确的结构依据；不能只说“用了二分查找”而省掉二分后怎样找到目标记录。

以一个简化页为例，逻辑键为 10、20、30、40、50、60。目录不必逐条列出六个键，可以保存若干组的边界。查找 35 时，目录先把范围缩到 30 附近与 40 所属的组，再比较记录确定 35 不存在。这个例子只说明两级定位，不对应真实页的最小记录数或精确槽位规则。

变长行格式进一步分开页大小与记录大小。VARCHAR 或可空列需要长度、空值标记等信息；长字段可能部分或全部存放到页外，通过引用关联。DYNAMIC 行格式改善了长变长列的页外存储方式，但不等于每一个长字段都必然离页，也不等于一行一定只读一个页。R8-08: InnoDB 行格式

这对“定长还是变长”的讨论很重要。InnoDB 并未因为要更新记录就把所有记录做成定长。固定的是主要页单位，页内可以保存变长记录，页外还可以继续保存长内容。格式可以在不同层次作不同选择，不能用行是否可修改直接推导整个存储必须选哪一种结构。

8.3.3 从一次变长更新看空间管理

设想一条记录的备注由十字节增长到几百字节。业务层仍然称它为一次 UPDATE，但页内必须容纳增长后的表示。原位置空间不够时，可能需要利用页内其他空闲空间、重组记录，或者触发更大范围的调整。具体行为取决于行格式、索引、字段值以及当前页的状态。

在 B+ 树中插入记录同样可能触发页分裂。原页容量不足，就需要分配新页并调整键的分布，同时更新父节点的分隔信息。若父节点也没有空间，影响还会向上继续。第 2 章解释了树结构为何有利于查询；磁盘格式把这个逻辑结构落实为页的读取、修改、日志记录和后续刷出。

分裂带来的成本不只是多写一页。缓存中的多个页需要被访问，索引结构的连接需要保持一致，事务和恢复机制还必须处理这些变化。插入顺序、主键宽度和行长度会改变页填充与分裂行为，因此“都是同一条 INSERT”并不意味着它们在存储层的成本相同。

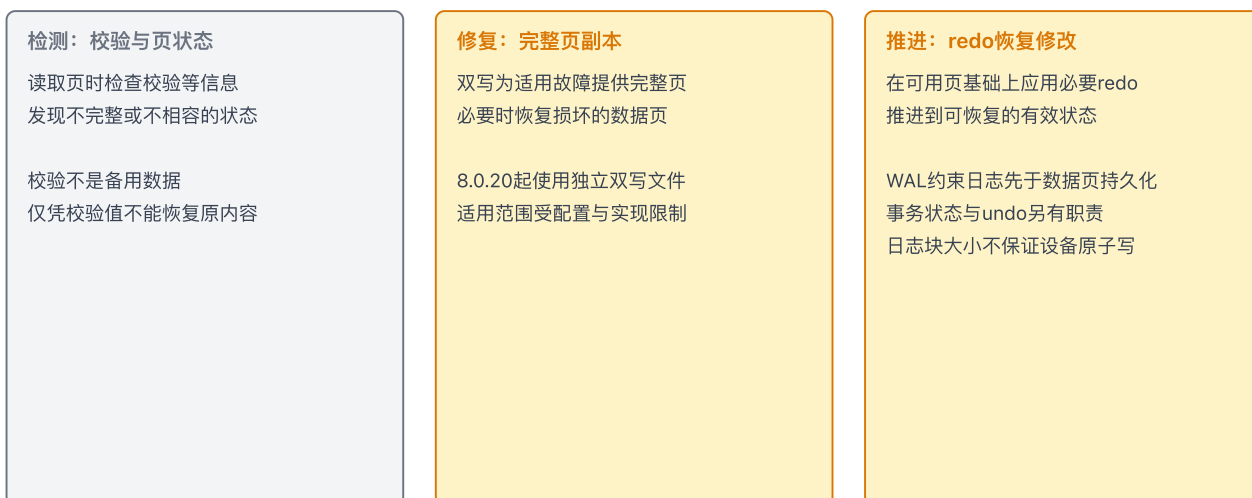
删除也不必立即使文件缩小。记录占用的空间可以先在页内变为可重用，页在某些条件下合并或回收，文件尺寸是否下降还取决于表空间管理与具体维护操作。把 DELETE 后文件没有缩小当成删除失败，是把逻辑可见性、页空间回收与文件归还混成一件事。

一个格式如果允许空闲空间长期存在，就要决定怎样记录空闲区、何时整理、整理是否阻塞前台。把页填得更满可以减少存储和扫描页数，但也可能让后续插入更容易触发调整。空闲空间因此既是浪费，也可能是为未来修改预留的资源；判断需要看后续写入模式。

8.3.4 页完整性与已提交修改是两个问题

数据页的一次写出，可能涉及底层多个写入单位。故障发生在写出过程中，恢复后读到的页可能混有新旧内容。校验和能够帮助发现损坏，却不能单凭校验和算回正确的数据。校验字段保存的信息远少于整页内容，检测与修复不能等同。

InnoDB 的双写机制在把脏页写到最终位置前，先保存可用于恢复的页副本。最终页写入不完整时，可以借助完整副本恢复页，再结合日志推进到可恢复状态。MySQL 8.0.20 起双写文件的组织有所变化，该版本之后的双写内容不再全部位于旧系统表空间位置。R8-09：InnoDB 双写缓冲与文件



页完整性恢复不等于所有事务恢复已完成；双写配置不能仅凭设备宣传的原子写能力决定。

图 8-5 校验发现页内容异常，双写提供完整页来源，redo 支持恢复尚未体现在数据页中的修改。三者不能互相替代。

redo 处理另一种常见情形：事务的相关日志已经持久化，但修改过的数据页还留在缓冲池，数据库此时崩溃。恢复需要根据日志把修改反映到页上。反过来，如果页本身已经损坏，单有“在这个位置做了什么修改”的记录未必足以恢复整个原始页。这正是完整页来源与变化记录分开存在的理由。R8-10: InnoDB redo log

这些机制各有条件。双写能否关闭，不能只看硬件宣传中出现“原子”二字；必须确认所用文件系统、写入方式、设备及 MySQL 配置提供了相应粒度的保证。日志是否足以保住已确认事务，还涉及刷盘设置、底层持久化行为和故障范围。整块存储丢失与一个页写入不完整显然不同，后者的恢复方案不会自动覆盖前者。

将故障分开后，性能讨论也更具体。校验增加计算，双写增加页写入，redo 增加日志记录和持久化工作。系统可以通过批量、异步刷出和其他安排减少部分开销，但不能据此宣称成本消失。评估撤掉某个机制，等于改变可恢复的故障集合，需要把改变写进系统保证。

8.4 Kafka：用分段和批格式保存有序记录

8.4.1 活跃段追加，历史段另行管理

Kafka 的一个分区在本地由多个日志段组成。活跃段接收追加，达到滚动条件后成为历史段，后续写入进入新的活跃段。日志文件与 offset 索引、时间索引等辅助文件共同服务读取。日志段名称中的基础 offset 帮助系统确定这个段对应的逻辑位置范围。

分段为维护提供了单位。基于时间或空间的保留策略可以淘汰符合条件的旧段，无须在每次删除一条消息时搬动整个分区。只要活跃段仍在接收写入，它的维护条件就与关闭的段不同。段太大，淘汰粒度较粗；段太小，文件、索引和轮转管理开销又会增加。

追加也不是“文件永不改变”。面向前台生产请求，新的记录追加到日志尾部；历史维护中，compaction 可以重新生成保留较新键值的段，故障恢复可能截断不属于有效历史的尾部，retention 可以删除旧段。把前台接口与后台文件维护分开，才不会从追加写推出所有文件永久不可变。R8-11: Kafka 3.9 设计与日志管理

分区内的 offset 是逻辑位置，文件字节位置是物理位置。记录长度、批量压缩和清理都会使二者不能简单按固定比例换算。读取 offset 1000 并不是到文件第 1000 字节；索引和批格式负责把逻辑位置转换成实际可读取的范围。

这种区分还影响日志清理后的语义。在开启 compaction 的日志中，某些旧键值对应的记录会被移除，而剩余记录仍保留原 offset。offset 不因此重新编号成无间隙的数组下标。消费程序不能把相邻两次 offset 的差值直接当成它实际处理的业务记录数。

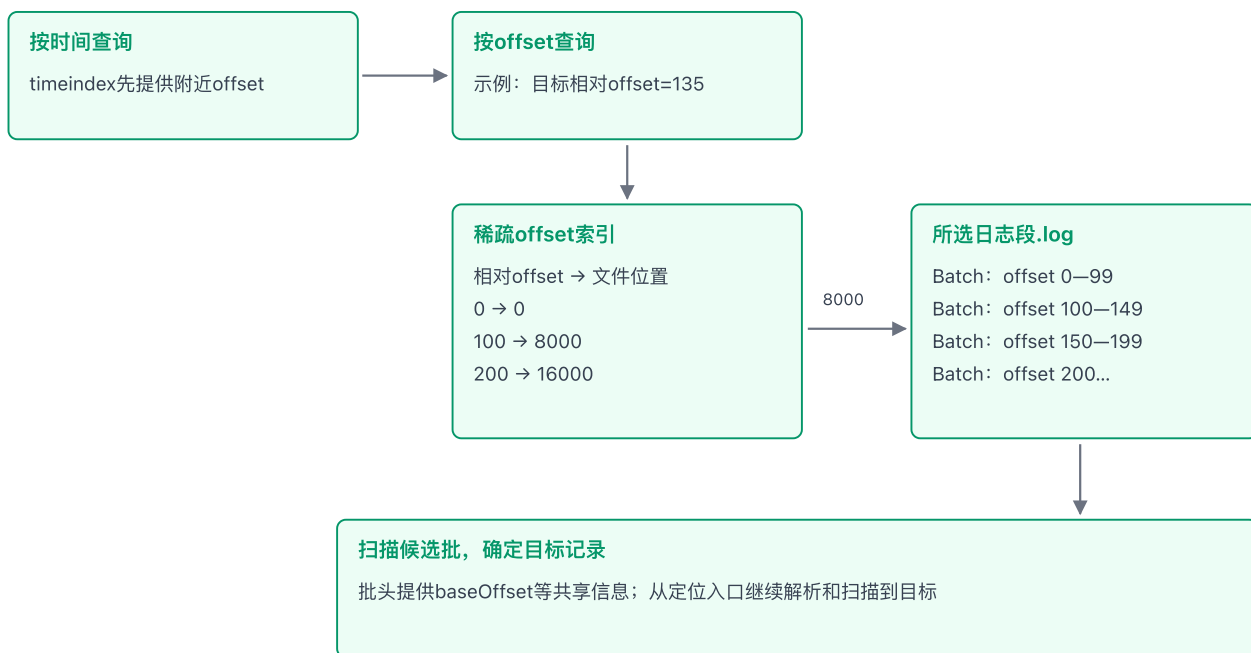
8.4.2 RecordBatch 把共享信息移到批头

Kafka 的 V2 记录格式以 RecordBatch 组织一组记录。批头保存基础 offset、长度、格式标识、校验和、属性、时间戳信息，以及生产者身份和序列相关字段；每条记录保存自身的长度、时间与 offset 差值、键、值和头部信息。精确字段顺序可查 3.9 的格式定义。R8-12: Kafka 记录格式

批量表示的好处可以从重复字段看出来。如果一批记录共享生产者、压缩方式和时间基准，把这些信息在每条记录中完整保存会产生冗余。把共有部分移入批头，记录内部只保留相对变化，可以减少小消息的固定开销。所示 V2 批头固定部分为 61 字节，但一批的总开销还包括每条记录的长度、键值、头部等信息，不能把 61 除以消息数就当成全部存储成本。

压缩也可以在批范围内进行。相近结构的数据通常有更多重复内容，压缩器可以利用这些共同部分；已压缩或高随机性内容可能收益很小。批越大未必越好：生产者等待凑批增加延迟，接收端需要解压更大的范围，内存峰值也可能变化。压缩率、吞吐和尾延迟要在同一负载下比较。

批内差值编码还依赖基准。没有基础 offset 和基础时间戳，记录中的差值就不能还原成完整位置与时间。为了节约每条记录的表示，格式增加了对批头的依赖。这让批成为自然的解析和校验范围，也使批头损坏可能影响多条记录的读取。



索引样例数字不代表默认间隔；条目不是每条消息一项，时间入口也不在日志扫描之后。

V2批固定部分61字节；批次记录可变长并压缩。目标offset与文件字节位置不能按固定比例换算。

图 8-6 批头保存共享信息，索引把 offset 定位到附近文件位置，再读取批并找到具体记录。

生产者 ID、epoch 和序列号不是装饰性的追踪字段。Broker 需要使用它们识别某些重试和生产者代际，恢复时也需要重建相应状态。它们为幂等和事务协议提供数据基础，但单有字段不能推出任意业务副作用只发生一次。第 9 章将讨论它们与复制历史的关系，第 10 章再区分消息写入与外部数据库操作。

8.4.3 稀疏索引留下哪一段扫描

Kafka 的 offset 索引没有为每条记录保存一个条目。条目记录相对段基础 offset 的位置，以及对应的日志文件字节位置。查找先选择相关段，再从索引中找到不晚于目标的条目，之后读取日志批，向前找到目标附近的记录。R8-13: Kafka 3.9.0 OffsetIndex

例如，一个简化索引只有 (0,0)、(100,8000)、(200,16000) 三项，其中第一列是相对 offset，第二列是字节位置。查询相对 offset 135，索引可以把扫描起点缩到 8000 字节附近，却不能直接返回第 135 条记录。读取者仍需解析后续批和记录。这个示例的数字只是说明两种位置的关系，不代表默认索引间隔或实际消息大小。

时间索引提供另一层入口：给定时间，先取得可继续查找的 offset，再通过 offset 索引和日志扫描找到满足条件的记录。时间戳索引条目本身不能直接替代日志中每条记录的时间判断，尤其是生产者时间戳并非严格递增时。R8-14: Kafka 3.9.0 TimeIndex

稀疏索引减少了条目数量，代价是保留局部扫描。它是否值得，取决于索引占用、记录大小、扫描时是否命中缓存，以及查询分布。索引可以更容易留在内存或页缓存中，不代表任意规模下都保证常驻。记录长度也会让一次扫描跨过的字节数变化，不能用索引配置的间隔直接承诺每次只扫描固定几 KB。

与 InnoDB 页目录比较，共性在于“用较少定位信息缩小搜索范围，再完成局部查找”。区别则在规模和维护：页目录管理页内有序记录，Kafka 索引管理日志段里的位置。Redis 的哈希查找不需要勉强归入这类稀疏索引；使用了局部遍历，不足以说明它们具有相同结构。

索引重建还说明了派生数据的价值。索引可由日志重新生成，但重建时必须读取和解释原日志，这会占用启动时间与 I/O。若应用恢复目标很短，索引文件即使不是唯一数据来源，也需要相应的保存、校验和恢复计划。

8.4.4 格式版本改变的是解释能力

Kafka 早期格式按单条消息组织，后来的格式增加时间信息，再以 RecordBatch 把批量与生产者状态纳入共同表示。版本演进不能简单讲成“后一版更省空间”：新字段让系统表达了先前没有表达的状态，也增加了解码和兼容责任。

格式中的 magic 帮助读取者选择解码规则，但它本身不实现兼容。Broker 支持哪些记录格式、客户端支持哪些协议版本、是否存在转换路径，以及转换成本是多少，都需要在升级方案中说明。一个程序能够识别“这是我不支持的版本”，已经比无边界地误读好，但不等于它能够继续服务。

变化	新格式增加的能力	仍需另行解决的问题
增加时间戳	记录可以携带时间语义，支持相应查询与处理	时间来自哪里、是否可信、是否严格有序
以批组织记录	共享字段、差值编码与批量压缩	凑批延迟、批内定位、兼容转换

变化	新格式增加的能力	仍需另行解决的问题
保存生产者身份与序列	为重试识别及生产者状态恢复提供依据	幂等范围、事务边界、外部副作用

字段与保证需要分开。字段提供证据，协议规定如何利用证据。拿掉协议处理，只留下文件中的几个整数，并不能维持相同语义。反过来，协议如果需要跨重启保留状态，文件或检查点中又必须有足够的信息重建它。

8.4.5 远程存储改变放置，也改变恢复路径

Kafka 3.9.0 的分层存储允许已关闭日志段进入远程存储，并通过远程日志元数据维护其位置与生命周期。活跃写入仍在本地日志进行，远程历史读取则通过相应接口完成。远程存储插件、权限、配置和保留策略都是该能力的组成条件，并非把本地目录替换成对象存储路径即可。R8-15: KIP-405 分层存储设计

消费者读取远程历史时，需要获取相关索引和数据范围。索引可以缓存到本地，读取过程也会使用内存缓冲；这些都不等于必须把整个历史日志段重新写成 Broker 的本地 `.log` 文件。描述“缓存到本地”时若不说明对象，容易把索引缓存、读取缓冲与日志段物化混为一谈。

Follower 的追赶还多一条路径。在 3.9.0 中，当所需位置进入远程历史范围，状态机可以恢复 leader epoch 信息、生产者快照等辅助状态，再转入从 Leader 本地日志继续获取记录的流程。实现会查找相应远程段和元数据，并检查远程存储条件；这不是“任何落后都能恢复”的保证。R8-16: Kafka 3.9.0 TierStateMachine

因此，本地日志保留窗口没有覆盖某个落后副本，不足以单独推出它永远无法追赶。还必须检查远程历史和辅助状态是否可用，以及当前版本的恢复路径。与此同时，远程存储不可达、元数据缺失或所需历史已经删除，仍可能阻止这条路径完成。第 9 章会把这些条件与“位点还在不在”分开。

远程段的形成

封闭日志段按条件上传
保存数据、索引及辅助状态
元数据标明可用远程历史

本地和远程保留策略分开
删除本地不等于远程已删除

消费者读取远程历史

按远程元数据定位段
读取或缓存所需索引
从远程读取所需记录

索引缓存不是历史.log整段
在本地永久重新物化

Follower恢复路径

所需位置早于Leader本地起点
进入适用的远程辅助状态恢复
恢复leader epoch与生产者快照
衔接后续本地日志获取

不要求把全部远程历史复制回来
也不保证任意故障都可恢复

前提：已启用远程存储，所需段、元数据与辅助状态存在且可访问；读取与恢复不是同一流程。

图 8-7 消费者远程读取与 Follower 辅助状态恢复是不同路径；索引缓存不等于历史日志段完整落回本地。

分层存储展示了格式复用的价值：记录和日志段组织可以继续服务远程历史，不需要为了移动存储位置重定义消息本身。但上层协议保持兼容，不代表复制实现无需调整。远程元数据、索引获取、辅助状态恢复和删除次序都增加了新的协调工作。

放置变化也会改变性能依据。本地缓存命中、读取本地文件、访问远程对象存储不具有相同延迟分布；具体差距取决于网络、对象大小、请求范围和缓存状态。本章不把远程读取固定成某个毫秒值。容量便宜了多少与最慢历史读取变慢了多少，需要分别测量。

8.5 比较文件职责，而不是给产品归类

8.5.1 同一个维度下比较

表 8-1 不同文件对象承担的工作

文件对象	内容组织	定位依据	常见维护工作	不能单独证明的保证
Redis RDB	某个快照状态的对象表示	顺序解析类型、长度与对象	生成新快照、加载、校验	快照之后的写入是否仍在
Redis 多部分 AOF	基础状态与有序增量	manifest 加各文件内部边界	重写、发布清单、回收历史文件	已确认命令在所设故障下必不丢失
InnoDB 数据页	固定页内的变长记录与索引信息	B+ 树、页号、页目录	分裂、合并、整理与刷出	事务提交状态及完整恢复历史
InnoDB redo	支持恢复的修改记录	日志位置与页状态	写入、刷盘、检查点推进与空间复用	所有页损坏都可只靠日志修复
Kafka 日志段	分区有序记录与 RecordBatch	段基础 offset、稀疏索引、扫描	滚动、保留清理、compaction、远程迁移	消费者业务处理进度与外部结果

表中一个产品出现多次，是因为它本来就组合了多种结构。将 Redis 只叫快照、InnoDB 只叫页、Kafka 只叫追加日志，会隐藏真正有价值的组合关系。

损坏粒度也不同。RDB 的整体完整性信息面向快照文件，InnoDB 的页结构与校验把问题定位到页，Kafka 的批边界支持对批进行解释和检查。这些范围影响故障后的检查与修复方式，却不能直接推导哪一家“更可靠”。副本、备份、日志持久化和读取时的错误处理还在文件格式之外。

8.5.2 一次点查和一次恢复，可能选择不同结构

如果只看正常请求，RDB 没有在线点查索引似乎是缺点；如果它的主要任务是整体重建，这个省略反而减少文件维护工作。InnoDB 正常查询随时需要按键定位，因此把树和页管理留在持久化结构中。Kafka 典型消费需要从位置向后读，段与批让连续处理更自然。

同一个系统还可能需要第二条路径。离线分析需要全表扫描，在线业务需要点查；日常恢复读取最近快照与短日志，审计却需要多年历史。一个格式可以同时支持这些操作，也可以通过派生文件或不同系统分别承担。比较时要计算生成和同步派生数据的成本，不能只展示各自最快的查询。

这也解释为什么快照不能简单替代日志。快照适合回答当前有什么，日志可以保留变化次序；如果业务只需要恢复当前状态，压缩历史可能合理。如果业务还需要追查哪一次操作导致余额变化，删除中间操作就丢掉了重要信息。技术上的可恢复对象与业务上的审计对象可能不是同一个集合。

8.5.3 一份恢复来源也会逐渐过期

变化记录与快照只有在存在衔接关系时才能组合。设快照反映位置 100 的状态，日志从 101 开始连续保留，就具备继续恢复的基本关系；如果日志已删到 150，仅有快照与后半段日志并不完整。反过来，日志从更早位置开始，也需要知道跳过哪些已包含在快照中的变化，避免重复应用。

这个例子使用抽象位置，不把 Redis、MySQL、Kafka 的具体位置结构等同。实际系统还需要历史身份、事务边界、检查点或 epoch 来判断能否衔接。第 9 章将详细讨论它们。文件格式在这里需要保存足够元数据，让恢复程序有证据作出判断。

同理，远程对象仍存在不等于可恢复。若定位元数据丢失、解密密钥不可用、格式读取程序不再兼容，字节虽在，恢复工作仍可能失败。备份清单应包含数据、元数据、所需密钥管理条件和可用读取程序的版本信息，而不是只统计上传成功的对象数。

8.6 从格式分析走到设计判断

8.6.1 先算一次操作实际触及多少内容

我评价一个存储单元，先看一次目标操作会触及什么。读取一条十字节记录却需要解压几十兆内容，问题可能在压缩分块；写入几百字节却需要更新多个索引和页，问题可能在修改的扇出。单看文件总大小，不会显现这些成本。

可以把一次操作分解成定位、读取、解码和维护四类工作。定位包括索引查找，读取包括有效内容与附带内容，解码包括校验和解压，维护包括因为这次更新需要改变的其他结构。减少其中一项可能增加另一项，例如更密的索引减少扫描，却增加内存和更新工作。

这不是要求把所有成本都折算成一个公式。初步分析只要能说明哪个成本在当前负载中占主导，就比“变长更省、定长更快”准确。若测量发现真正的瓶颈是网络往返，继续压缩一两个长度字节可能对端到端延迟没有明显帮助。

表 8-2 格式决策需要的输入

需要作出的决定	必需输入	可以比较的方案	验证指标
压缩块大小	访问范围、内容相似度、内存限制	小块、大块、不压缩	有效读取比例、解压CPU、尾延迟
索引密度	点查比例、记录大小、缓存预算	稠密条目、稀疏条目、分级索引	定位内存、扫描字节、更新成本
基础状态生成频率	增量增长、恢复时限、生成成本	定期、按大小、按恢复预算触发	恢复用时、额外I/O、峰值空间
文件切分单位	清理精度、写入规模、文件管理成本	时间切分、大小切分、二者结合	清理延迟、文件数、资源峰值

这些选项并不自动落到某个产品。它们用于检查设计缺少什么输入，以及下一步需要取得什么证据。只有当范围收窄到具体系统与版本，才能进一步讨论参数和实现。

条件	可使用的组织方式	还需回答
定位与修改	页、树、稀疏索引或其他索引	更新粒度、空间回收和写放大是什么
顺序输入与重放	追加日志、批次与快照	什么历史仍在，怎样建立恢复起点
损坏与兼容	长度、类型、版本与校验	能检测到什么，能从哪里取得恢复数据

页、日志和快照可以并用；访问模式约束选择，但不唯一推出一种文件格式。

图 8-8 定位、更新、恢复与兼容共同约束格式；同一系统可以组合多种文件组织方式。

8.6.2 把不完整的中间状态写出来

多部分 AOF 的清单切换，让“哪些文件共同有效”成为可见状态。类似问题也会出现在自己设计的导出文件、索引重建或备份归档中：新文件可能只写了一半，旧文件还在，新索引可能已经生成但尚未对应最新数据。

如果读取程序只根据目录里哪个文件名最大就选择最新文件，发布中的临时内容可能被误认成有效版本。若用一个清单发布组合，又必须保证清单引用的是已经准备好的文件，并规定发现缺失文件时的处理。清单的写入、持久化与解析也需要保护。

一个可检查的方案应列出生成前、生成中、发布后、清理后各阶段的有效入口。随后逐个假设进程在阶段之间停止：旧入口还能用吗，新入口会指向不完整数据吗，清理能否重试？这种推演经常比再加一个版本字段更早暴露问题。

这里的判断不能只停在文件系统操作是否原子。某个改名可以原子地改变名称关系，设备掉电后该目录项与文件内容是否都已持久化，是不同层面的条件。系统依赖的层及其提供的保证需要明确，才不会把未写出的数据用一个成功改名掩盖过去。

8.6.3 版本号后面要有读写矩阵

导读里的事故中，文件已经有版本号，仍然缺少新旧读写组合的约定。假设存在旧版 A 与新版 B，可以得到四个基础组合。

写入者	读取者	必须明确的行为
A	A	现有格式继续可读
A	B	B是否兼容旧记录，如何补缺省值
B	B	新字段、新类型与错误处理按新约定工作
B	A	A能否跳过扩展；不能时是否可识别拒绝，回退如何处理

允许旧程序跳过未知字段，需要字段本身可跳过：例如有明确长度，且未知字段不是理解后续记录必需的内容。如果新字段改变了旧字段含义，即使能跳过也可能读出错误业务语义。因此“未知字段忽略”不是普遍安全策略。

版本兼容还有操作顺序。先部署能够同时读旧新格式的程序，再逐渐启用新写格式，可以减少混合版本阶段的冲突；但这只是常见路径，是否适用仍取决于具体软件的升级规则。文件格式转换后，回退通常比二进制程序回退多一步，部署系统需要记录这个不可逆点。

我更愿意用一组旧文件样本和中断实验验收兼容能力，而不是用“支持长期演进”作结论。样本要包含旧字段、新字段、边界长度、截断文件和不支持类型。它们只能证明已覆盖的情况，但能把未来兼容的承诺拆成现在可以执行的检查。

8.6.4 冗余是否值得，取决于故障是否被覆盖

第 4 章已经说明持久化策略的代价，这里再向下检查数据形态。双写保存完整页，redo 保存恢复变化，校验保存检查信息；删掉其中一项，需要明确由什么替代原来的职责。若底层确实提供相应页粒度的原子写保证，某类完整页保护可以有不同实现，但已提交变化仍然要有恢复来源。

同样，Kafka 的一个副本、远程历史段和消费者自己的业务数据库，不是三份完全等价的备份。副本可能同步同一错误写入，远程历史可能受保留策略约束，业务数据库可能只有处理后的结果。把副本数相加无法直接得到业务恢复能力。

我会在每一份额外数据旁边写明它能恢复什么、从哪个状态开始恢复、还依赖什么。某份数据只减少恢复用时也有价值，无须把它夸成新的正确性保证。相反，若所有恢复都依赖同一份已经过期的历史，再多文件也不能补上缺失的变化。

8.7 小结

Redis 的 RDB 保存可整体重建的状态，AOF 用基础状态与后续变化缩短和组织恢复；InnoDB 用页、页内目录和行格式支持按键定位及局部修改，并把完整页保护与日志恢复分开；Kafka 用分段、批和稀疏索引服务有序读写，历史段还能参与清理与远程存储。三个系统都组合了不止一种结构。

格式选择要同时看操作与故障。定长页可以容纳变长记录，追加写不意味着后台永不重写，校验能发现损坏却不能单独修复，版本号能够标识格式却不自动保证兼容。文件更紧凑也不必然让在线请求更快，索引可以重建也不表示恢复没有成本。

这一章留下的具体检查是：目标操作触及多少数据，发布中断时哪组文件仍有效，旧程序会怎样处理新文件。文件保存的位置和状态还需要与复制历史对应。下一章从这里继续：两个节点各有一份文件，怎样知道它们能否从某个位置接着同步，而不是把不相容的历史接在一起。

第 9 章 数据同步：确认、历史与续传

本章导读

我处理过一次下单后查不到订单的告警。订单已在主节点写入，查询却落到尚未回放这次修改的从节点。写入成功和随后读到之间有一个窗口，用户不会因为系统采用了读写分离，就把这个窗口当成正常结果。后来我们增加了写后读的路由安排，但先要承认：复制在运行，不等于每个读取位置都已经具有相同状态。

第 7 章讨论谁负责数据、谁能成为新的主节点。本章进一步追问它们之间的内容：一条修改传到哪里了，副本确认了什么，断线后还能从哪里继续。一次换主之后，两个节点的 offset 即使相同，也可能属于不同历史；一份日志即使没有损坏，也可能已经删除了落后副本需要的部分。

Redis 用复制命令流同步数据集，MySQL 用 binlog 事件推进副本，Kafka 用分区日志复制记录。它们都需要保存历史和进度，但这些标识不能合成一个通用的“越大越新”数字。下面按传播、确认、断线和换主四个问题展开，版本范围仍为 Redis 7.x、MySQL 8.0.x 与 Kafka 3.9.0。

9.1 先把“已经同步”拆开

9.1.1 同步到了哪一步

一条修改到达副本，可能只是进入网络缓冲，也可能已经写入本地日志、刷盘、应用到数据结构，或成为允许对外读取的状态。这些时刻并不重合。一个 ACK 若没有说明对应哪一步，就不足以推导故障后的行为。

以数据库复制为例，副本可以先把事件保存到 relay log，之后再回放到表。网络接收很快、回放很慢时，数据已经在副本机器上，却还不能由普通查询看到。Kafka 副本追加到日志后，记录是否进入消费者可见范围还涉及复制与事务边界。Redis 的复制确认也不能自动视为副本完成了持久化。

这不是文字上的细分。主节点等待哪一个时刻，决定成功响应前必须付出的延迟；主节点故障后，哪一个副本可以接替，决定已经确认的内容是否仍在；读取路由则决定客户端能否观察到相应状态。三项安排需要配合，不能只调一个等待参数就把它们全部解决。

问题	需要确定的对象	常见误判
传播什么	命令效果、行事件或日志记录	输入相同就一定执行出相同结果
到达哪里	接收、写日志、刷盘、应用、对外可见	收到ACK就等于所有副本已提交
从哪里继续	历史身份、已处理位置、仍保留的数据	offset相同就属于相同历史
失败后怎么办	拒写、等待、降级、换主、重建	超时就等于服务器没有执行

问题	需要明确的内容	错误推导
传播内容	命令效果、行事件或日志记录	输入文本相同就一定效果相同
确认阶段	接收、写日志、刷盘、应用、可见	ACK等于所有节点完成全部工作
续传条件	历史身份、进度与仍可得的内容	offset相同就属于相同历史
故障处理	等待、拒绝、降级、换主或重建	超时就表示服务器没有执行

复制协议、容量与故障假设共同决定行为。

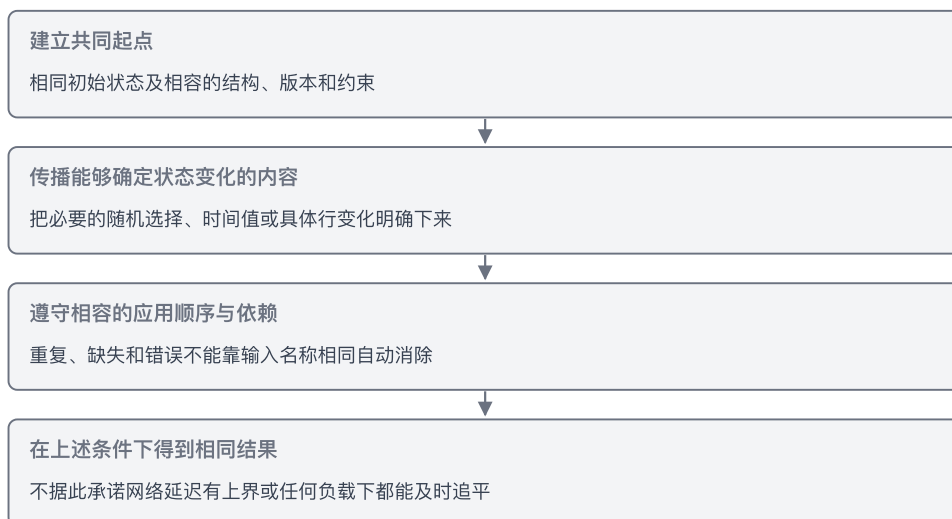
图 9-1 传播内容、确认阶段、续传条件和故障处理共同决定同步语义。

9.1.2 相同输入需要怎样的执行条件

状态机复制提供一种分析方式：从相同初始状态出发，按相同顺序执行具有确定效果的输入，副本才能得到相同结果。随机数、本地时间、外部服务返回值和不同初始内容，都可能破坏这个前提。

因此系统往往传播已经确定的效果，或在输入中明确必要信息。随机删除一个集合成员时，主节点可以把实际删除了谁表达出来，而不是要求副本再随机选一次；数据库可以记录受影响行的变化，而不是让每个副本重新执行含有不确定条件的查询。

即使采用效果记录，副本也仍需具有相容的结构和先前状态。目标行不存在、表定义不同、应用顺序被破坏，都可能使回放失败。效果复制减少了一类不确定性，没有消除数据维护、版本兼容和错误处理。



Redis、MySQL与Kafka不是彼此的数据副本；本图描述复制同一状态时的必要条件。

图 9-2 共同起点、相容顺序及确定的状态变化是副本对齐的条件，图中不表示网络延迟具有固定上限。

有界延迟还需要资源和故障假设。如果网络长期不可达，或副本处理速度持续低于新增写入速度，系统无法同时保证立即完成同步和不断接受写入。它只能增加缓冲、限制入口、降级，或停止一部分服务。同步协议提供路径，容量和准入决定这条路径能否跟上负载。

9.1.3 历史身份、进度和数据范围

假设节点 A 保存历史 H 中位置 100 之前的数据，节点 B 保存同一历史到位置 120。只要所需变化仍在，A 有机会获取差额。现在节点 C 也到位置 120，却来自另一条历史 J；相同数字不说明 B 与 C 内容相同。

历史身份标明“这是谁的哪段变化”，进度标明“已经走到哪里”。还需要第三项：发送方是否仍保留进度之后的内容。A 保存着位置 100，并不意味着 B 还存着 101 到 120。日志清理或缓冲覆盖可能使历史内容不再可得，位点元数据却仍然完好。

分开三项，就能解释不同恢复分支。身份相容、进度可衔接、内容仍在，才可能部分续传；身份分叉，需要识别共同前缀并处理冲突尾部；内容已过期，需要新的基础状态或其他恢复来源；元数据损坏，则连“应该从哪里开始”都可能要重新确定。

9.2 Redis：复制流怎样接续

9.2.1 从写入效果到复制流

Redis 主节点把引起数据集变化的命令效果通过复制连接传播给副本，副本应用这些变化。默认复制异步推进，普通写命令的成功响应不要求等待每个副本完成相应位置。主节点仍维护副本进度，用于监控和需要等待的操作。R9-01：Redis 复制机制

复制流与 AOF 的传播机制存在复用，但目的不同。AOF 服务于本机的持久化恢复，复制流服务于其他实例的数据集；两条路径的缓冲、传输和确认不会因此完全相同。一条命令已经进入复制流，不直接说明 AOF 已刷盘，反过来也一样。

Redis 7.x 的脚本采用效果复制，传播脚本执行产生的写入效果，而不是要求副本任意重新执行同一段脚本并自行取时间或随机值。命令传播也会根据操作语义改写。这样可以减少副本再次求值导致的差异。R9-02：Redis 脚本复制说明

普通异步复制减去了成功响应前的副本等待，但不意味着复制没有成本。命令编码、缓冲占用、网络发送、慢副本处理和全量同步仍会消耗主节点资源。一个慢副本可以间接造成内存或带宽压力，不能从“写命令不等 ACK”推出副本状态对主节点性能完全没有影响。

9.2.2 全量同步先建立共同起点

新副本没有可用的共同历史，或部分同步条件不满足时，需要获得一份完整数据集。Redis 可以生成 RDB 状态并将它传给副本，再补上生成与传输期间发生的变化。带磁盘中转和无盘传输的具体路径不同，但共同任务都是把一个快照时刻与随后复制流衔接起来。

这个过程可以拆成基础状态、增量保留和副本追赶三部分。基础状态提供某个时刻的数据集；增量保留确保快照之后主节点继续接受的修改没有遗漏；副本加载完成后继续应用这些修改，逐步追到正在变化的主节点。

如果只把 RDB 文件传完就宣布同步完成，会遗漏快照之后的写入。如果先向副本发送后续命令，却没有确保它在正确基础状态上执行，也可能得到错误结果。因此“传输文件”和“使副本进入可继续复制的状态”不是同一件事。

全量同步的成本随数据量、写入速率和副本加载能力变化。生成快照要使用 CPU 与 I/O，fork 与写时复制可能影响暂停和内存，传输占用带宽，副本加载期间的可服务状态也受配置与实现限制。写时复制不等于 fork 瞬间必然复制整个地址空间；后续被修改的页才会形成相应额外内容。

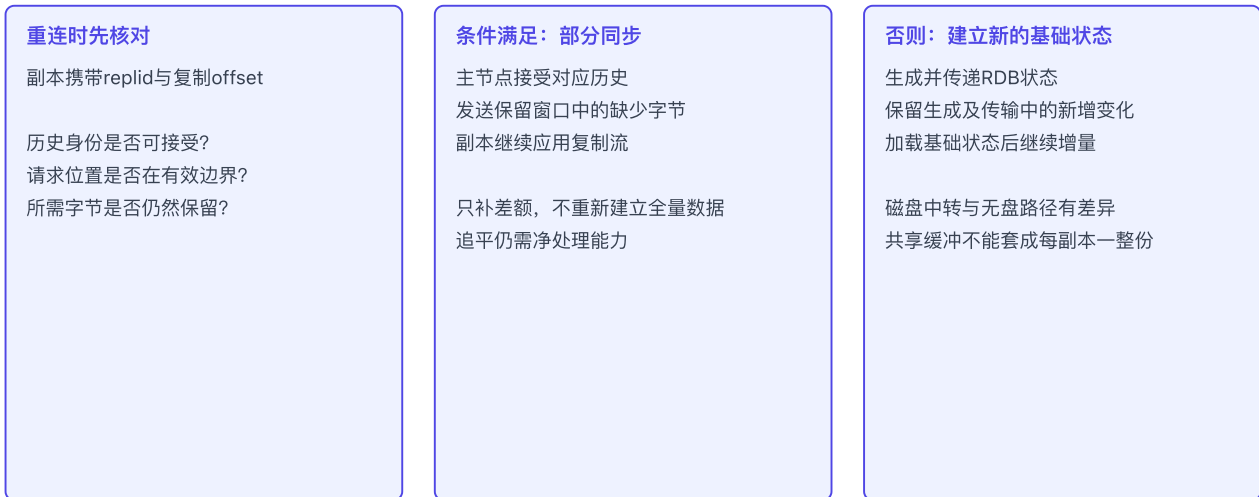


图 9-3 部分同步需同时满足历史相容与所需数据仍在；不满足时重新建立基础状态。

Redis 7.x 复制缓冲的实际组织包含共享复制缓冲等实现，不能把旧版本“每个副本独立保存一份全部增量”的内存模型原样套上去。概念上需要分别统计断线续传保留的数据，以及仍连接但发送较慢的副本需要的数据；它们在实现上可能共享底层块。R9-03：Redis 7.2 复制实现

9.2.3 replid 与 offset 共同标识历史

Redis 的复制 offset 以复制流字节为单位。比较同一历史中的 offset，可以知道两个实例在这条流上的进度差；它不是命令计数，也不是键数量。一个大值写入与多个很小的写入，可能产生相近字节量而有不同命令数。

replid 标识一段复制历史。副本请求部分同步时，发送自己所依据的历史身份和位置；主节点需要判断它是否属于可接受的历史，以及请求范围是否仍在保留窗口中。只判断 offset 是否小于当前值还不够，因为另一条历史也可能使用相同数字。

故障转移后，新主生成新复制身份，同时可以保留先前身份及其有效边界。其他副本若持有的是旧身份，在仍然相容的边界内可能继续部分同步。这不是无限期接受旧历史：边界之后可能已经出现分叉，不能把旧主后来独自接受的修改当成新主历史的组成部分。

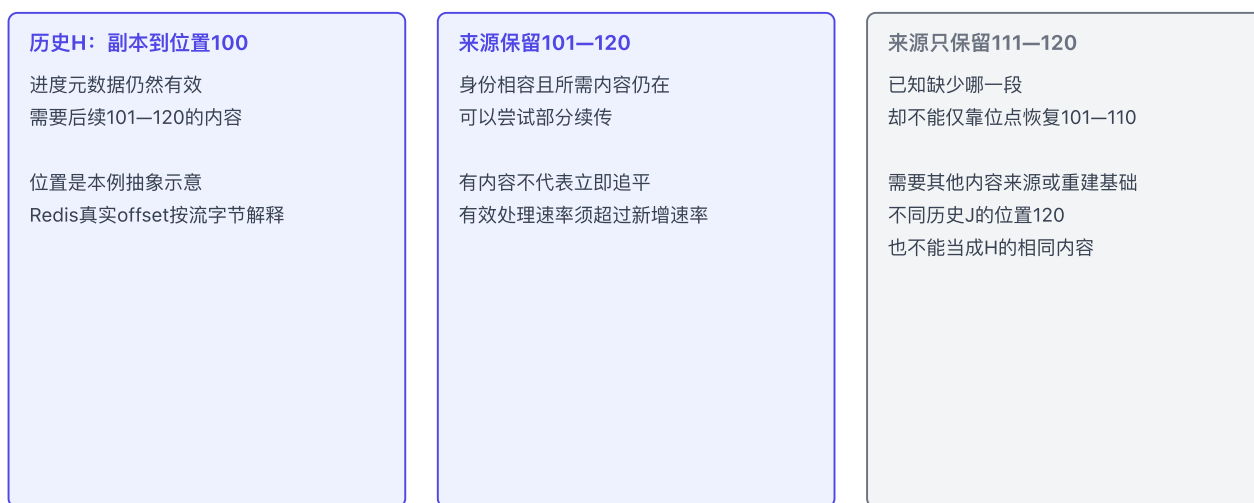
设旧历史 H 在位置 1000 前为各节点共有，新主从这里继续形成新历史 J。另一副本只到 H 的位置 900，只要新主仍保留相应内容，便可能从共同部分追赶。旧主若在隔离期间把 H 推到 1100，这额外一百字节不能仅凭数字较大就覆盖新历史 J。示例位置只用于说明身份与边界，不模拟真实协议逐字节握手。

这个设计降低了某些换主后的全量同步需求，但不会把分叉自动合并成业务上正确的状态。重新加入的旧主需要接受当前权威历史，它此前独有的写入可能不再保留在服务数据集中。应用若把这些写入当成已经完成的交易，还需要独立处理业务后果。

9.2.4 backlog 保存的是可重传内容

复制积压缓冲区 backlog 保留近期复制流，为断线后的部分同步提供内容。它在逻辑上是一个随新写入移动的窗口，旧数据会退出可重传范围。窗口大小决定能够保留多少字节，能够覆盖多少时间则取决于这段时间的复制流速率。

一个明确标为估算的例子：假定复制流平均每秒生成 2MiB，计划覆盖 60 秒断线，单就这段历史需要约 120MiB。若高峰速率升到每秒 8MiB，同样容量只能覆盖约 15 秒。实际配置还要考虑突发、复制流附加内容、实现分配和其他内存用途，不能把平均速率估算当成硬保证。



位点元数据与可重传数据是不同对象；保留窗口预算与追赶能力预算需要分别计算。

图 9-4 offset 元数据仍可存在，但它之后所需的字节可能已经退出 backlog；二者不是同一种丢失。

发生一次短断线后，能部分同步也不代表能瞬间追平。如果落后量为 B 字节，副本有效处理速率为 C ，主节点继续产生写入的速率为 W ，只有 C 大于 W ，才有净追赶能力。用 $B/(C-W)$ 可以得到稳定速率假设下的粗略时间估算； C 不大于 W 时，单纯增大 backlog 只能延后问题。

这里有两个不同的预算：保留窗口解决“数据还在不在”，净处理能力解决“追不追得上”。一个系统可能保留了很多历史，却因为副本磁盘或 CPU 持续饱和而越落越远。恢复分析如果只列一个缓冲参数，就漏掉了后一项。

9.2.5 准入门槛与等待确认

`min-replicas-to-write` 与 `min-replicas-max-lag` 为主节点写入设置条件：需要足够数量、最近有确认活动的副本，条件不足则拒绝写入。这里的 lag 门槛不是逐条证明“当前这条写入已经在 N 个副本上”。它根据副本状态作入口判断，随后的写入仍然要经历复制过程。

`WAIT N T` 则针对同一客户端连接此前的写入，等待指定数量副本确认相应复制位置，超时返回实际确认数量。应用必须检查返回值，不能只因为命令结束就当目标已经满足。`WAIT` 也不等于副本 AOF 刷盘，更不会自动把所有故障切换变成无损。R9-04: `WAIT` 的范围与限制

Redis 7.2 增加的 `WAITAOF` 可以等待本地及副本的 AOF 持久化条件，它与普通 `WAIT` 的等待对象不同，且依赖相应持久化配置。即使等待完成，仍需另外考虑后续选主、多个故障及应用事务边界，不能把一个命令名称直接翻译成系统级强一致。R9-05: `WAITAOF`

假设应用先写入，再等待副本确认，等待超时。此时原写入可能已经执行，并且某些副本稍后仍会收到它。把超时当作“写入未发生”，然后以新的业务请求重复扣减，就可能制造业务重复。等待用于满足确认条件，不会回滚先前已经执行的操作。

入口检查、复制等待、持久化等待可以组合，但各自增加的保证必须逐项表达。还有一项不能遗漏：故障后究竟允许哪个副本成为新主。如果被提升的节点没有相应历史，先前在另一节点取得的确认并不会自动转移成新主的数据。

9.3 MySQL：接收事件与回放事务

9.3.1 binlog 事件保存什么

MySQL 复制可以使用不同 binlog 格式。ROW 格式记录行变化及必要元信息，STATEMENT 格式记录语句，MIXED 在规定条件下选择。这里以 ROW 为主要观察对象，它可以避免由副本重新执行同一语句时出现的一部分非确定性问题。R9-06: binlog 格式

ROW 并不简单等于“每行一条独立事件”。事件可以包含多行，记录哪些列还受行映像等配置影响；表映射、事务边界和其他事件共同组成可解释的复制内容。比较日志大小时，要看被修改行数、字段宽度和行映像，不能只数 SQL 条数。

例如，一条批量 UPDATE 影响很多行，语句文本可能很短，而行变化表示较大。这份额外内容减少了副本重新计算影响集合的工作。它没有免除副本查找、修改索引、执行约束和持久化的代价，也不能保证一个已经被人工改坏的副本必然继续回放成功。

传输的是逻辑变化还带来结构约束。副本需要相容的表定义和可应用状态；升级或 DDL 的顺序不当，可能使后续事件无法解释。复制遇到错误时停止，并报告错误，通常比静默跳过错误更容易保住数据关系，但恢复还需要查明究竟缺了哪一段变化。

9.3.2 relay log 将接收与应用分开

传统复制中，副本接收线程从源端获取 binlog 事件并写入 relay log，应用线程或协调器与工作线程再执行这些事件。接收与应用分开，使网络传输可以先推进，暂时不必等待每一条事件都完成引擎操作。R9-07: 复制线程



接收、刷盘、应用和对外可见是不同阶段；半同步仍需合适的故障切换候选。

图 9-5 接收到日志不等于已应用到表；半同步确认与GTID分别增加确认阶段和事务身份信息。

relay log 形成一个有限的待处理区。它可以吸收短期速率差，却不能让慢应用永久追上快写入。磁盘空间、接收限制和应用能力仍然约束系统。接收位点已经接近源端，而应用位点明显落后，说明网络未必是主要瓶颈；应该继续检查锁等待、大事务、索引成本与工作线程调度。

并行回放能够利用事务间独立性，但不能随意改变存在依赖的修改次序。多个事务争用同一行，增加工作线程未必改善吞吐；一个特别大的事务也可能使局部进度长期不动。判断副本落后，既要整体指标，也要找出正在等待的事务与阶段。R9-08：多线程副本配置

接收与应用的这种分离，正是导读事件的机制来源。订单变化可能已经到达副本日志，但尚未应用到查询使用的表。把查询切回主节点可以避开这一特定副本延迟，但还需要事务隔离、连接路由和故障切换保证读到合适状态；“读主”不是所有一致性问题的通用解法。

9.3.3 GTID 让事务身份跨文件保留

传统复制位置使用 binlog 文件名和文件内位置。文件轮转或更换来源后，需要把物理位置与事务历史对应起来。GTID 给事务分配由来源标识与序号组成的身份，使副本能够表达自己已执行了哪些事务，而不只说明读到了某个文件字节。R9-09：GTID 格式与存储

GTID 的全局唯一性，不等于跨来源存在一个可以直接比较大小的总序。A:1-10 表示来源 A 的一段事务集合；A:1-8,B:1-3 则包含两个来源的范围。前者多了 A 的 9、10，后者多了 B 的 1、2、3，不能说其中一个简单“大于”另一个。

自动定位需要比较集合关系，找出副本缺少而来源能够提供的事务。如果某些必需事务已经从源端日志清理，知道它们的 GTID 仍然不等于能把内容发送出来。这里再次出现身份元数据与数据可获得范围的区别。

R9-10：GTID 自动定位

还要警惕额外事务。某个副本上独立产生了源端没有的修改，即使它能继续接收后续事务，数据也不因此自动重新一致。GTID 集合提供发现历史差异的线索，却不替业务决定冲突修改是否保留。错误地跳过事务或注入空事务可以改变进度表象，但不能证明表里的内容正确。

GTID 因而既带来便利，也增加约束。它使迁移与换源时可以按事务身份推理，同时要求维护人员知道哪些操作会产生本地独有历史。只盯着“复制线程在运行”，无法证明所有节点拥有相同事务集合和相同业务数据。

9.3.4 半同步确认的准确位置

MySQL 半同步复制在满足配置条件时等待副本 ACK。副本确认的是事务事件已经写入 relay log 并刷到磁盘，不要求相应事务已经在副本引擎中应用。这里比“副本收到网络字节”多了持久化条件，又比“副本查询已经能看到”少了应用阶段。R9-11：半同步复制

`AFTER_SYNC` 与 `AFTER_COMMIT` 改变源端等待的位置。前者在源端 binlog 同步后、引擎提交前等待相应确认；后者先完成引擎提交，再等待确认。后者的等待期间，其他会话可能已经看到事务结果，但副本尚未确认接收。在特定故障切换路径中，新主可能没有这笔此前已被读到的事务。R9-12：半同步等待点与可见性

故障切换后先前可见事务可能消失，与 SQL 隔离级别里的幻读是两类问题。幻读讨论一个事务中重复范围查询看见新的匹配行，两者的问题边界不同。术语分清，读者才不会误以为提高隔离级别就能修复复制确认窗口。

等待指定数量的副本，也不等于等待所有副本。系统按配置要求取得确认，故障转移方案必须选择具有适当历史的候选。副本已经收到日志但尚未应用时，恢复还要处理回放；确认不应该被误读成所有读取位置都即时可见。

半同步超时可以退回异步运行。这改变了后续写入的确认条件，不只是增加一个告警数字。观察半同步时，要区分功能被配置为启用、当前实际处于等待确认状态，以及曾经发生过多少次降级。配置存在不能证明每笔事务都取得了相同确认。

本书使用 `source/replica` 命名说明 MySQL 8.0 较新版本的相关变量；这些名称在 8.0.26 起替代对应的 `master/slave` 名称。复制拓扑的组织与组复制另见第 7 章，不把传统异步、半同步和组复制压成一个只由延迟决定的档位表。

9.4 Kafka：复制位置之外还有有效历史

9.4.1 Follower 获取记录，Leader 判断进度

Kafka 的 Follower 按分区从 Leader 获取日志记录，带着自身的获取位置请求后续内容。Leader 因而能够观察副本的追赶进度。拉取安排让副本按自身节奏继续获取，但不能据此简单认定拉取比推送更解耦；Leader 仍要处理每个副本的状态和请求，慢副本也影响复制集合。

一段分区日志有若干不同的位置。日志末端表示本地已经追加到哪里；高水位 HW 表示复制协议认可的提交边界；事务消费还要考虑未完成事务限制形成的稳定边界。通常以“下一个 offset”表达边界，因此可读取范围的端点要注意是否包含该位置，不能把图中的边界数与最后一条记录编号随意混用。

对于普通非事务消息，可消费范围受 HW 限制；对于 `read_committed` 消费，未完成事务还可能使可见范围停在更早位置。一个 Follower 物理文件中已经有某条记录，不代表当前消费者就一定能得到它。这是复制追加、提交和事务可见性三个阶段的差别。

单分区本地日志示例



边界按下一个offset表示：普通非事务读取受HW限制，`read_committed`还受未完成事务的稳定边界限制。

HW是复制协议边界，不是所有时刻的即时min(LEO)等式，也不表示每条记录已经fsync。

图 9-6 本地日志末端不等于已提交范围；事务读取还需要考虑稳定边界。offset 均在各自分区内解释。

复制保证依赖仍有符合条件的副本保存相应历史。Broker 本地追加也不等同每条记录已经单独 `fsync`；多副本复制与本地持久化使用不同机制。分析整机掉电、存储丢失和多个副本同时故障时，需要把这些假设一起列出，而不能只看 `acks`。

9.4.2 ISR 与 `acks` 必须连着解释

ISR 是当前满足同步条件的副本集合，不是“系统里所有副本”的别名。副本长时间不能跟上，会失去集合成员资格；恢复追赶后可以按协议重新加入。集合随运行状态变化，是复制确认与故障选择的重要依据。

`acks=all` 要求按当前同步集合完成相应确认条件，`min.insync.replicas` 规定接受这类写入所需的最小 ISR 数量。两者配合才表达“至少保有多少同步副本时继续写”的意图。只设 `acks=all` 而让 ISR 收缩到很小，实际确认覆盖也会随之变化。R9-13：生产者 `acks`、R9-14：最小 ISR 配置

以三副本分区、最小 ISR 为 2、生产者使用 `acks=all` 为设定。三者正常时，可以继续写；一个副本暂时落后并被移出后，剩余两个同步副本仍可能满足要求；如果同步集合进一步不足两个，对应写入将无法按该条件成功。这个例子说明门槛，并不覆盖所有请求到达与 ISR 变更的竞态，也不表示每次错误都证明日志未追加。

最小 ISR 是下限，不是“只等这两个就行”的固定副本名单。确认范围与当前 ISR 及协议状态有关。它也不等于多数派公式：给不同副本数配置相同门槛，含义会变。只有把副本数量、ISR 状态、确认方式和选主规则一起看，才知道当前系统在何种故障下选择继续或拒绝。

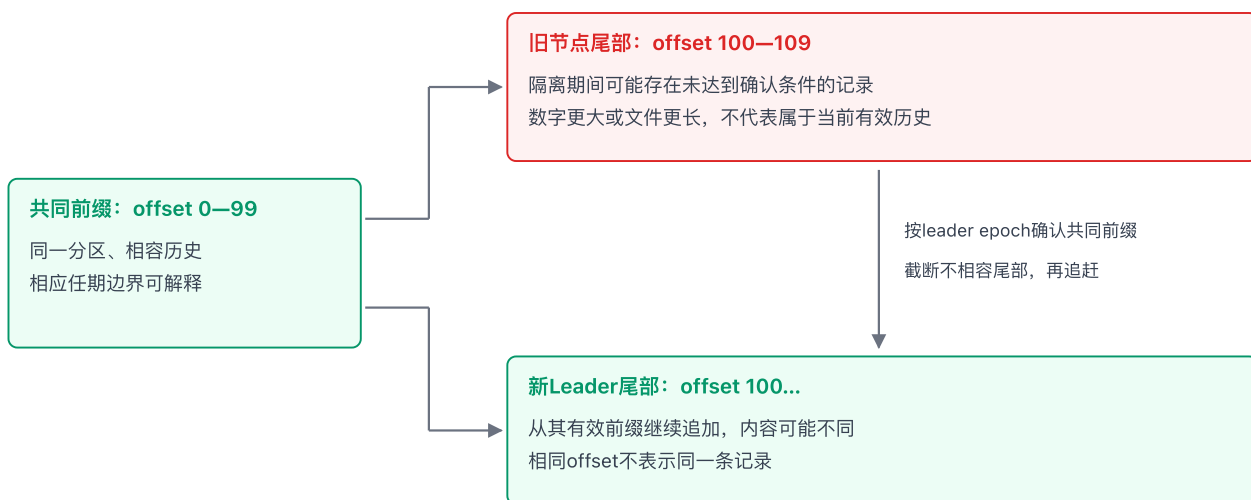
不干净选举允许不满足通常同步资格的副本接任时，可能以丢失部分历史换取恢复服务。这个选择属于故障策略，不能被一个生产者参数掩盖。第 7 章已经讨论候选资格；这里关心的是新 Leader 的有效日志到底包含哪些记录。

9.4.3 leader epoch 识别分叉，不靠长度猜测

假设旧 Leader 与一个 Follower 原先都有 offset 0 到 99。随后旧 Leader 又追加了 100 到 109，其中某些记录没有达到提交条件，便与集群失去联系。新的 Leader 从它拥有的有效前缀继续写入，也可能产生编号 100 之后的记录。旧节点回来后，两边日志长度相同也不代表内容相同。

leader epoch 为不同 Leader 任期的历史提供区分。副本通过相关协议查询某个 epoch 的结束位置，从而识别共同前缀与需要截断或补齐的部分。相比仅比较 HW，这使恢复能够利用更明确的历史边界，而不是把一个提交位置同时当成任期识别信息。R9-15: KIP-101 的恢复问题与方案

epoch 不是业务时间戳，也不能跨分区给所有消息排序。它服务于某个分区的 Leader 历史。一个分区的新 epoch 与另一个分区的旧 epoch，没有“哪个业务事件更晚”的直接比较意义。



图示假设共同前缀到99；实际截断边界由协议确定，不按文件长度或业务时间猜测。

恢复保证仍依赖此前确认、选主与故障条件；epoch不提供跨分区业务排序。

图 9-7 换主后先识别共同历史，再处理不相容尾部。相同 offset 或日志长度不足以证明内容一致。

截断也不等于随意删除所有较新的数据。协议需要区分哪些位置属于当前有效历史、哪些尾部没有相应保证。若发生了超出系统假设的故障，已确认数据也可能面临风险；恢复流程的存在不应被写成“任何切换都不丢”。分析要回到此前确认所依赖的副本和存储状态。

9.4.4 远程历史与本地续传的衔接

第 8 章介绍了 Kafka 3.9.0 的远程存储恢复路径。Follower 需要的 offset 早于 Leader 本地日志起点时，不能立刻以“本地没有”认定全部历史已经丢失；对应远程段与元数据可能仍在。

实现可以获取远程辅助状态，建立 leader epoch 信息与生产者快照，然后从对应后续位置进入本地日志获取流程。这里恢复的是继续工作的必要状态，不要求把历史远程 `.log` 全部变成 Follower 本地日志。R9-16：

3.9.0 TierStateMachine 与恢复条件

这条路径仍然受约束：远程存储必须启用并可访问，所需元数据和段须存在，状态恢复必须成功。远程与本地保留策略不合理、插件失败或权限变化，都可能阻止恢复。监控只看本地磁盘空间和复制延迟，已经不足以覆盖这组依赖。

“Follower 获取由本地日志服务”这一运维经验受实现版本限制，不能直接作为永久规则。更稳妥的概括是：续传必须有可解释的历史及所需状态来源；来源究竟在本地还是远程，由具体版本实现决定。

9.4.5 消费进度不能由消息内容推导

Broker 保存消息，消费者读取消息并产生业务结果。消费者的当前读取位置、已提交 offset 和外部业务系统的处理结果，是三种状态。一次 `poll` 返回消息，只说明客户端已经取到；提交 offset，只说明下次从哪里继续的进度被保存；外部数据库是否提交，Kafka 不能单靠消息内容判断。

设消息表示“为订单发送通知”。日志中能看见这条消息，无法由此推断通知是否已经发送，发送成功但响应丢失时也一样。删除消费者进度后重新读日志，会再次遇到同一条消息；是否能安全重做，取决于通知服务的业务幂等与结果记录。R9-17：KafkaConsumer 的位置与提交说明

因此，消费者 offset 不能被概括为“丢了再从日志算出来”。某些纯派生计算可以从规定起点重新计算，但需要输入历史仍然完整、计算规则可复现，并正确重建输出状态。已经产生不可撤回副作用的业务，通常还要单独保存处理标识和结果。

Kafka 事务可以把特定 Kafka 读写流程中的输出记录与消费 offset 纳入协议协调，但不会自动包含任意外部数据库事务或短信发送。把产品中某个范围的 exactly-once 称谓搬到端到端业务，往往遗漏最关键的外部边界。第 10 章会用一个业务事件处理流程说明怎样保留这条边界。

9.5 用相同故障比较不同机制

9.5.1 同样是确认，条件不同

表 9-1 确认对象与尚未覆盖的工作

机制	当前等待或检查的对象	不应据此推出
Redis 副本准入门槛	最近满足条件的副本状态	当前写入已经复制到这些副本
Redis WAIT	本连接此前写入对应的复制确认	所有副本落盘、任意换主都无损
Redis WAITAOF（7.2起）	规定的本地/副本AOF持久化条件	自动获得完整业务事务和故障选主保证
MySQL 半同步	所需副本的relay log接收并刷盘确认	副本表已经应用，超时后仍按原保证运行
Kafka acks=all配合最小ISR	复制协议及当前ISR条件	每条独立刷盘、消费者外部操作已完成

这些机制无法用一个“弱、中、强”顺序完整表达。某个机制等待副本日志落盘，另一个依赖多个副本的复制状态；某个机制超时后继续，另一个返回写入错误。比较时应先给出相同故障和相同业务要求，再判断哪个安排满足它。

9.5.2 网络断开后的三个问题

断线恢复可依次检查历史身份、可用内容和追赶能力。这个顺序避免一上来就调大超时，或直接重建副本。

观察结果	主要问题	下一步证据
身份相容、所需历史仍在	可能继续增量	获取与应用速率是否足以净追赶
身份相容、历史内容已清理	已知进度无法取回对应变化	新基础状态、备份或远程历史是否可用
同一数字位置对应不同任期或来源	历史可能分叉	共同前缀与不相容尾部的边界
进度元数据本身缺失或损坏	起点不明确	可验证的检查点、事务集合与恢复工具
网络已恢复，落后仍持续增加	容量或执行瓶颈	接收、写日志、应用各阶段耗时

Redis backlog 被覆盖、MySQL 所需 binlog 被清理、Kafka 远程元数据不可用，在实现上不同；共同点是恢复需要的依据不再完整。把它们全部叫作“位点丢了”，会使排查方向错开：有时位置非常清楚，只是内容不在了。

9.5.3 主节点故障后的确认写

考察一条客户端已经收到成功的写入。先找出成功响应发生时系统承诺了什么，再判断新主是否包含这项承诺所依赖的历史。异步复制中，成功可能早于副本接收；带等待的复制中，还要检查等待是否超时降级，以及候选副本是否属于取得确认的范围。

如果客户端没有收到成功，问题反而更不确定：请求可能尚未到达，也可能已经提交只是响应丢失。服务端复制日志能够帮助恢复状态，却不能替客户端区分这两种情况。业务请求标识、结果查询与幂等处理，是端到端协议的一部分。

这里没有哪一个产品名称可以直接替代推理。MySQL 的本地事务保证不等于异步副本无损，Redis 的低延迟不等于所有写都必须容忍丢失，Kafka 的复制确认不等于下游数据库结果已经持久化。真正的比较对象是带有配置、拓扑和故障假设的一段写入路径。

9.6 架构启示：把恢复要求写进运行方式

9.6.1 拒写与降级是两种业务选择

我在涉及金额或库存的写路径上，更在意系统是否悄悄降低了确认条件。MySQL 半同步超时可以退回异步；Kafka 在 `acks=all` 且最小 ISR 条件不满足时，会使相应写入失败。前者保留服务能力的方式与后者不同，不能只比较正常情况下谁的延迟低。

这个偏好并不是建议所有系统故障时停机。日志采集可能更愿意继续接收并记录损失风险；交易状态可能宁愿暂时拒绝新的确认。业务要决定允许丢哪类记录、允许等待多久以及拒绝后如何回应用户。技术方案则需要使这些选择真的成为运行行为。

降级如果被选择，必须有可观察的开始和结束，也要知道降级期间产生了哪些业务结果。只在仪表盘显示一个红点，而入口仍向用户给出同样承诺，不能算完成了风险处理。恢复正常也不意味着降级期间的历史差异已经自动消失。

9.6.2 写后读需要一个可检查的目标

导读中的订单查询问题，可以有不同处理方式。写后短时间路由到主节点是一种办法，但时间窗口只是经验值；等待副本应用到某个事务位置，则把条件写得更明确，却增加等待和超时分支。返回写入结果供页面直接显示，可以减少即时查询，但不替代后续状态读取。

我还排查过一次外观相似的问题：用户修改资料后，数据库已经写入，刷新页面仍看到旧值。查到最后，是不同代码路径里删缓存与写数据库的先后顺序不一致，缓存没有按预期失效。那次问题与订单读取落后副本不同：等待数据库副本追平，不能修复另一条缓存失效路径。

选择依赖操作语义。用户刚保存资料时，希望立即看到刚才的结果；排行榜允许几秒延迟；后台离线统计可以接受更长时间。把所有读取都叫最终一致，既不说明窗口，也不说明超时后行为。设计应落到某个读操作所需的最低状态，而不是全系统一句统一口号。

如果用固定等待时间保证可见性，必须承认它不是进度证据。平时几十毫秒能追上，不代表大事务、暂停或故障期间也能追上。需要严格写后读的操作，要么获得适当的读取位置，要么明确等待到哪个可验证条件；条件不能满足时应返回可理解的状态。

9.6.3 恢复预算由历史保留与净追赶能力共同决定

我查看副本恢复方案时，会把历史保留和追赶速度放在一起。保留一天日志却需要两天才能重建的实例，恢复过程中仍可能越过保留边界；把网络恢复了当成故障结束，也可能使长期落后的副本继续占用系统资源。

一次完整恢复应统计基础状态生成、传输、加载、增量追赶和对外就绪。每段都可能成为最长阶段，重试还会放大资源消耗。特别是在多个副本同时重建时，源节点要一边处理正常请求，一边提供恢复数据，不能用空载传输速度估算整体完成时间。

有些恢复可以通过限流降低前台影响，但恢复变慢又要求更长历史保留。这是明确的循环约束：恢复速率影响窗口，窗口影响容量，容量又影响保留和维护。把这组关系写清后，才能决定是增加资源、减少入口，还是改变恢复来源。

9.6.4 进度保存要与副作用对应

如果消费者把 offset 提交在业务修改之前，崩溃后可能跳过尚未完成的工作；如果在业务修改之后提交，响应丢失或崩溃可能导致同一记录重做。两种顺序都没有解决跨系统原子性，只是选择了不同失败窗口。

对可安全重做的派生计算，重复通常可以通过覆盖写或带版本更新处理。对扣款、发货等操作，需要业务标识、唯一约束、结果状态，或由目标系统提供适当的幂等接口。保存消费进度与保存业务结果，至少要有一条明确的衔接规则。

我不会因为一条消息已经落在 Kafka，就认为后续状态都能从它“重算”。有些状态表达的正是消息之外的事实：第三方是否执行、用户是否确认、重复请求对应哪一次结果。重放输入不能替代这些事实的记录。

9.6.5 用故障轨迹验收，而不是只看连通状态

同步链路全绿只能说明当前监控条件满足，不证明每个故障窗口都符合业务要求。有效的检查应选择明确时点：请求已执行但响应未送达，副本已接收但尚未应用，主节点确认后立即失联，副本断线超过本地历史窗口，以及恢复期间再次中断。

每个检查都需要记录输入标识、已确认结果、节点历史和恢复后可见内容。否则只能得到“集群又起来了”，无法知道哪几笔操作保留下来。实验环境可以缩小规模，但不能把故障注入影响范围与生产安全混为一谈；这些检查应在隔离环境执行并记录所用版本和配置。

验证还要检查错误路径本身。请求返回错误之后是否继续重试、是否换了新的业务ID、是否可能重复执行，往往比正常路径多一个副本更影响最终结果。第 10 章把这项检查放进一个完整的库存预留设计中。

9.7 小结

Redis 的 replid 与 offset、MySQL 的 GTID 集合、Kafka 的 leader epoch 与分区 offset，分别为历史与进度提供证据。它们不具有相同的比较规则。部分续传还需要对应内容仍然存在，以及副本具有净追赶能力；保留一个位置数字不会让被清理的数据重新出现。

确认也有不同阶段。Redis 的入口门槛、WAIT 与 7.2 起的 WAITAOF，MySQL 的 relay log 刷盘确认与等待点，Kafka 的 ISR 与提交边界，都要按自身条件解释。默认异步、超时降级、候选选择、本地持久化和外部副作用，不能被一个“强一致”标签合并。

同步协议帮助副本维持和恢复有效历史，业务仍需决定成功响应意味着什么、读操作需要看到哪里、超时后如何查询和重试。下一章将这些问题与数据结构、生命周期、持久化和分层放到同一个设计里，检查多个局部合理的选择，组合之后是否仍然成立。

第 10 章 把观察用于一次完整设计

本章导读

我曾在活动库存项目里把 Redis Cluster 作为主要存储。网络分区后，旧主短暂接受的部分写入没有保留在新主的数据中，最终出现了超卖。后来改成以 MySQL 记录库存结果，Redis 放在前面承担预处理。这段经历让我重新区分了几件原先混在一起的事：单次操作快、节点能切换、已经向用户确认的结果在切换后仍然存在，并不是同一种保证。

这段经历不是一份完整设计说明。仅有“MySQL 加 Redis”几个字，无法知道超时怎样处理、预扣后失败怎样回补、重复请求如何识别。本章不把缺少记录的历史细节补写成当年的事实，而是从这个问题出发，构造一个条件明确的库存预留示例，逐步给出可以检查的方案。

以下业务数字、团队条件和实现流程都是本章设定，用于展示推理，不是作者项目的测量结果。示例不会证明一种架构适合所有交易业务；它要证明的是，在这组条件下，哪些决定有根据，哪些仍要经过实验才能作出。

10.1 先确定成功意味着什么

10.1.1 本章的业务与资源设定

业务是活动商品库存预留。一次请求只预留一种 SKU 的正整数件商品，预留成功后可以确认成交，也可以取消释放。本章不包含付款、跨商家购物车、多 SKU 原子预留和退货补货；这些不是被忽略的异常，而是明确不属于这个事务范围的业务。

活动开始前导入库存，活动期间每个 SKU 的总量固定。预留占用可用库存，确认把已预留数量转成已售，取消把已预留数量还给可用库存。取消与确认互斥，一旦进入任一终态，不允许直接走另一条分支。新的购买意图使用新的请求标识，同一次请求的网络重试沿用原标识。

为使性能问题有具体输入，设定一个小团队维护服务，已有 MySQL 8.0 运维能力，已有 Kafka 用于业务事件，Redis 用于商品展示缓存。峰值预留入口为每秒 2000 次，其中最热 SKU 可能占 40%；单次请求数量上限为 10；正常状态下希望 99% 请求在 200ms 内返回明确结果或可查询的处理中状态。以上均为设计目标与假设，不是任何软件的能力上限。

库存正确性优先于故障期间继续接收新预留。无法确定权威存储可用时，可以拒绝新的预留请求；已经确认的预留不能因为客户端重试而再次扣减。展示库存以正常运行时数秒内更新为目标；展示记录到期且无法从权威库取得有效结果时，返回库存状态未知。最终预留结果必须由权威事务决定。允许展示有货而下单失败，不允许展示缓存单独产生预留成功。

资源与故障范围也要写明。本章先设计一个受控的单写入端数据库方案，允许在数据库主节点故障时暂停预留，恢复或验证切换后再开放；不承诺一个机房和全部数据副本同时丢失时仍然无损。数据库的本地持久化、备份及候选切换需要满足明确运行条件，不能由应用中的一条 SQL 代替。

10.1.2 三个不变量

对一个 SKU，定义 `total`、`available`、`reserved`、`sold`。在本章不含补货与退货的范围内，始终要求：

```
total = available + reserved + sold
available >= 0, reserved >= 0, sold >= 0
```

这是库存数量关系。还需要请求关系：一个请求标识最多创建一份预留结果，相同标识不能被不同用户或不同 SKU、数量复用；预留从已预留状态最多转为已确认或已取消之一。最后是响应关系：服务只能在权威事务提交得到确认后返回预留成功，提交结果未知时不能猜测失败，也不能再用新标识重试。

三个不变量对应不同机制。数量关系依赖对库存行的受控更新；请求关系依赖唯一键和状态转移；响应关系依赖客户端与服务端协议。给数据库设置“双 1”只涉及部分持久化条件，不能自动建立后两项。

10.1.3 前九章在这里分别提供什么

表 10-1 主题知识进入本案例的位置

前文主题	在案例中改变的决定
第1章的约束分析	把暂停写入的可接受性与不重复扣减放在产品选择前
第2章的数据结构与协议	请求标识、主键查找、批量事件与数量类型
第3章的生命周期	就绪前停止接单，退出前处理在途事务与结果未知
第4章的内存与磁盘	库存事务持久化，Redis缓存可丢失，恢复来源明确
第5章的分层	API输入、库存事务、事件发布和展示缓存分开承担职责
第6章的访问边界	用户只能操作自己的预留，发布者与服务账号分权
第7章的集群	单写入口、故障时停止旧主写入、候选历史验证
第8章的存储格式	事件字段与版本，备份恢复，派生状态重建条件
第9章的数据同步	写后读位置、超时重试、事件重复与消费者结果

主题	进入本案例的决定
第1—3章	明确成功与故障条件；请求身份与数量；就绪、在途请求与退出
第4—6章	提交持久化与缓存重建；库存事务和发布职责；用户与服务账号权限
第7—9章	单写入口与候选历史；事件与恢复格式；续传、幂等和消费者结果

这里只作主题索引；完整约束与九项映射见正文表10-1。

图 10-1 每个主题对应一个可检查的设计问题；这张图是主题索引，不是产品评分。

10.2 从候选中选择一个可以验证的起点

10.2.1 为什么先考虑数据库直接事务

本案例最核心的修改是：在库存足够时，原子地减少可用量，同时记录请求结果。库存行与请求表可以放在同一个 InnoDB 数据库，用一个事务提交。与跨系统预扣相比，这减少了需要补偿的中间状态，也使相同请求的结果有明确查询来源。

这不说明数据库一定能满足入口峰值。最热 SKU 的行更新仍可能串行竞争，2000 次入口中若有大量请求针对同一行，平均全库吞吐没有决定意义。数据库直接事务只是复杂度较低、正确性容易表达的初始候选，容量要用真实记录宽度、索引、事务长度和热点分布验证。

第二个候选是 Redis 先扣，MySQL 再确认。它可能过滤部分请求，却新增两个系统之间的状态：Redis 已扣但数据库失败、数据库已成功但客户端只看到前置结果、Redis 重建时如何恢复未完成占用。若未经测量就采用，等于先承担这些复杂度，却不知道省下的数据库压力是否值得。

第三个候选是先把请求写入 Kafka，由每个 SKU 的顺序处理逻辑决定结果。它可以让入口接受与最终处理分离，但接口就必须表达排队和稍后查询，不能把消息入队当成库存预留成功。本例允许返回可查询的处理中状态，因此这一候选在接口上可行。暂不采用它的理由是：在直接事务尚未被证明不足之前，无需先承担排队后的状态保存、消费者切换与重复消息处理。异步受理成为常规路径以后，还需要单独约定从受理到最终结果的等待目标。

候选	本例收益	新增负担	本轮决定
MySQL直接事务	数量与请求结果在一个事务中	热点锁、数据库故障时暂停	作为基线实现并测试
Redis预扣加MySQL确认	可能减少部分数据库入口	两处占用、回补、重建与补偿	暂不采用，保留为后续候选

候选	本例收益	新增负担	本轮决定
Kafka排队后处理	控制处理速率，按键组织任务	排队状态、延迟、消费者恢复	接口允许；先验证直接事务，再比较是否值得引入



候选淘汰理由随业务条件变化；本章方案不声称适合所有交易系统。

图 10-2 先建立直接事务基线；出现被测量证实的瓶颈后，再比较改变入口与状态管理的方案。

10.2.2 “先不做”也需要边界

初版不做跨 SKU 原子预留，不意味着以后简单循环执行几次单 SKU 事务即可支持购物车。若要求全成或全不成，就要扩展事务边界，或设计跨分区协调与补偿。这个新要求足以改变当前数据放置方案。

初版不把 Redis 作为准入依据，也不是否定缓存价值。商品标题、图片地址、近似库存展示可以由缓存承接，但库存成功必须回到数据库事务。在这种安排中，Redis 全部丢失主要影响展示与数据库读负载，不改变已经记录的预留结果。

初版不自动提升任意数据库副本。运维可以选择经过验证的高可用方案，但应用在重新开放写入口前必须确认旧写端已被隔离，新写端包含所需历史。若团队暂时做不到，就选择故障期间暂停，不能让“自动恢复可用”成为默认覆盖“不丢确认结果”的理由。

10.3 把请求身份和库存放进一个事务

10.3.1 数据表与约束

下列 SQL 是本章示例的核心表结构，使用 MySQL 8.0.16 及之后的 InnoDB；这一小版本起 CHECK 约束才具有这里需要的执行语义。示例使用整数件数和大小写敏感的 ASCII 标识，省略商品描述等非核心字段。

R10-01: CHECK 约束

```

CREATE TABLE inventory (
    sku_id BIGINT PRIMARY KEY,
    total BIGINT NOT NULL,
    available BIGINT NOT NULL,
    reserved BIGINT NOT NULL DEFAULT 0,
    sold BIGINT NOT NULL DEFAULT 0,
    CHECK (total >= 0 AND available >= 0
        AND reserved >= 0 AND sold >= 0),
    CHECK (total = available + reserved + sold)
) ENGINE=InnoDB;

CREATE TABLE reservation (
    request_id VARCHAR(64) CHARACTER SET ascii COLLATE ascii_bin PRIMARY KEY,
    user_id BIGINT NOT NULL,
    sku_id BIGINT NOT NULL,
    quantity INT NOT NULL,
    state VARCHAR(16) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
    state_version INT NOT NULL,
    created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    updated_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)
        ON UPDATE CURRENT_TIMESTAMP(6),
    CHECK (quantity BETWEEN 1 AND 10),
    CHECK (state IN ('PENDING', 'REJECTED', 'RESERVED',
        'CONFIRMED', 'CANCELED')),
    CHECK (state_version >= 0)
) ENGINE=InnoDB;

CREATE TABLE outbox (
    event_id VARCHAR(80) CHARACTER SET ascii COLLATE ascii_bin PRIMARY KEY,
    request_id VARCHAR(64) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
    state_version INT NOT NULL,
    event_type VARCHAR(32) NOT NULL,
    payload JSON NOT NULL,
    sent_at TIMESTAMP(6) NULL,
    created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    UNIQUE KEY request_version (request_id, state_version),
    KEY pending_events (sent_at, created_at)
) ENGINE=InnoDB;

```

`request_id` 是业务重试的身份，由客户端在一次意图形成时生成，服务端限制格式和长度。认证用户、SKU 与数量共同构成请求内容，同一标识再次出现时必须比较这些字段。仅凭标识相同就返回结果，会让错误复用标识的请求得到另一笔操作的回应；仅凭标识难猜也不能省掉用户权限检查。

PENDING 只作为创建事务内部的临时状态，不允许它独立提交。事务结束时必须将请求改成 **RESERVED** 或 **REJECTED**；事务失败则整笔回滚，不留下这条临时记录。这个规则不能只靠上述 **CHECK** 自动实现，库存服务的事务流程与测试必须保证它。

outbox 是与业务结果同事务保存的待发布事件，不是另一个库存正本。它解决数据库提交与发送消息不能自然组成原子操作的接口问题。即使 Kafka 暂时不可用，预留结果与需要发布的事件仍可以先在数据库一起提交，后续发布者再传递。R10-02：事务性 outbox

表没有给 `sku_id` 添加示例外键，这不意味着可以接受任意未知SKU。入口验证商品存在，事务中的库存 UPDATE 还要检查影响行数；不存在与库存不足都不能产生预留成功。若要给客户端区分两类失败，应在受控读取下确定原因，而不是改变原子扣减条件。

10.3.2 一次新请求的成功路径

应用使用同一数据库连接开始事务，先插入请求，再执行条件更新。下列参数以命名占位符表示，由驱动绑定；这些片段描述应用控制流，不是可以直接整段粘贴到 mysql 客户端执行的存储过程。

```
START TRANSACTION;

INSERT INTO reservation
    (request_id, user_id, sku_id, quantity, state, state_version)
VALUES
    (:request_id, :user_id, :sku_id, :quantity, 'PENDING', 0);

UPDATE inventory
SET available = available - :quantity,
    reserved = reserved + :quantity
WHERE sku_id = :sku_id AND available >= :quantity;
```

数量在进入事务前被验证为 1 到 10 的整数。UPDATE 把条件检查和修改放在同一条语句中，InnoDB 对匹配的更新记录进行相应加锁；并发请求不能各自拿到旧可用量，再分别无条件写回。R10-03：UPDATE 与锁

若影响一行，应用把请求改为已预留，并在同一事务写入版本 1 的事件，再提交：

```
UPDATE reservation
SET state = 'RESERVED', state_version = 1
WHERE request_id = :request_id;

INSERT INTO outbox
    (event_id, request_id, state_version, event_type, payload)
VALUES
```

```
(CONCAT(:request_id, ':1'), :request_id, 1, 'ReservationReserved',
JSON_OBJECT('schema_version', 1,
            'request_id', :request_id,
            'user_id', :user_id,
            'sku_id', :sku_id,
            'quantity', :quantity,
            'state', 'RESERVED',
            'state_version', 1));

COMMIT;
```

只有 COMMIT 得到成功结果后，服务才返回预留成功。事务内的库存变化、请求结果与事件要么一起提交，要么在回滚后都不生效。这个事务不包含 Kafka 消息发送或远程服务调用；把这些操作放进事务会延长持锁时间，也不会使跨系统提交自动具有原子性。R10-04：事务提交与回滚

如果库存 UPDATE 影响零行，应用将请求改为 **REJECTED**、版本设为 1，然后提交拒绝结果；库存没有变化，本例不发布库存状态事件。同一个请求标识以后仍然得到这次拒绝，不能因为后来库存释放就突然成为成功请求。用户重新发起购买意图，才使用新标识。

若任何 SQL 出现错误、约束失败或连接中断，应用不能继续执行后续成功分支。明确回滚或关闭不可用连接，并进入结果查询与重试流程。死锁可能使数据库选择一个事务回滚；锁等待超时也需要应用按规定结束事务，不能以为每种错误都自动回滚了所有先前语句。

10.3.3 同一请求同时到达两次

两个服务实例可能同时收到同一个 **request_id**。唯一主键决定只能存在一份记录，但重复插入的处理不能随意用 **INSERT IGNORE** 忽略后继续扣库存。忽略错误后进入更新，会把重复请求当成新请求执行。

本例把插入成功作为新请求流程的入口。重复键时，当前尝试结束并回滚，随后在权威写入端的新事务中查询已有结果，比较用户、SKU和数量；内容相同则返回已记录状态，内容不同则返回标识冲突。若另一事务回滚而记录不存在，可以用同一请求标识重新走插入流程。

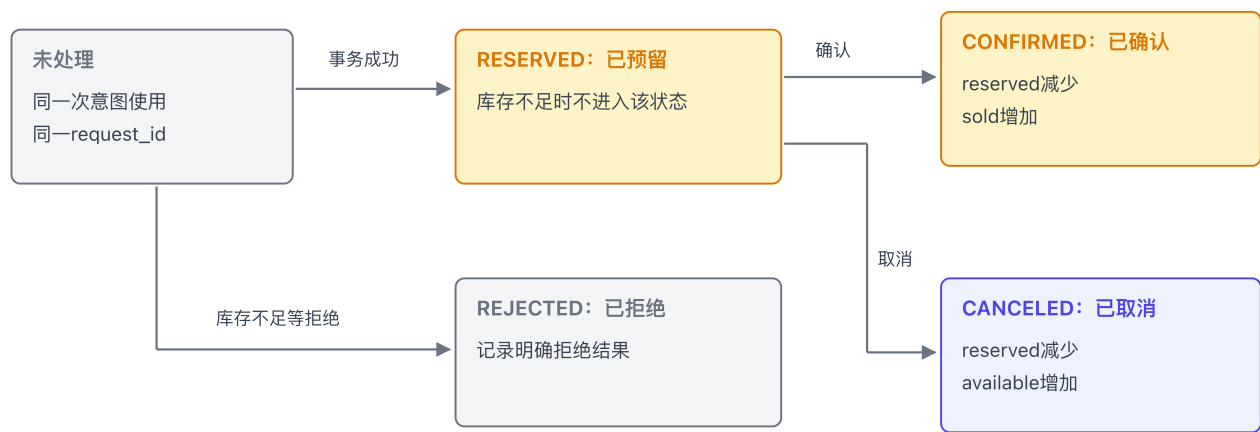
并发唯一键竞争还可能等待或产生死锁。应用需要有界重试，保留相同业务标识，并对数据库错误类别作区分。重试次数耗尽时返回可查询的未知结果，而不是替用户生成新标识。MySQL 的锁与死锁处理提供数据库层行为，业务协议规定接下来怎样对外回应。R10-10：死锁处理

这个安排把幂等建立在持久化结果上。服务实例的本地内存去重、短期 Redis 键或客户端按钮置灰，都可以减少重复入口，但它们丢失后不应改变库存事务的正确性。请求结果保留期限也因此属于业务协议；删除太早，会让很晚到达的重试再次变成“新请求”。

10.4 确认与取消怎样互斥

10.4.1 服务端状态与客户端未知

服务端的预留状态是 `RESERVED`、`CONFIRMED`、`CANCELED` 或 `REJECTED`。客户端超时是一种观察结果，不是新的库存状态。一个超时请求在服务端可能已经成功，也可能不存在；因此查询接口是写接口的组成部分，而非可有可无的运维功能。



超时是客户端对结果未知，不是新的服务器失败状态。已确认与已取消之间没有直接转移。
PENDING仅在创建事务内部使用，不独立提交；同一请求重试应返回持久化的既有结果。

图 10-3 已预留可转为已确认或已取消；超时表示客户端尚不知道结果，不触发自动重复扣减。

本例查询接口从权威写入端读取，并校验用户身份。它返回请求参数、当前状态与版本；客户端看到 `RESERVED` 才能继续确认，看到 `REJECTED` 则结束该请求。不存在的记录在有在途事务时不能证明未来不会提交，安全后续动作仍是携带同一标识重试原请求，由唯一键协调。

`CONFIRMED` 与 `CANCELED` 是本例终态。若用户已取消后又要买，需发起新预留；若已确认后需要退货，属于本章范围之外的新业务状态，不能把已确认直接改回可用。这样可以让每一份数量变化有对应的业务原因。

10.4.2 锁住一份预留，再转移数量

确认事务先按主键锁定预留记录。锁定读取在事务内生效，读取到当前可用于修改的状态，而不是依赖先前非锁定查询的旧快照。R10-05：InnoDB 锁定读取

```
START TRANSACTION;
SELECT user_id, sku_id, quantity, state, state_version
FROM reservation
```

```
WHERE request_id = :request_id
FOR UPDATE;
```

应用验证记录归属和状态。已确认则提交空操作并返回当前结果；已取消或已拒绝则返回不允许确认；只有已预留进入数量转移。SKU和数量来自锁定后的记录，不接受客户端再次提交一个不同数量。

记录不存在、身份不符或状态不允许时，应用先回滚本次未修改数据的事务，再返回相应结果；不能把仍处于事务中、可能持有锁的连接直接交回连接池。若连接已经不可用，则丢弃连接并按结果未知规则处理。取消接口也遵守同样的事务结束要求。

```
UPDATE inventory
SET reserved = reserved - :stored_quantity,
    sold = sold + :stored_quantity
WHERE sku_id = :stored_sku_id
    AND reserved >= :stored_quantity;

UPDATE reservation
SET state = 'CONFIRMED', state_version = state_version + 1
WHERE request_id = :request_id;
```

应用必须检查库存影响一行，否则回滚并报告不变量异常，不得只更新请求状态。随后写入版本 2 的 `ReservationConfirmed` 事件，携带完整请求身份、数量、终态与版本，和前述操作一起提交。事件ID为请求标识加 `:2`，唯一键防止同一状态版本被重复创建。

取消使用相同的加锁顺序：先请求记录，再库存行。只在 `RESERVED` 状态下把 `reserved` 减少、`available` 增加，状态改成 `CANCELED`，版本加一，并写入版本 2 的取消事件。若两条路径并发，先取得并提交该请求状态锁的一方确定终态，另一方随后读取终态，不再修改库存。

新预留也先创建并锁定请求身份再更新库存，三个修改流程保持相同顺序，减少反向锁顺序导致的死锁机会。但相同顺序不能保证永远无死锁，其他索引和维护操作仍可能形成竞争；有界事务重试仍是正常设计的一部分。

10.4.3 到期取消怎样参与竞争

本例可以增加一个到期扫描任务，但“时钟到了”本身不修改库存。任务找出候选预留，再执行与用户取消相同的受控状态转移。如果确认已经完成，取消读取到终态便停止；如果取消先完成，迟到确认被拒绝，由上层业务处理后续行为。

候选扫描可以使用数据库时间和明确的到期字段，索引帮助减少扫描；扫描结果只是候选，真正修改前仍要重新锁定并检查状态与到期条件。否则扫描到某条记录后，另一请求已经改变状态，旧候选就可能造成重复释放。

本章核心表没有加入付款与到期字段，因为没有假定库存服务决定付款是否成功。若业务要求“已经扣款就不能超时取消”，确认消息可能迟到的问题就进入跨系统协议，必须补上支付结果查询、补偿或仲裁规则。不能只在表里加一个过期时间，就宣称完成了整个交易闭环。

10.5 事件与缓存放在事务之外，关系留在事务之内

10.5.1 发布者为什么允许重复

outbox 行与预留结果一起提交以后，发布者扫描未发送事件，将其写入 Kafka，取得规定的生产者确认后再标记 `sent_at`。本例先采用一个受控的发布工作进程，避免在基本说明中引入多工作进程抢占与租约协议；工作进程故障后可以重启继续扫描。

发送成功与标记已发送之间仍有窗口。如果 Kafka 已接受事件，发布器在更新 `sent_at` 前崩溃，重启后会再次发送。反过来，若先标记再发送，就可能在崩溃后永久跳过尚未发布的事件。因此本例选择允许重复，并让消费者用事件身份处理重复。

Kafka 生产者幂等有助于处理其协议范围内的重试，但不能把发布器重启后从 outbox 重新提交同一业务事件自动合并成一次业务发布。`event_id` 与消费者的结果记录仍然需要保留。成功写 Kafka 也不表示下游数据库已经提交，发布器只完成了自己的阶段。R10-06: Kafka 生产者幂等配置

未发送事件不能仅凭创建时间过旧就删除。它记录尚未完成的发布任务；清理前要满足已确认发布和规定审计保留期。发布器持续失败会使 outbox 增长，容量告警、发布延迟和故障处理因此成为系统运行要求。事务性 outbox 消除了业务提交与记录发布意图之间的空缺，没有消除消息系统依赖。

10.5.2 不依赖消息到达顺序重建状态

事件携带请求标识、SKU、数量、完整目标状态和 `state_version`。一个请求的 `RESERVED` 为版本1，终态为版本2。消费者维护各请求已应用的版本，只有更大版本到达时才更新该请求的派生状态；重复或更旧版本可以识别后忽略。

这样即使版本2先到，也能够把派生记录直接设为完整终态，后到的版本1不会把它改回已预留。前提是事件携带足够的完整状态，消费者更新不依赖“先减一次、再加一次”的增量运算。若业务需要严格按每一步产生副作用，就不能用简单版本覆盖替代事件顺序处理。

本例不让消费者通过所有事件的增减自行决定权威库存。报表可以维护每份预留的派生状态，再汇总；展示缓存可以失效后从数据库重载。消息中的顺序与重复由此影响展示新鲜度和处理效率，而不是库存成功的判定。

版本也不是跨所有请求的全局时钟。两个请求都具有版本2，不表示同时发生，也不能比较先后。该整数只在同一 `request_id` 内有意义。这与第9章的 GTID、epoch 讨论相通：可比较范围必须跟着标识一起保存。

10.5.3 消费者把去重与输出一起提交

如果消费者将结果写入另一个关系数据库，可以在同一个目标库事务中保存已处理 `event_id` 和派生状态更新。唯一键阻止同一事件重复修改输出，版本检查阻止旧状态覆盖新状态。只有目标库事务成功，消费者才推进相应 Kafka offset；遇到崩溃重读，去重记录帮助识别已经处理的内容。

这里有一个必要细节：提交位置只能覆盖该分区已交付记录中处理完成的前缀。这里不要求 offset 的整数连续；compaction 或事务过滤可能使可见记录之间存在空洞。若并行处理时后一条先完成，不能直接提交越过前面尚未处理完的已交付记录，否则重启后可能跳过它。本例消费者按分区顺序完成数据库处理，以减少这个额外的协调问题。

数据库提交后、offset提交前崩溃，事件会重复读取，但输出事务已经保存了事件身份。offset提交失败不能通过删除去重记录解决；应保留已提交业务结果并允许再次读取。反过来，去重记录若提前提交而输出失败，就会制造“看似已处理、结果却没有”的空缺，因此二者必须在同一目标库事务。

这是一组条件下的业务幂等处理，不是对短信、邮件或任意第三方接口的全局 exactly-once 承诺。外部副作用需要对方提供幂等标识或可查询结果，否则仍会存在未知窗口。Kafka 的自动提交配置和客户端提交接口只管理消费位置，不替应用完成这项协调。R10-07：消费者进度提交

10.5.4 Redis 丢了以后恢复什么

Redis 保存商品展示与近似库存，不保存本例唯一的预留结果。实例丢失后，可以限流回源并逐步重建缓存；在重建期间，预留结果仍由数据库事务判定，展示接口则可以暂时返回简化内容或降低请求量。

本例给展示记录设置5秒使用期限，这个数值是示例策略。期限从发起权威数据库读取时计算，查询耗时不能通过“完成后重新计时”被排除。缓存记录带上这一时点，展示服务使用统一时间基准，并将时钟误差保守地计入期限；时间基准无法确认时，不继续把记录当作有效库存。到期后若回源失败，接口返回库存状态未知，不继续报告旧数量。缓存缺失也遵循这个分支。

缓存失效不保证下一次读到的就一定是查询瞬间的最新提交。并发回源可能取到不同时间的结果并反向覆盖，因此本例从权威写入端用新的读取事务取得结果，写缓存时保留原读取时点，不让晚到的旧查询重新获得5秒期限。这个策略限制展示记录的使用时间，正常更新延迟仍需监控；它不是跨数据库、缓存和客户端的线性一致性承诺。更严格的新鲜度需要额外的版本检查或读取协议。

缓存中“已售罄”也可能因取消释放而过期。本例不据此永久拒绝预留，避免把一个派生旧值提升成权威决定。如果未来为了入口压力允许缓存直接拒绝部分请求，应把可能的误拒作为新的业务代价，经明确选择后再实现。

10.6 把每个失败点走一遍

10.6.1 成功路径不够证明方案

表 10-2 预留与发布的失败分支

失败时点	可能已经发生的事	下一步行为	保持的关系
请求未到服务端	数据库没有相应事务	同标识重试	不因网络重发形成新意图
请求行插入后，事务未提交	只有未提交变化	回滚；重试仍用原标识	无独立提交的PENDING
库存已更新，事件插入失败	同事务内部分语句成功	整体回滚	库存、结果、事件一起生效
COMMIT已成功，响应丢失	预留与事件已提交	查询或同标识重试	只返回已有结果，不再次扣减
COMMIT连接中断，结果未知	可能提交，也可能回滚	权威端查询，同标识协调	不猜测失败，不换新标识
Kafka暂时不可用	预留已提交，事件待发送	继续保留outbox并重试发布	消息故障不回滚已确认库存
事件发出后发布者崩溃	Kafka可能已有消息，sent_at未更新	重发同event_id	消费者识别业务重复
消费输出已提交，offset未提交	派生结果与去重记录已在	允许重读，按event_id去重	不提前跨越未处理进度
Redis全部丢失或展示记录到期	展示缓存不可用或超出使用期限	限流回源；失败则显示库存未知	权威预留仍从数据库读取，不继续报告过期数量
确认与取消同时到达	同一预留存在竞争	按锁定后的状态仅允许一条转移	不重复释放或同时成交取消

表里的“结果未知”不是容错失败，而是分布式请求必须表达的一种现实。服务端收到超时不应自动释放库存，因为原事务可能已经成功；客户端也不应无条件再减一次。查询、稳定请求标识和可重入状态转移共同使未知结果可以收敛。

10.6.2 数据库故障时，谁可以重新开放入口

应用事务正确的前提是受控的写入历史。若数据库主节点失联，本例停止新的预留写入，避免应用自行轮询多个地址寻找“能写的那一个”。服务发现返回一个可连接实例，不证明它具有此前确认的全部事务。

恢复原写端或切换前，需要确认旧写端不能继续接收有效业务写入，并检查恢复后历史与已确认结果的关系。采用异步复制时，落后副本可能缺少已经返回成功的事务；MySQL本地双1不能消除这个复制窗口。若无法证明候选具有要求的历史，就继续暂停并使用受控恢复流程，而不是把数据缺失当作正常回滚。

本案例接受这种暂停，换取不盲目扩大写入历史。若新的需求是主机故障时仍须立即自动恢复且不丢已确认结果，就需要选择并验证相应高可用协议、故障检测与隔离安排。这可能增加节点、网络 and 运维成本，属于条件变化后的重新设计。

数据库本地持久化也需要实测和运行保障。`innodb_flush_log_at_trx_commit=1` 与 `sync_binlog=1` 表达日志同步策略，依赖存储正确兑现同步请求；它们不能防止全部存储损毁、误删或恶意修改。备份恢复应与复制分开演练，恢复点和恢复时间分别记录。R10-08：InnoDB 与事务持久性、R10-09：binlog同步

10.6.3 从观测判断问题位于哪一层

入口延迟变高时，至少区分等待连接、等待行锁、SQL执行、日志提交和应用处理。若热点 SKU 的行锁等待占主要部分，增加无关索引或扩大 Kafka 批大小不会改善这个瓶颈。若提交阶段占比高，比较存储和日志同步才有依据。

事件延迟则拆成数据库提交到发布、发布到 Kafka 确认、消息到达消费者、消费者输出提交。把总延迟只标为“Kafka慢”，会掩盖 outbox 扫描停滞或目标库锁等待。每个阶段的成功计数与失败计数还要能以请求或事件身份关联。

数量检查定期验证 `total=available+reserved+sold`，并把 `reserved`、`sold` 与预留记录在同一一致性视图下的汇总对照。不同事务时刻分别读取两张表，可能出现检查过程自身制造的短时不一致；对账需要统一快照或适当暂停条件。

对账发现异常不能直接以展示缓存覆盖数据库。应保留异常证据，确定涉及哪些请求、哪条维护操作和哪段历史，再决定修复。把任何一张表称为正本，不意味着它永远不会因程序缺陷或人工操作而错误。

10.7 验证容量，再决定是否增加组件

10.7.1 性能实验必须保留热点

本案例的入口目标为每秒2000次，其中最热SKU占40%。测试如果把请求均匀分散到十万种商品，就回避了最难的行锁竞争。至少要分别测试均匀分布、设定热点，以及极端集中到一个SKU的情况，并观察每种条件下的吞吐与尾延迟。

数据也要接近运行形态：请求表已经有历史记录、outbox有一定待发量、索引存在、日志持久化与生产一致。空表、关闭刷盘、仅执行库存UPDATE的基准，不能代表包含幂等与事件记录的完整事务。

度量还要区分业务成功、库存不足拒绝、系统错误和客户端超时。库存耗尽后，大量请求走快速拒绝分支，可能使表面吞吐升高，却没有说明正常预留能力改善。性能测试应控制库存补充或分组重置，明确各分支比例，重置属于测试管理而非本章生产事务。

10.7.2 热点瓶颈出现后的选择

若基线达不到目标，先缩短持锁事务：移出远程调用，删除不必要索引写入，限制单请求数量和事务工作，给连接池设置合理并发。并发连接更多并不代表同一库存行能同时被更多事务修改；排队过长还会放大超时与重试。

可以在入口按SKU限流，把超出处理能力的请求明确拒绝或排队。它不增加库存行的实际更新能力，但能避免系统用无限等待掩盖容量不足。此时响应协议也要区分库存不足与服务繁忙，前者是业务结果，后者通常允许同一请求标识稍后重试。

进一步按SKU分区可以把不同商品放到不同写入单元，提高跨SKU的并行能力，却不能自动解决一个SKU特别热的问题。把同一SKU的库存拆成多个额度桶，可以增加并行性，但要分配额度、防止跨桶重复，并处理某桶售罄而其他桶尚有剩余的协调。它改变数据模型，不是简单增加数据库分片。

如果测量表明需要把更多请求转为排队处理，可以重新比较Kafka方案：按SKU组织处理，入口返回请求已受理，最终预留结果另行查询。本例已允许处理中状态，但常态排队仍需约定最终结果的等待目标与积压上限。选择它的理由应是这些安排能满足已测出的容量需要，而不是“Kafka吞吐高，所以库存一定适合放Kafka”。

Redis前置预扣也可以重新进入候选表，但必须量化过滤比例，并完整设计回补与恢复。前置层占用成功后数据库失败，若只设置TTL等待自动释放，重复释放、迟到成功和重新加载时的未决占用仍需处理。多一个高速组件不能替代这些状态关系。

10.7.3 一个具体的验收集合

下面列出的检查是本案例的验收设计，不宣称已经在作者项目或生产环境执行。它们可以在隔离环境逐步自动化，并保存版本、配置、输入种子与观测记录。

检查	输入与故障安排	通过条件
不超额预留	总量有限，多客户端同时请求，需求合计超过库存	成功预留总量不超过总量；数量关系成立
相同请求重发	同ID同内容并发重发，包含响应丢失	只有一份预留结果与一次有效扣减
标识错误复用	同ID换用户、SKU或数量	拒绝冲突，不返回他人结果，不追加扣减
确认取消竞争	对同一预留并发发起两种操作	一个终态，数量仅转移一次
事务中断	在插入请求、更新库存、写事件后分别终止连接	未提交事务不留下部分业务结果
提交结果未知	在提交完成附近切断响应	同标识查询/重试最终收敛，无新ID重复扣减
发布重复	发送后、标记前停止发布者，再启动	允许重复消息，派生结果不重复累计
消费乱序与重复	版本2先于版本1，同事件多次到达	终态不被旧版本覆盖，输出与去重同事务
缓存故障与过期	清空或停止测试Redis；延迟权威读取并使展示记录到期	不改变权威预留；旧查询不重新获得完整期限；回源失败显示未知
写端失联	隔离数据库写入口并验证恢复流程	停止新预留；未知结果不被猜测为失败

这些检查通过，仍不意味着可以省去安全和操作验收。服务账号只能执行所需数据操作，发布者无需修改库存，消费者不应持有库存写权限。请求结果查询必须验证所有权；记录日志时要避免暴露认证信息和不必要的个人数据。第6章的访问边界在这里对应具体账号与接口，而不是一张通用安全清单。

发布前还要验证生命周期。服务只有在数据库连接、必要配置和写端身份确认后才就绪；退出时先停止接收新写，再让在途事务结束或返回未知状态。进程被强制终止后，数据库事务恢复与客户端同标识重试共同继续处理，不依赖退出钩子一定执行。

10.8 回到架构判断

10.8.1 这次选择接受了什么代价

本案例选择数据库事务作为库存结果的确认点，接受热点行竞争与数据库故障时暂停新预留。选择outbox允许事件稍后发布，接受额外表空间、发布延迟和消费者去重。选择Redis只承担派生展示，接受缓存故障时展示降级与回源压力。

这些代价不是设计结束后的附注，它们构成方案本身。若业务不接受故障暂停，当前高可用安排就不够；若不接受展示陈旧，缓存更新与读取条件就要变；若每次购买必须跨多个SKU原子完成，单SKU事务也必须重做。条件变化足以改变方案，不必等到某个软件被证明“不先进”。

同样，数据库、Kafka与Redis不是按“可靠、吞吐、快”的固定角色被放进图中。数据库维护本例容易放在一个事务里的不变量，可延后传递的事件通过Kafka发送，Redis保存可以重新加载的展示数据。改变业务对象后，分工也可以改变。

10.8.2 观察的结果应当能够被质疑

前几章分析成熟系统，容易看到最终成形的实现，却看不到它曾经面对的所有候选。本章把候选和失败分支写出来，是为了让判断能够被检查：如果热点分布不同，数据库基线是否还成立；如果事件消费者不支持幂等，outbox的重复该由谁承担；如果副本落后，谁有权恢复写入。

我在设计评审里更愿意留下这样的记录：当前约束是什么，选择省掉了哪一步，新增了哪些状态，什么证据会使我改判。它未必比一张漂亮的产品架构图简短，但后来发生故障或业务变化时，团队知道需要重新检查哪一个前提。

一本观察笔记无法替读者确定生产环境的全部条件。它能提供的是有边界的机制解释，以及把机制转成判断的过程。Redis的复制身份、InnoDB的页与事务、Kafka的批和历史位置，都不是待背诵的标签；它们说明系统怎样保存必要证据，怎样在限制之内继续工作。

本章的设计也保留了未覆盖范围：支付与履约、跨SKU事务、多地域无损切换，以及任意第三方副作用的原子完成。明确范围后，范围之内的行为仍要经过测试和运行验证。到这里，选择已经足够具体：有可实现的数据关系，有失败后的动作，有能够推翻当前选择的指标。后续优化可以围绕这些证据推进，而不必再次从产品印象开始。

后记

写这本书时，我越来越少把一个软件的特点当成最后的答案。Redis的响应快，仍需问这次成功确认了什么；MySQL有事务，仍需问事务之外的缓存与消息怎么办；Kafka保留日志，仍需问消费进度和外部结果能否一起恢复。同一个词用在不同层次上，含义可能已经变了。

刚工作时，我关心命令、索引和参数，能让系统正常运行，就认为问题已经解决。后来开始负责设计，选型会上要解释理由，事故后要回看原先的假设。很多问题没有发生在某个组件内部，而是发生在几个各自合理的选择之间：对下一层的预期是已经完成，实际保证却只是收到；失败被理解为没有生效，实际操作却已经提交。

库存事故是其中一次。网络分区后，系统自动恢复了一个可服务的主节点，却没有保住我们以为已经确定的全部结果。之后改用MySQL记录库存，Redis承担前置处理，是当时作出的调整。这个经历能说明原先的假设有问题，却不能单独证明任何“MySQL加Redis”的组合都正确。第10章另设完整推演，把事务、重试、状态转移和消息传播展开，正是为了不让故事替代设计。

书中仍然保留了不少实现细节。我不想把观察方法写成几句不依赖对象的原则。没有RDB与AOF怎样衔接，便难以讨论重建；没有页、redo和双写的不同职责，便难以讨论哪份冗余可以撤掉；没有replid、GTID与leader epoch，便容易把所有恢复都说成从某个数字继续。细节只有参与解释时才值得留下，但解释也需要足够细节才能成立。

另一方面，成熟软件的实现不会为我们的业务条件提供现成答案。本书没有给三个产品排出一个总名次，也没有把十个主题收成一张自动选型表。一个方案是否适合，仍然要回到数据关系、失败后允许的行为、资源和团队能够承担的工作。第10章的选择因此只是有条件的起点，输入改变，结论也应该重新检查。

我希望这本笔记保留一种可以回看的判断方式：当时为什么如此选择，哪些证据支持它，后来哪一项条件变了。这样的记录不一定让每次决定都正确，但有助于在错误发生之后找到需要修正的地方，而不是只把责任归给一个产品、一个参数或一句已经过时的经验。

致谢与本版说明

感谢Redis、MySQL与Apache Kafka社区的作者、维护者和贡献者。公开的实现、文档、问题讨论与改进提案，使这些设计可以被具体分析，而不仅是使用体验。本书的参考文献尽量指向与命题对应的版本和章节；文档仍会演进，涉及部署时应核对实际使用版本。

本书的初稿与多轮修订使用了生成式AI工具。本数字稿在既有稿件基础上由AI协助重写，保留已有作者经历，并将新增设计示例明确标为设定的推演。AI参与不等于事实已经自动可靠，文中技术依据、故障假设和未覆盖范围仍需接受核对与勘误。书中的假设数字不是生产实测，示例方案也没有被表述为作者当年的完整实施记录。

本版为2026年9月5日重写数字稿，文件版本标识为 20260905gpt5.6。它没有新增ISBN、出版社、印次或未经确认的授权信息。原稿与此前分析报告另行保留，便于比较本次对结构、事实与表达的调整。

关于作者

作者是软件架构师，参与过IoT、电商订单和SaaS系统，从一线开发工作到首席架构师。书中的既有案例来自这些年的设计、故障排查和复盘；本版没有为案例追加未经记录的项目规模、人物对话或实施结果。

勘误

本书使用Redis、MySQL、Apache Kafka等名称进行技术讨论，不表示相关项目或商标权属方对内容的认可。具体软件的许可与使用条件，应查阅相应版本的官方文件。

勘误记录包含章节、使用版本、复现条件和对应资料，有助于下一版判断是事实错误、边界遗漏，还是文字没有把关系解释清楚。

参考文献与版本资料

本版以 Redis 7.x、MySQL 8.0.x、Apache Kafka 3.9.0 为机制讨论基线。涉及小版本新增功能或精确实现的地方，正文另行限定；MySQL 5.7 Query Cache 等资料仅用于历史对照。

以下按章节列出正文实际引用的资料。相同资料可在不同章节出现，所支撑的是相邻机制或具体限定，不表示整章全部论断均由该资料证明。正文的 R5-01 等短标识与本表对应；前四章使用内容标题链接。本版资料整理与引用日期为 **2026-09-05**，未给出无法确认的网页发布日期。

在线手册可能随维护版本更新。Redis 源码链接分别固定到 7.2 分支或 7.2.5 标签，分支链接仍可能变化；Kafka 恢复实现链接固定到 3.9.0 标签，3.9 系列 Javadoc 则可能显示该系列后续补丁版。涉及复现实验时，软件构建版本、配置与故障条件仍需一并记录。

本书中的 SQL、命令和验证方案是机制说明及设计示例。本版未以运行真实 Redis、MySQL、Kafka 服务或集群故障演练来认证这些示例；第 10 章明确区分设定的目标、待执行的验收与已完成的推理。

第 1 章 引言：为什么是这三个软件

- **R1-01** Redis 项目。Redis 数据类型说明。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R1-02** Oracle。MySQL8.0一致性非锁定读。MySQL 8.0。[引用日期：2026-09-05]。
- **R1-03** Oracle。MySQL8.0多版本机制。MySQL 8.0。[引用日期：2026-09-05]。
- **R1-04** Apache Kafka 项目。3.9设计文档。Kafka 3.9 文档。[引用日期：2026-09-05]。

第 2 章 数据结构与协议：为各自的目标而设计

- **R2-01** Redis 项目。SDS实现。Redis 7.2.5 源码。[引用日期：2026-09-05]。
- **R2-02** Redis 项目。字典实现。Redis 7.2.5 源码。[引用日期：2026-09-05]。
- **R2-03** Redis 项目。Redis7.2.5集合实现。Redis 7.2.5 源码。[引用日期：2026-09-05]。
- **R2-04** Oracle。MySQL8.0行格式说明。MySQL 8.0。[引用日期：2026-09-05]。
- **R2-05** Oracle。自适应哈希索引说明。MySQL 8.0。[引用日期：2026-09-05]。
- **R2-06** Apache Kafka 项目。Kafka3.9消息格式。Kafka 3.9 文档。[引用日期：2026-09-05]。
- **R2-07** Redis 项目。RESP规范。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R2-08** Oracle。文本结果集。MySQL 协议开发资料。[引用日期：2026-09-05]。
- **R2-09** Oracle。二进制结果集。MySQL 协议开发资料。[引用日期：2026-09-05]。
- **R2-10** Apache Kafka 项目。Kafka3.9协议。Kafka 3.9 文档。[引用日期：2026-09-05]。

第 3 章 生命周期管理：优雅启动与关闭

- **R3-01** Redis 项目. SHUTDOWN文档. Redis 7.x 讨论基线, 在线文档. [引用日期: 2026-09-05]。
- **R3-02** Oracle. MySQL8.0选项文件. MySQL 8.0. [引用日期: 2026-09-05]。
- **R3-03** Oracle. InnoDB恢复说明. MySQL 8.0. [引用日期: 2026-09-05]。
- **R3-04** Oracle. innodb_fast_shutdown 参数定义. MySQL 8.0. [引用日期: 2026-09-05]。
- **R3-05** Apache Kafka 项目. Kafka3.9.0 BrokerLifecycleManager. Kafka 3.9.0 源码. [引用日期: 2026-09-05]。
- **R3-06** Kubernetes 项目. 探针定义. 在线文档; 行为范围见正文. [引用日期: 2026-09-05]。
- **R3-07** Kubernetes 项目. 容器生命周期钩子. 在线文档; 行为范围见正文. [引用日期: 2026-09-05]。

第 4 章 内存与磁盘：速度与持久化的平衡

- **R4-01** Redis 项目. Redis淘汰说明. Redis 7.x 讨论基线, 在线文档. [引用日期: 2026-09-05]。
- **R4-02** Redis 项目. Redis持久化说明. Redis 7.x 讨论基线, 在线文档. [引用日期: 2026-09-05]。
- **R4-03** Redis 项目. WAITAOF定义. Redis 7.2 起. [引用日期: 2026-09-05]。
- **R4-04** Oracle. 缓冲池抗扫描说明. MySQL 8.0; 小版本限制见正文. [引用日期: 2026-09-05]。
- **R4-05** Oracle. redo提交策略. MySQL 8.0; 小版本限制见正文. [引用日期: 2026-09-05]。
- **R4-06** Oracle. binlog同步选项. MySQL 8.0; 小版本限制见正文. [引用日期: 2026-09-05]。
- **R4-07** Oracle. doublewrite说明. MySQL 8.0; 小版本限制见正文. [引用日期: 2026-09-05]。
- **R4-08** Oracle. change buffer说明. MySQL 8.0; 小版本限制见正文. [引用日期: 2026-09-05]。
- **R4-09** Linux 内核项目. Linux虚拟内存参数. 内核虚拟内存参数在线文档. [引用日期: 2026-09-05]。
- **R4-10** Apache Kafka 项目. 生产者配置. Kafka 3.9. [引用日期: 2026-09-05]。

第 5 章 分层架构设计 — 存储层 / 逻辑层 / 交互层

- **R5-01** Redis 项目. Redis 客户端处理. Redis 7.x 讨论基线, 在线文档. [引用日期: 2026-09-05]。
- **R5-02** Redis 项目. UNLINK 命令. Redis 7.x 讨论基线, 在线文档. [引用日期: 2026-09-05]。
- **R5-03** Oracle. 可插拔存储引擎概述. MySQL 8.0. [引用日期: 2026-09-05]。
- **R5-04** Oracle. Query Cache 的工作方式 (MySQL 5.7 历史资料). MySQL 5.7 (历史). [引用日期: 2026-09-05]。
- **R5-05** Oracle. MySQL 8.0 新增及移除的功能. MySQL 8.0. [引用日期: 2026-09-05]。
- **R5-06** Apache Kafka 项目. Kafka 协议. Kafka 3.9 文档. [引用日期: 2026-09-05]。

- **R5-07** Apache Kafka 项目。Kafka 设计。Kafka 3.9 文档。[引用日期：2026-09-05]。
- **R5-08** Apache Kafka 项目。Kafka 分层存储。Kafka 3.9 文档。[引用日期：2026-09-05]。

第 6 章 安全机制 — 权限、加密、审计

- **R6-01** Redis 项目。Redis 访问控制列表。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R6-02** Redis 项目。ACL DRYRUN 命令。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R6-03** Oracle。Caching SHA-2 认证插件。MySQL 8.0。[引用日期：2026-09-05]。
- **R6-04** Apache Kafka 项目。使用 SASL 认证。Kafka 3.9 文档。[引用日期：2026-09-05]。
- **R6-05** Redis 项目。Redis 安全机制。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R6-06** Oracle。访问控制：请求验证。MySQL 8.0。[引用日期：2026-09-05]。
- **R6-07** Oracle。部分撤销权限。MySQL 8.0。[引用日期：2026-09-05]。
- **R6-08** Apache Kafka 项目。授权与 ACL。Kafka 3.9 文档。[引用日期：2026-09-05]。
- **R6-09** Redis 项目。Redis TLS。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R6-10** Oracle。InnoDB 静态数据加密。MySQL 8.0。[引用日期：2026-09-05]。

第 7 章 集群架构 — 从单点到分布式

- **R7-01** Seth Gilbert、Nancy Lynch。Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services。ACM SIGACT News, 2002, 33(2): 51–59。[引用日期：2026-09-05]。
- **R7-02** Redis 项目。WAIT 命令。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R7-03** Redis 项目。Redis Sentinel。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R7-04** Redis 项目。Redis Cluster 规范。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R7-05** Oracle。半同步复制。MySQL 8.0。[引用日期：2026-09-05]。
- **R7-06** Oracle。Group Replication。MySQL 8.0。[引用日期：2026-09-05]。
- **R7-07** Oracle。组复制一致性保证。MySQL 8.0。[引用日期：2026-09-05]。
- **R7-08** Apache Kafka 项目。生产者配置。Kafka 3.9 文档。[引用日期：2026-09-05]。
- **R7-09** Apache Kafka 项目。主题配置。Kafka 3.9 文档。[引用日期：2026-09-05]。
- **R7-10** Apache Kafka 项目。KRaft 运维。Kafka 3.9 文档。[引用日期：2026-09-05]。

第 8 章 磁盘存储格式：数据写下以后

- **R8-01** Redis 项目。Redis 7.2 RDB 类型及操作码。Redis 7.2 分支源码。[引用日期：2026-09-05]。
- **R8-02** Redis 项目。RDB 保存与加载实现。Redis 7.2 分支源码。[引用日期：2026-09-05]。
- **R8-03** Redis 项目。RESP 规范。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R8-04** Redis 项目。Redis 持久化与多部分 AOF。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R8-05** Redis 项目。Redis 7.2 AOF 清单及重写实现。Redis 7.2 分支源码。[引用日期：2026-09-05]。
- **R8-06** Oracle。InnoDB 页大小配置。MySQL 8.0。[引用日期：2026-09-05]。
- **R8-07** Oracle。InnoDB 文件空间管理。MySQL 8.0。[引用日期：2026-09-05]。
- **R8-08** Oracle。InnoDB 行格式。MySQL 8.0。[引用日期：2026-09-05]。
- **R8-09** Oracle。InnoDB 双写缓冲与文件。MySQL 8.0。[引用日期：2026-09-05]。
- **R8-10** Oracle。InnoDB redo log。MySQL 8.0。[引用日期：2026-09-05]。
- **R8-11** Apache Kafka 项目。Kafka 3.9 设计与日志管理。Kafka 3.9 文档。[引用日期：2026-09-05]。
- **R8-12** Apache Kafka 项目。Kafka 记录格式。Kafka 3.9 文档。[引用日期：2026-09-05]。
- **R8-13** Apache Kafka 项目。Kafka 3.9.0 OffsetIndex。Kafka 3.9.0 源码。[引用日期：2026-09-05]。
- **R8-14** Apache Kafka 项目。Kafka 3.9.0 TimeIndex。Kafka 3.9.0 源码。[引用日期：2026-09-05]。
- **R8-15** Apache Kafka 项目。KIP-405 分层存储设计。设计提案；采用范围见正文。[引用日期：2026-09-05]。
- **R8-16** Apache Kafka 项目。Kafka 3.9.0 TierStateMachine。Kafka 3.9.0 源码。[引用日期：2026-09-05]。

第 9 章 数据同步：确认、历史与续传

- **R9-01** Redis 项目。Redis 复制机制。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R9-02** Redis 项目。Redis 脚本复制说明。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R9-03** Redis 项目。Redis 7.2 复制实现。Redis 7.2 分支源码。[引用日期：2026-09-05]。
- **R9-04** Redis 项目。WAIT 的范围与限制。Redis 7.x 讨论基线，在线文档。[引用日期：2026-09-05]。
- **R9-05** Redis 项目。WAITAOF。Redis 7.2 起。[引用日期：2026-09-05]。
- **R9-06** Oracle。binlog 格式。MySQL 8.0。[引用日期：2026-09-05]。

- **R9-07** Oracle. 复制线程. MySQL 8.0. [引用日期: 2026-09-05]。
- **R9-08** Oracle. 多线程副本配置. MySQL 8.0. [引用日期: 2026-09-05]。
- **R9-09** Oracle. GTID 格式与存储. MySQL 8.0. [引用日期: 2026-09-05]。
- **R9-10** Oracle. GTID 自动定位. MySQL 8.0. [引用日期: 2026-09-05]。
- **R9-11** Oracle. 半同步复制. MySQL 8.0. [引用日期: 2026-09-05]。
- **R9-12** Oracle. 半同步等待点与可见性. MySQL 8.0. [引用日期: 2026-09-05]。
- **R9-13** Apache Kafka 项目. 生产者 acks. Kafka 3.9 文档. [引用日期: 2026-09-05]。
- **R9-14** Apache Kafka 项目. 最小 ISR 配置. Kafka 3.9 文档. [引用日期: 2026-09-05]。
- **R9-15** Apache Kafka 项目. KIP-101 的恢复问题与方案. 设计提案; 采用范围见正文. [引用日期: 2026-09-05]。
- **R9-16** Apache Kafka 项目. 3.9.0 TierStateMachine 与恢复条件. Kafka 3.9.0 源码. [引用日期: 2026-09-05]。
- **R9-17** Apache Kafka 项目. KafkaConsumer 的位置与提交说明. Kafka 3.9 系列 API 文档. [引用日期: 2026-09-05]。

第 10 章 把观察用于一次完整设计

- **R10-01** Oracle. CHECK 约束. MySQL 8.0. [引用日期: 2026-09-05]。
- **R10-02** Amazon Web Services. 事务性 outbox. AWS Prescriptive Guidance. [引用日期: 2026-09-05]。
- **R10-03** Oracle. UPDATE 与锁. MySQL 8.0. [引用日期: 2026-09-05]。
- **R10-04** Oracle. 事务提交与回滚. MySQL 8.0. [引用日期: 2026-09-05]。
- **R10-10** Oracle. 死锁处理. MySQL 8.0. [引用日期: 2026-09-05]。
- **R10-05** Oracle. InnoDB 锁定读取. MySQL 8.0. [引用日期: 2026-09-05]。
- **R10-06** Apache Kafka 项目. Kafka 生产者幂等配置. Kafka 3.9 文档. [引用日期: 2026-09-05]。
- **R10-07** Apache Kafka 项目. 消费者进度提交. Kafka 3.9 系列 API 文档. [引用日期: 2026-09-05]。
- **R10-08** Oracle. InnoDB 与事务持久性. MySQL 8.0. [引用日期: 2026-09-05]。
- **R10-09** Oracle. binlog 同步. MySQL 8.0. [引用日期: 2026-09-05]。

引用与示例的使用范围

本文的产品名称和相关标识归其各自权利人所有。引用官方文档、源代码页面和原始论文用于说明技术机制，不表示这些项目或机构对本书作出认可。正文图表为本稿已有插图的整理与重绘；本版不对第三方资料重新授予许可。

第 10 章库存系统是为展开设计判断而构造的示例，事务表结构、请求状态及目标负载均已在正文声明为设定。书中保留的作者亲历与该示例分别叙述，参考文献也不被用来证明作者经历或示例的实测结果。